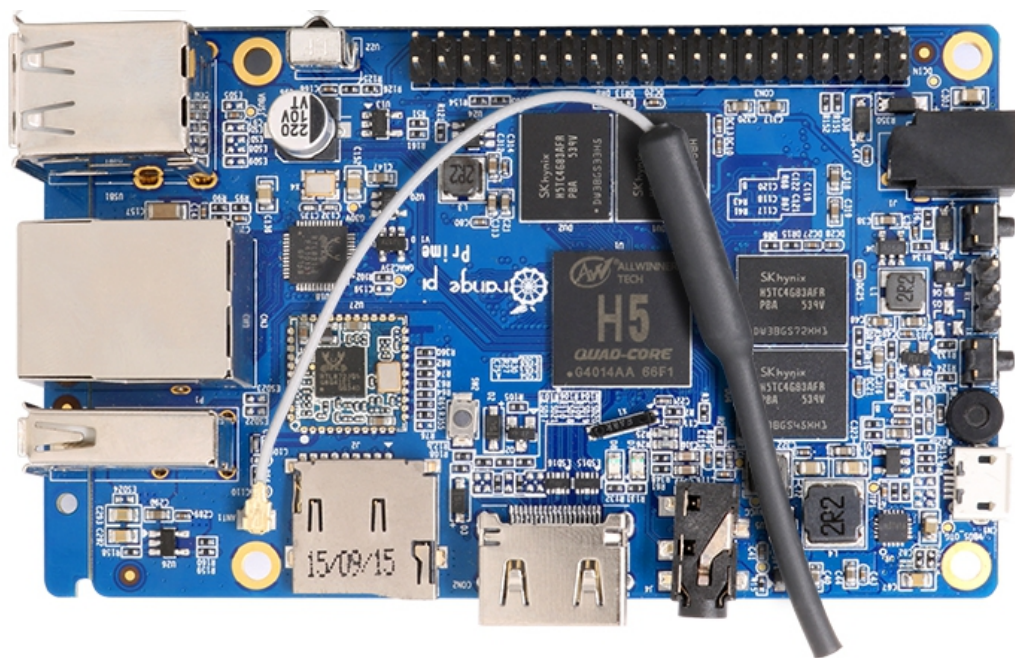




Orange Pi Prime 用户手册





目录

1. Orange Pi Prime 的基本特性.....	1
1.1. 什么是 Orange Pi Prime.....	1
1.2. Orange Pi Prime 的用途.....	1
1.3. Orange Pi Prime 是为谁设计的.....	1
1.4. Orange Pi Prime 的硬件特性.....	2
1.5. Orange Pi Prime 的顶层视图和底层视图.....	3
1.6. Orange Pi Prime 的接口详情图.....	4
2. 开发板使用介绍.....	5
2.1. 准备需要的配件.....	5
2.2. 下载开发板的镜像和相关的资料.....	7
2.3. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法.....	8
2.4. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法.....	10
2.5. 烧写 Android 固件到 TF 卡的方法.....	13
2.6. 启动香橙派开发板.....	16
2.7. 调试串口的使用方法.....	16
2.7.1. 调试串口的连接说明.....	16
2.7.2. Ubuntu 平台调试串口的使用方法.....	17
2.7.3. Windows 平台调试串口的使用方法.....	20
3. Linux 系统使用说明.....	24
3.1. 已支持的 linux 发行版类型和内核版本.....	24
3.2. Linux3.10 内核镜像驱动适配情况.....	24
3.3. Linux5.4 内核镜像驱动适配情况.....	25
3.4. 登录账号和密码.....	25
3.5. 板载 LED 灯测试说明.....	26
3.6. Linux 桌面版系统自动登录说明.....	26



3. 7. Linux 系统第一次启动自动扩容 rootfs.....	27
3. 8. Linux 系统修改日志级别（loglevel）的方法.....	28
3. 9. HDMI 显示测试.....	29
3. 10. 以太网口测试.....	30
3. 11. WIFI 连接测试.....	31
3. 11. 1. 服务器版镜像通过命令连接 WIFI.....	31
3. 11. 2. 服务器版镜像通过图形化方式连接 WIFI.....	33
3. 11. 3. 桌面版镜像的测试方法.....	36
3. 12. SSH 远程登录开发板.....	38
3. 12. 1. Ubuntu 下 SSH 远程登录开发板.....	38
3. 12. 2. Windows 下 SSH 远程登录开发板.....	39
3. 13. Linux3.10 系统 HDMI 分辨率设置.....	41
3. 14. Linux3.10 系统 Framebuffer 宽度和高度的修改方法.....	42
3. 15. 蓝牙测试.....	42
3. 15. 1. 桌面版镜像的测试方法.....	42
3. 15. 2. 服务器版镜像的测试方法.....	46
3. 16. USB 接口测试.....	47
3. 16. 1. 连接鼠标或键盘测试.....	47
3. 16. 2. 连接 USB 存储设备测试.....	48
3. 17. USB 以太网卡测试.....	48
3. 18. USB 摄像头测试.....	50
3. 19. 音频测试.....	51
3. 19. 1. 耳机接口播放音频测试.....	51
3. 19. 2. HDMI 音频播放测试.....	52
3. 19. 3. Mic 录音测试.....	52
3. 20. linux5.4 红外接收测试.....	52
3. 21. linux3.10 红外接收测试.....	54
3. 22. 硬件看门狗测试.....	54
3. 23. CSI 摄像头测试.....	56



3. 23. 1. CSI 摄像头接口规格.....	56
3. 23. 2. Linux3.10 系统 gc2035 摄像头测试.....	57
3. 23. 3. Linux3.10 系统 ov5640 摄像头测试.....	60
3. 23. 4. Linux5.4 系统 ov5640 摄像头测试.....	62
3. 24. 40pin 引脚说明.....	64
3. 25. 安装 wiringOP.....	65
3. 26. 40pin 的 GPIO、I2C、UART、SPI 测试.....	66
3. 26. 1. 普通 GPIO 口测试.....	66
3. 26. 2. SPI 接口测试.....	68
3. 26. 3. I2C 测试.....	70
3. 26. 4. UART 测试.....	72
3. 27. SPI LCD 显示屏使用方法.....	74
3. 27. 1. 2.4 寸 SPI LCD 显示屏.....	74
3. 27. 2. 3.2 寸 RPi SPI LCD 显示屏.....	78
3. 27. 3. 3.5 寸 SPI LCD 显示屏.....	82
4. Linux SDK 使用说明.....	85
4. 1. 获取 linux sdk 的源码.....	85
4. 1. 1. 从 github 下载 orangepi-build.....	85
4. 1. 2. 下载交叉编译工具链.....	86
4. 1. 3. orangepi-build 完整目录结构说明.....	87
4. 1. 4. 从百度云盘下载 orangepi-build.....	88
4. 2. 编译 u-boot.....	90
4. 3. 编译 linux 内核.....	94
4. 4. 编译 rootfs.....	99
4. 5. 编译 linux 镜像.....	102
5. Android 系统使用说明.....	105
5. 1. 已支持的 Android 版本.....	105
5. 2. Android 5.1 功能适配情况.....	105
5. 3. 板载 LED 灯显示说明.....	106
5. 4. ADB 使用说明.....	106



5.4.1. 打开 USB debugging 选项.....	106
5.4.2. 使用网络连接 adb 调试.....	107
5.4.3. 使用数据线连接 adb 调试.....	108
5.5. USB 摄像头的使用方法.....	109
5.6. CSI 摄像头接口规格.....	110
5.6.1. CSI 摄像头接口规格.....	110
5.6.2. Android 系统 gc2035 摄像头的使用方法.....	110
5.6.3. Android 系统 ov5640 摄像头的使用方法.....	112
6. Android SDK 使用说明.....	114
6.1. Android 5.1 SDK 使用说明.....	114
6.1.1. 下载 android 5.1 sdk 的源码.....	114
6.1.2. 搭建 android 编译环境.....	116
6.1.3. 编译 android 镜像.....	116



1. Orange Pi Prime 的基本特性

1.1. 什么是 Orange Pi Prime

香橙派是一款开源的单板卡片电脑,新一代的 arm64 开发板,它可以运行 Android 5.1、Ubuntu 和 Debian 等操作系统。香橙派开发板 (Orange Pi Prime) 使用全志 H5 系统级芯片,同时拥有 2GB DDR3 内存

1.2. Orange Pi Prime 的用途

我们可以用它搭建:

- 一台计算机
- 一个无线网络服务器
- 游戏机
- 音乐播放器
- 高清视频播放器
- 扬声器
- Android

当然还有其他更多的功能,因为 Orange Pi 是开源的

1.3. Orange Pi Prime 是为谁设计的

Orange Pi 开发板不仅仅是一款消费品,同时也是给任何想用技术来进行创作创新的人设计的。它是一款非常简单、有趣、实用的工具,你可以用它去打造你身边的世界



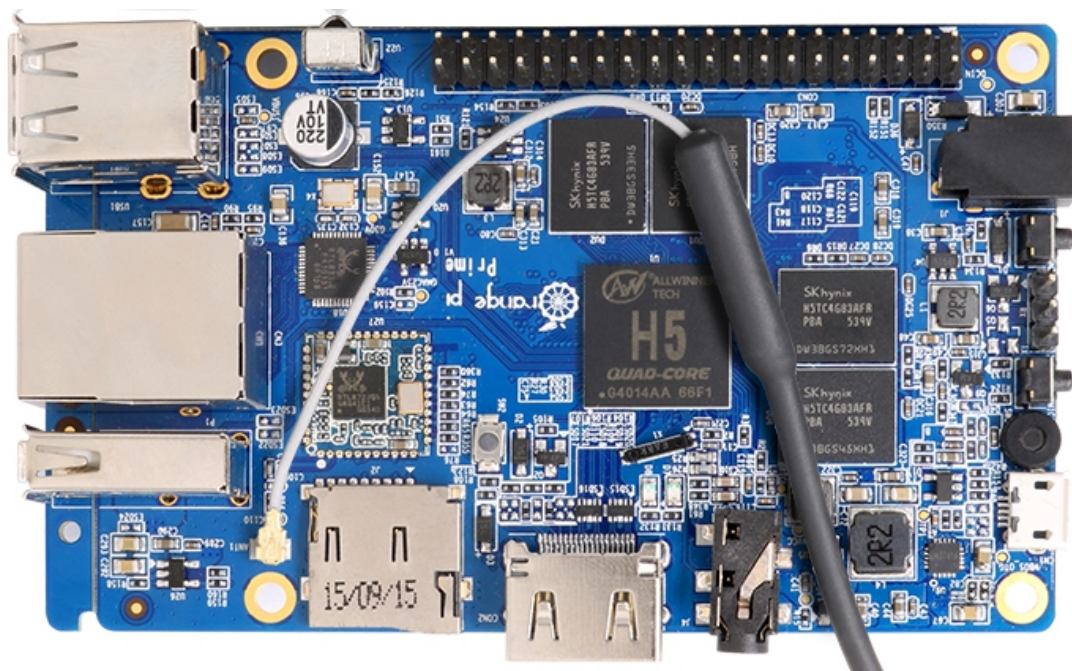
1. 4. Orange Pi Prime 的硬件特性

硬件特性介绍	
CPU	全志 H5 四核 64 位 Cortex-A53 处理器
GPU	六核 Mali450 GPU 支持 OpenGL ES 2.0 和 OpenVG1.1
内存	2GB DDR3 (与 GPU 共享)
板载存储	TF 卡插槽, 2MB SPI Flash(可选)
板载 WIFI+蓝牙	RTL8723BS, IEEE 802.11 b/g/n,BT4.0
板载网络	10/100/1000M 以太网
音频输入	MIC
视频输入	CSI 接口的摄像头
视频输出	HDMI 和 CVBS 输出
音频输出	HDMI 和 3.5 mm 音频口
电源	DC 输入, MicroUSB (OTG)不能用作电源输入
USB 2.0 端口	3 个 USB 2.0 HOST, 1 个 USB 2.0 OTG
40pin 接头	带有 I2C、SPI、UART 以及多个 GPIO 口
按键	Recovery 键(SW2), U-boot 键(SW3), 电源键 (SW4)
红外接收	支持红外遥控
调试串口	UART-TX,UART-RX,GND
LED 灯	电源指示灯和状态指示灯
支持的操作系统	Android,Ubuntu,Debian 等操作系统
外观规格介绍	
产品尺寸	98mm×60mm
重量	68g
 rangePi™是深圳市迅龙软件有限公司的注册商标	

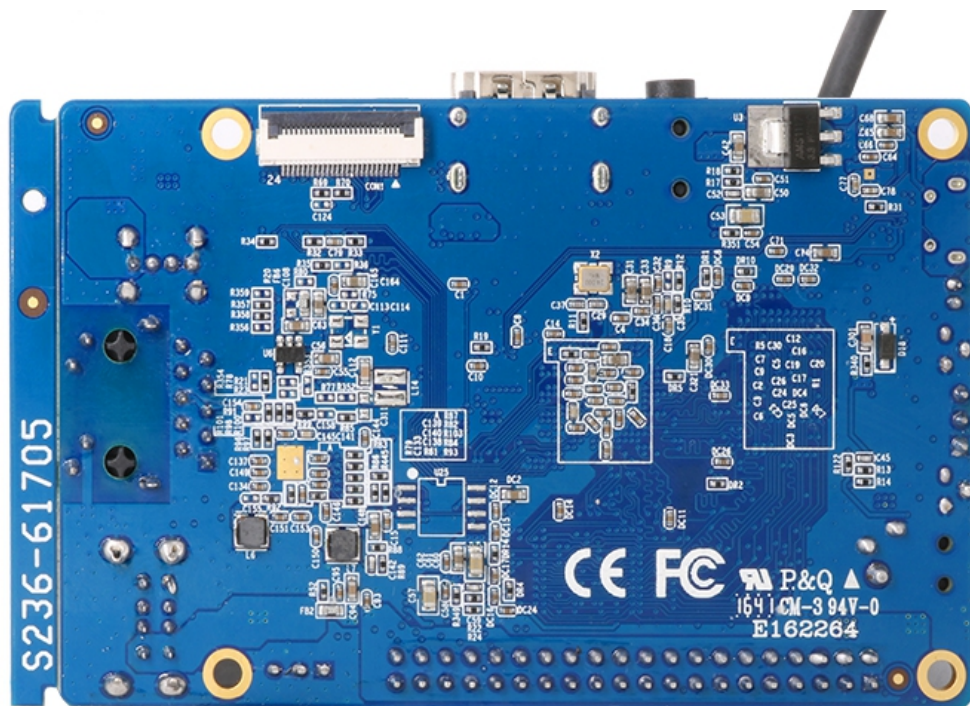


1.5. Orange Pi Prime 的顶层视图和底层视图

顶层视图:

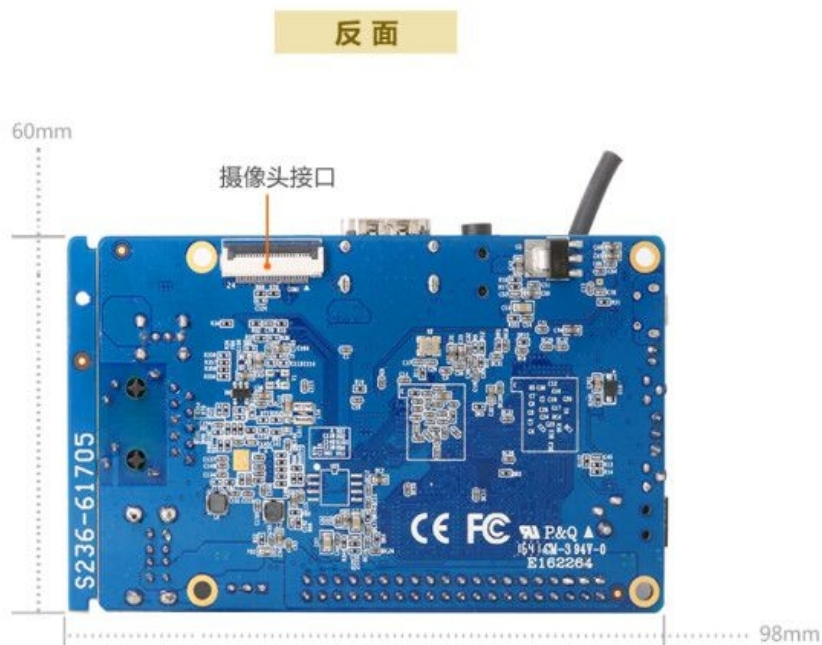
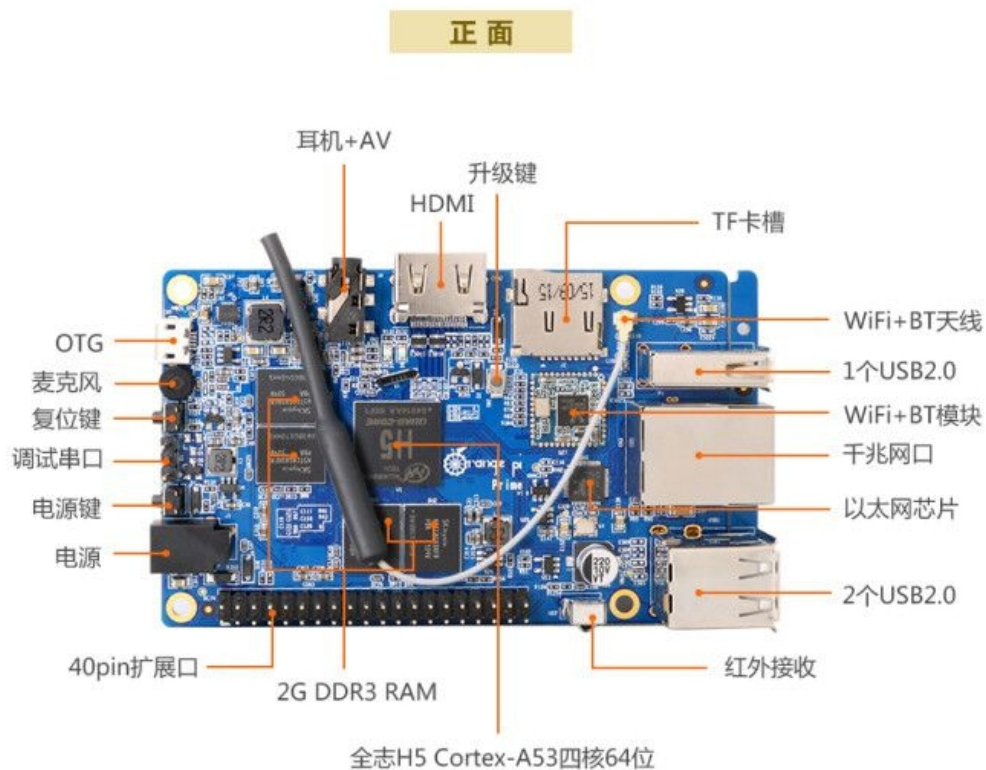


底层视图:





1.6. Orange Pi Prime 的接口详情图

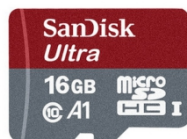


2. 开发板使用介绍

2.1. 准备需要的配件

1) TF 卡, 最小 8GB 容量的 class10 级以上的高速卡, 建议使用闪迪的 TF 卡, Orange Pi 测试都是使用闪迪的 TF 卡, 其他牌子的 TF 卡可能会出现系统无法启动的问题

SanDisk 闪迪



2) TF 卡读卡器, 用于读写 TF 卡



3) 标准 HDMI 转 HDMI 连接线, 用于将开发板连接到 HDMI 显示器或者电视进行显示



4) 电源适配器, 至少 5V/2A 的高品质 Micro USB 接口的电源适配器



5) USB接口的鼠标和键盘，只要是标准USB接口的鼠标和键盘都可以，鼠标和键盘可以用来控制Orange Pi开发板

6) 百兆或者千兆网线，用于将开发板连接到因特网

7) 红外遥控器，主要用于控制安卓系统



8) AV 视频线，如果希望通过 CVBS 接口而不是 HDMI 接口来显示视频，那么就需要通过 AV 视频线将开发板连接到电视



9) GC2035 或者OV5640 摄像头套件，可以连接到开发板的CSI接口来显示视频图像



Orange Pi 开发板 200万像素摄像头



10) Micro USB 接口的数据线，使用 Android 系统 ADB 调试功能时，需要通过 Micro USB 接口的数据线将开发板和电脑连接起来



11) USB 转 TTL 模块和杜邦线，使用串口调试功能时，需要 USB 转 TTL 模块和杜邦线来连接开发板和电脑



12) 安装有 Ubuntu 和 Windows 操作系统的个人电脑

1	Ubuntu14.04 PC	可选，用于编译 Android 源码
2	Ubuntu18.04 PC	可选，用于编译 Linux 源码
3	Windows PC	用于烧录 Android 和 Linux 镜像

2.2. 下载开发板的镜像和相关的资料

1) 中文版资料的下载网址为



<http://www.orangepi.cn/downloadresourcescn/>

2) 英文版资料的下载网址为

<http://www.orangepi.org/downloadresources/>

3) 资料主要包含

- Android 源码**: 保存在百度云盘和谷歌网盘上
- Linux 源码**: 保存在 github 上, 链接地址为

<https://github.com/orangepi-xunlong>

- 用户手册和原理图**: 芯片相关的数据手册也会放在这里
- 官方工具**: 主要包括开发板使用过程中需要用到的软件
- Android 镜像**: 保存在百度云盘和谷歌网盘上
- Ubuntu 镜像**: 保存在百度云盘和谷歌网盘上
- Debian 镜像**: 保存在百度云盘和谷歌网盘上
- Armbian 镜像**, 由 Armbian 社区开发的镜像, 使用过程中如果遇到问题, 请先去 [armbian 论坛](#) 反馈, Armbian 镜像的维护者以及其他使用 Armbian 镜像的人会协助解决各种问题, 这是解决问题最快的方法。Orange Pi 不负责维护这个镜像



Android 源码
更新: 2019-11-11
[Download Now](#)



Linux 源码
更新: 2019-11-11
[Download Now](#)



用户手册和原理图
更新: 2019-11-11
[Download Now](#)



官方工具
更新: 2019-11-11
[Download Now](#)



Android镜像
更新: 2019-11-11
[Download Now](#)



Ubuntu镜像
更新: 2019-11-11
[Download Now](#)



Debian镜像
更新: 2019-11-11
[Download Now](#)



Armbian镜像
更新: 2019-11-11
[Download Now](#)

2.3. 基于 Windows PC 将 Linux 镜像烧写到 TF 卡的方法

1) 首先准备一张 8GB或更大容量的TF卡, TF卡的传输速度必须为**class10** 以上, 建议使用闪迪等品牌的TF卡

2) 然后使用读卡器把TF卡插入windows电脑

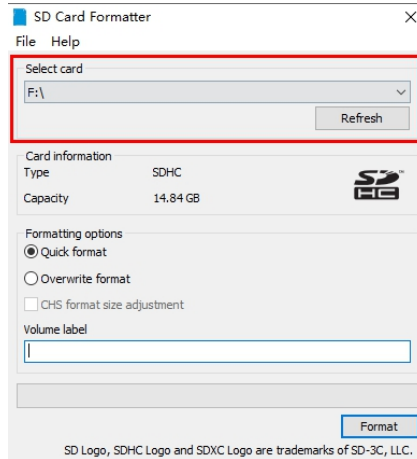
3) 接着格式化TF卡

- 可以使用**SD Card Formatter** 这个软件格式化TF卡, 其下载地址为

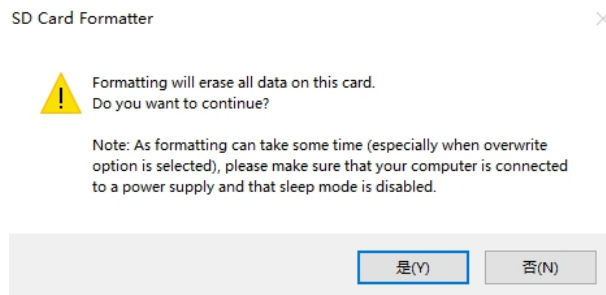
https://www.sdcard.org/downloads/formatter/eula_windows/SDCardFormatterv5_WinEN.zip



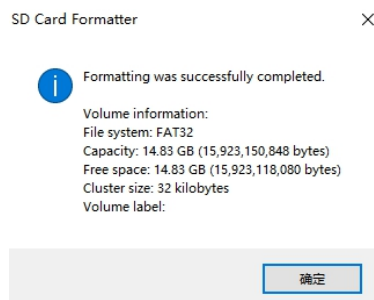
- b. 下载完后直接解压安装即可，然后打开软件
- c. 如果电脑只插入了TF卡，则**Select card**一栏中会显示TF卡的盘符，如果电脑插入了多个USB存储设备，可以通过下拉框选择TF卡对应的盘符



- d. 然后点击 **Format**，格式化前会弹出一个警告框，选择“是(Y)”后就会开始格式化



- e. 格式化完TF卡后会弹出下图所示的信息，点击确定即可



4) 从[Orange Pi的资料下载页面](#)下载想要烧录的Linux操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以“.img”结尾的文件就是操作系统的镜



像文件，大小一般都在 1GB 以上

5) 使用Win32Diskimager烧录Linux镜像到TF卡

a. Win32Diskimager的下载页面为

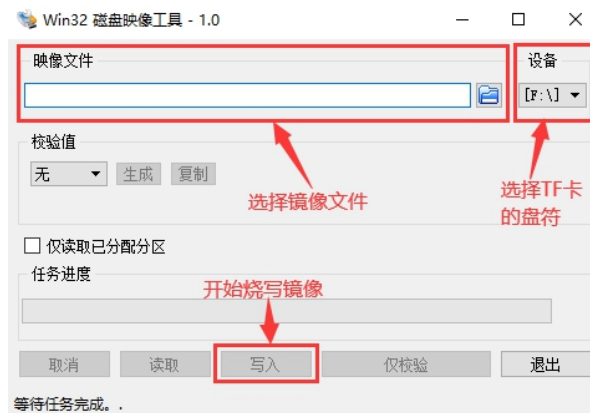
<http://sourceforge.net/projects/win32diskimager/files/Archive/>

b. 下载完后直接安装即可，Win32Diskimager界面如下所示

a) 首先选择镜像文件的路径

b) 然后确认下TF卡的盘符和“设备”一栏中显示的一致

c) 最后点击“写入”即可开始烧录



c. 镜像写入完成后，点击“退出”按钮退出即可，然后就可以拔出TF卡插到开发板中启动

2.4. 基于 Ubuntu PC 将 Linux 镜像烧写到 TF 卡的方法

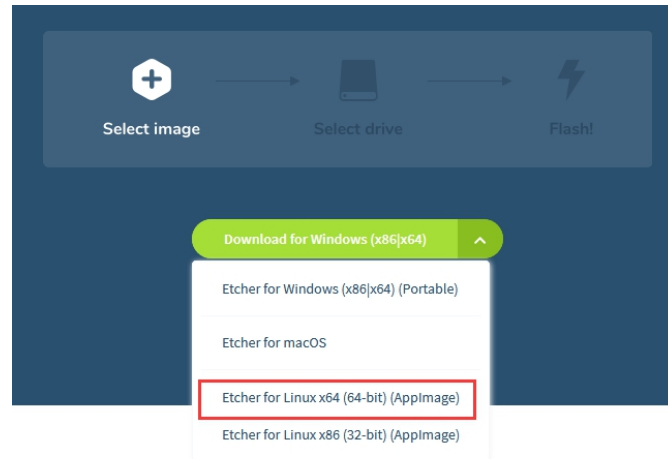
1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 以上，建议使用闪迪等品牌的 TF 卡

2) 然后使用读卡器把 TF 卡插入电脑

3) 下载 balenaEtcher 软件，下载地址为

<https://www.balena.io/etcher/>

4) 进入 balenaEtcher 下载页面后，请通过下拉框选择 Linux 版本的软件进行下载



5) 下载完后使用 **unzip** 进行解压，解压后的 **balenaEtcher-1.5.109-x64.AppImage** 就是烧录需要的软件

```
test@test:~$ unzip balena-etcher-electron-1.5.109-linux-x64.zip
Archive:  balena-etcher-electron-1.5.109-linux-x64.zip
  inflating: balenaEtcher-1.5.109-x64.AppImage
test@test:~$ ls
balenaEtcher-1.5.109-x64.AppImage  balena-etcher-electron-1.5.109-linux-x64.zip
```

6) 从 [Orange Pi 的资料下载页面](#) 下载想要烧录的 Linux 操作系统镜像文件压缩包，然后使用解压软件解压，解压后的文件中，以 “.img” 结尾的文件就是操作系统的镜像文件，大小一般在 1GB 以上

a. 7z 结尾的压缩包的解压命令如下所示

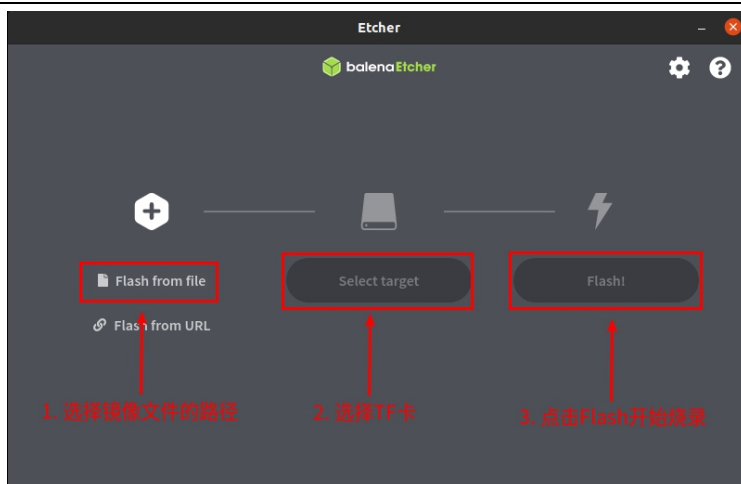
```
test@test:~$ 7z x image_filename.7z
```

b. tar.gz 结尾的压缩包的解压命令如下所示

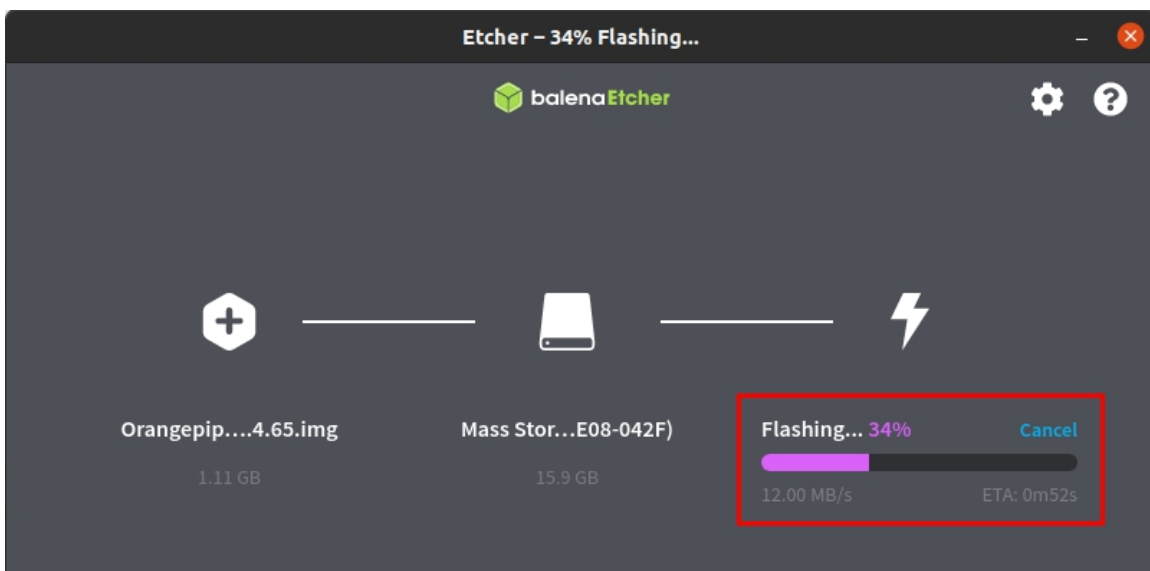
```
test@test:~$ tar -xzf image_filename.tar.gz
```

7) 在 Ubuntu PC 的图形界面双击 **balenaEtcher-1.5.109-x64.AppImage** 即可打开 balenaEtcher，打开后的界面如下图所示

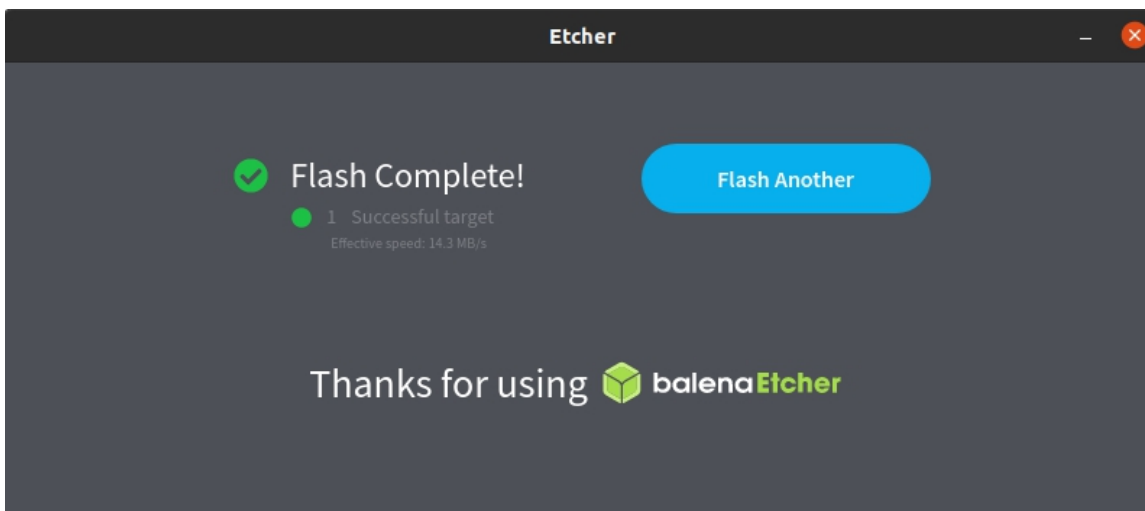
- 首先选择镜像文件的路径
- 然后选择 TF 卡的设备号
- 最后点击 Flash 开始烧录进行



8) 烧录过程会提示写入的速度和剩余时间



9) 烧录完后会显示下面的界面，此时就可以把 TF 卡从电脑中拔出来插到开发板中启动了



2.5. 烧写 Android 固件到 TF 卡的方法

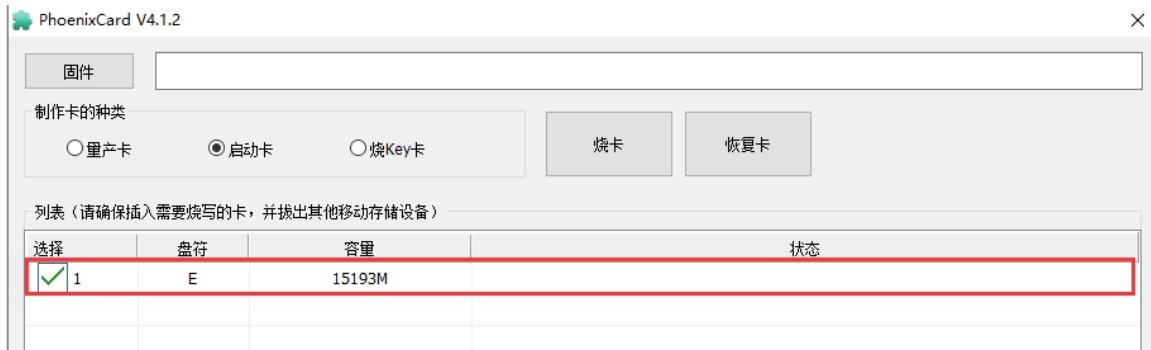
Android 镜像只能在 Windows 平台下使用 PhoenixCard 软件烧录到 TF 卡中，在 Linux 平台下无法烧录

- 1) 首先准备一张 8GB 或更大容量的 TF 卡，TF 卡的传输速度必须为 **class10** 以上，建议使用闪迪等品牌的 TF 卡
- 2) 然后使用读卡器把 TF 卡插入电脑
- 3) 从 [Orange Pi 的资料下载页面](#) 下载 Android 固件和 PhoenixCard 烧写工具，请确保 PhoenixCard 工具的版本为 **PhoenixCard v4.1.2**
- 4) 使用解压软件解压下载的 Android 固件的压缩包，解压后的文件中，以 “.img” 结尾的文件就是 Android 固件
- 5) 使用解压软件解压 PhoenixCard v4.1.2.rar，此软件无需安装，在解压后的文件夹中找到 **PhoenixCard** 打开即可

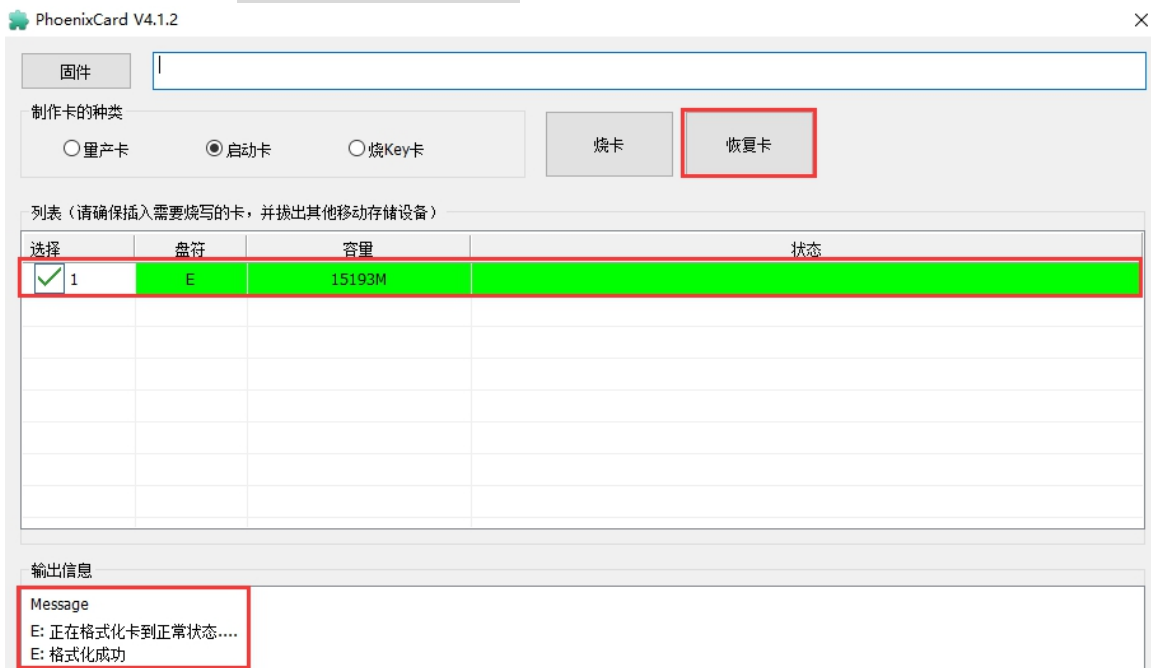
	option.cfg	2017/6/27 16:53	CFG 文件	1 KB
	ParserManager.dll	2017/6/28 17:51	应用程序扩展	81 KB
	PhoenixCard ManualV4.1.1	2017/7/5 15:05	DOC 文档	382 KB
	PhoenixCard	2017/10/25 15:03	应用程序	1,742 KB
	PhoenixCard.lan	2017/10/25 15:16	LAN 文件	3 KB
	PhoenixCard.pdb	2017/10/25 15:03	PDB 文件	22,971 KB



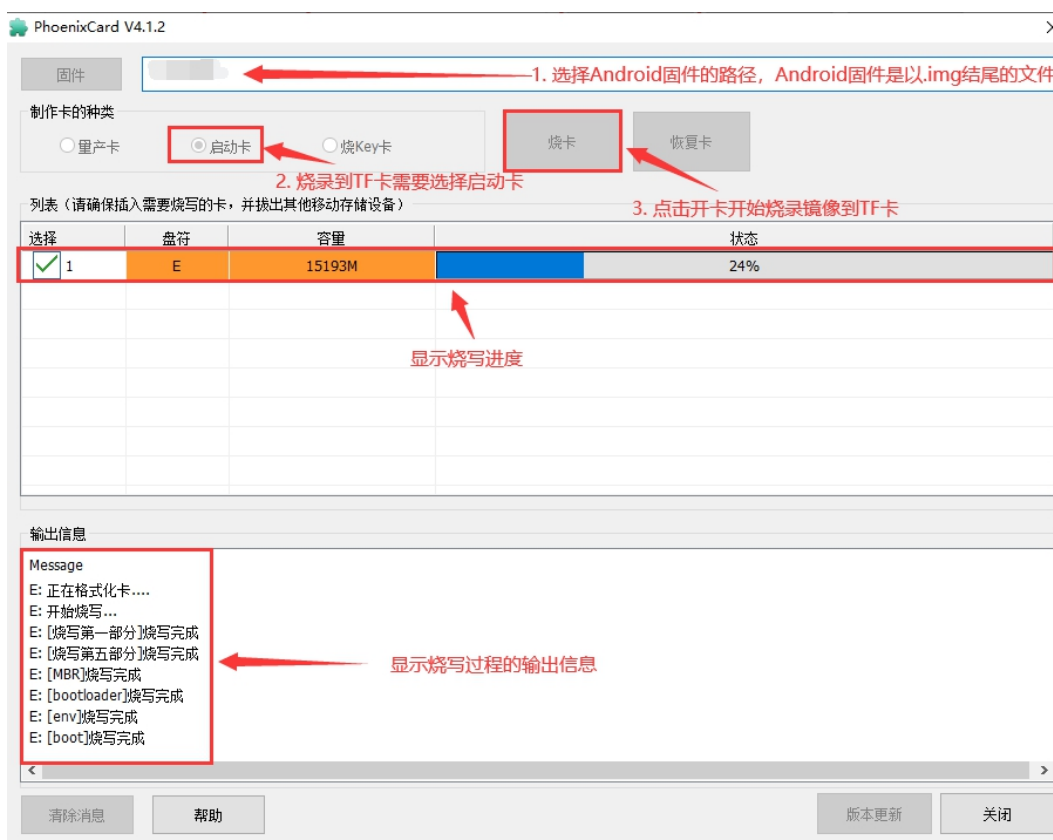
6) 打开 PhoenixCard 后, 如果 TF 卡识别正常, 会在中间的列表中显示 TF 卡的盘符和容量, **请务必确认显示的盘符和你想烧录的 TF 卡的盘符是一致的**, 如果没有显示可以尝试拔插下 TF 卡



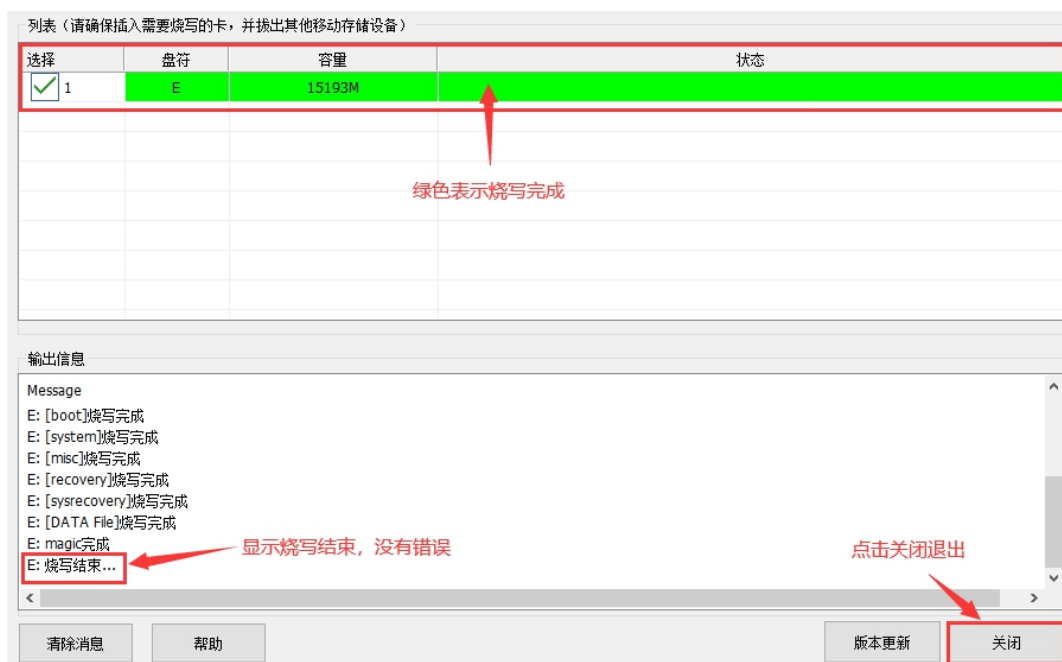
7) 确认完盘符后, 先格式化 TF 卡, 点击 PhoenixCard 中**恢复卡**按钮即可, 也可以使用前面提到的 **SD Card Formatter** 进行 TF 卡的格式化



- 8) 然后开始将 Android 固件写入 TF 卡
- 首先在**固件**一栏中选择 Android 固件的路径
 - 在**制作卡的种类**中选择**启动卡**
 - 然后点击**烧卡**按钮就会开始烧录



9) 烧录完后 PhoenixCard 的显示如下图所示, 此时点击**关闭**按钮即可退出 PhoenixCard, 然后就可以把 TF 卡从电脑中拔出来插到开发板中启动了





2.6. 启动香橙派开发板

- 1) 首先将烧录好镜像的 TF 卡插入香橙派开发板的 TF 卡插槽中
- 2) 开发板有 HDMI 接口, 可以通过 HDMI 连接线把开发板连接到电视或者其他的 HDMI 显示器
- 3) 接上 USB 鼠标和键盘, 用于控制香橙派开发板
- 4) 连接一个 5V/2A (5V/3A 的也可以) 的高品质的电源适配器
 - a. 切记不要插入 12V 的电源适配器, 如果插入了 12V 的电源适配器, 会烧坏开发板
 - b. 系统上电启动过程中很多不稳定的现象基本都是供电有问题导致的, 所以一个靠谱的电源适配器很重要
- 5) 然后打开电源适配器的开关, 如果一切正常, 此时 HDMI 显示器就能看到系统的启动画面了
- 6) 如果想通过调试串口查看系统的输出信息, 请使用串口线将开发板连接到电脑, 串口的连接方法请参看[调试串口的使用方法](#)一节

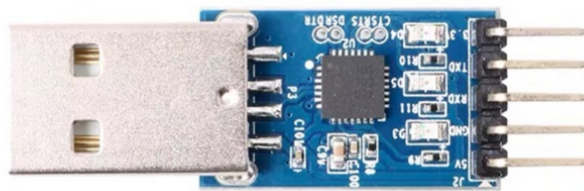
2.7. 调试串口的使用方法

2.7.1. 调试串口的连接说明

- 1) 首先需要准备一个 USB 转 TTL 模块, 此模块在 Orange Pi 的店铺中可以买到, 如果有其他类似的 USB 转 TTL 模块也可以, 然后将 USB 转 TTL 模块的 USB 一端插入到电脑的 USB 接口中

香橙派

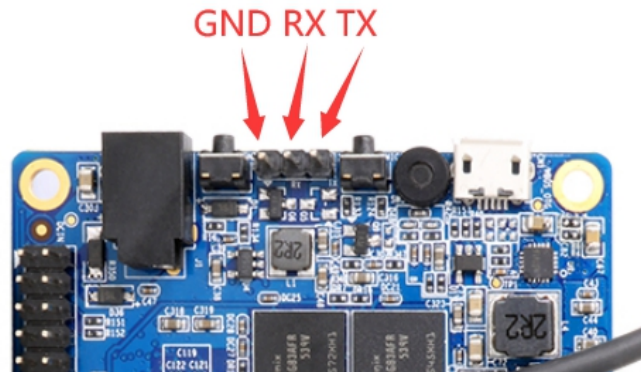
USB转TTL模块



- USB转TTL模块的3.3V不需要接
- USB转TTL模块的TXD接开发板调试串口的RXD
- USB转TTL模块的RXD接开发板调试串口的TXD
- USB转TTL模块的GND接开发板调试串口的GND
- USB转TTL模块的5V不需要接



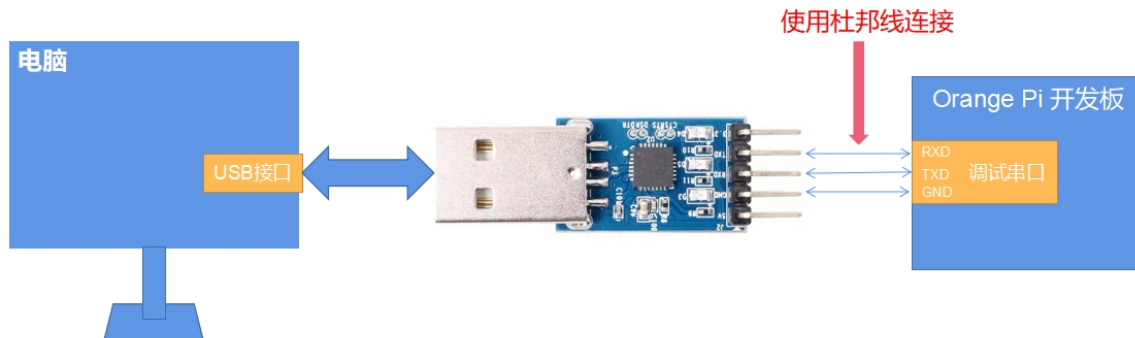
2) 开发板的调试串口 GND、TXD 和 RXD 引脚的对应关系如下图所示



3) USB 转 TTL 模块 GND、TXD 和 RXD 引脚需要通过杜邦线连接到开发板的调试串口上

- USB 转 TTL 模块的 GND 接到开发板的 GND 上
- USB 转 TTL 模块的 **RXD** 接到开发板的 **TXD** 上
- USB 转 TTL 模块的 **TXD** 接到开发板的 **RXD** 上

4) USB 转 TTL 模块连接电脑和 Orange Pi 开发板的示意图如下所示



USB转TTL模块连接电脑和 Orange Pi 开发板的示意图

2.7.2. Ubuntu 平台调试串口的使用方法

1) 如果 USB 转 TTL 模块连接正常，在 Ubuntu PC 的/dev 下就可以看到对应的设备节点名，记住这个节点名，后面设置串口软件时会用到

```
test@test:~$ ls /dev/ttyUSB*  
/dev/ttyUSB0
```

2) linux 下可以使用的串口调试工具有很多，如 putty、minicom 等，下面演示 putty 的使用方法



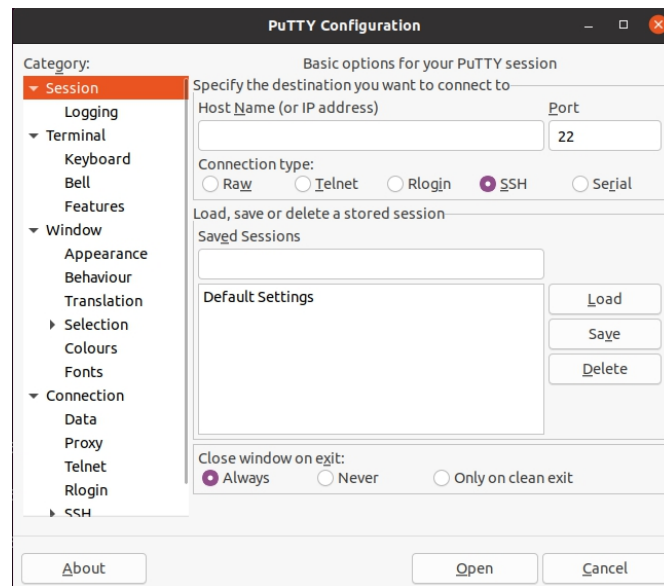
3) 首先在 Ubuntu PC 上安装 putty

```
test@test:~$ sudo apt update  
test@test:~$ sudo apt install putty
```

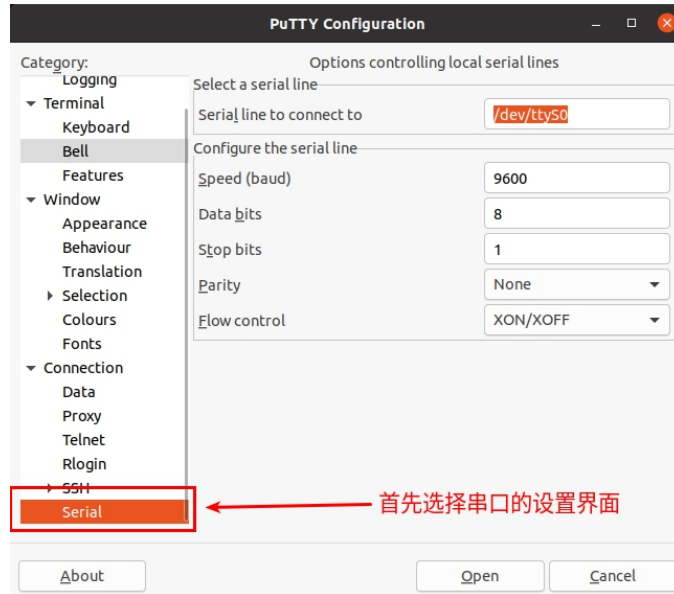
4) 然后运行 putty，记得加 sudo 权限

```
test@test:~$ sudo putty
```

5) 执行 putty 命令后会弹出下面的界面

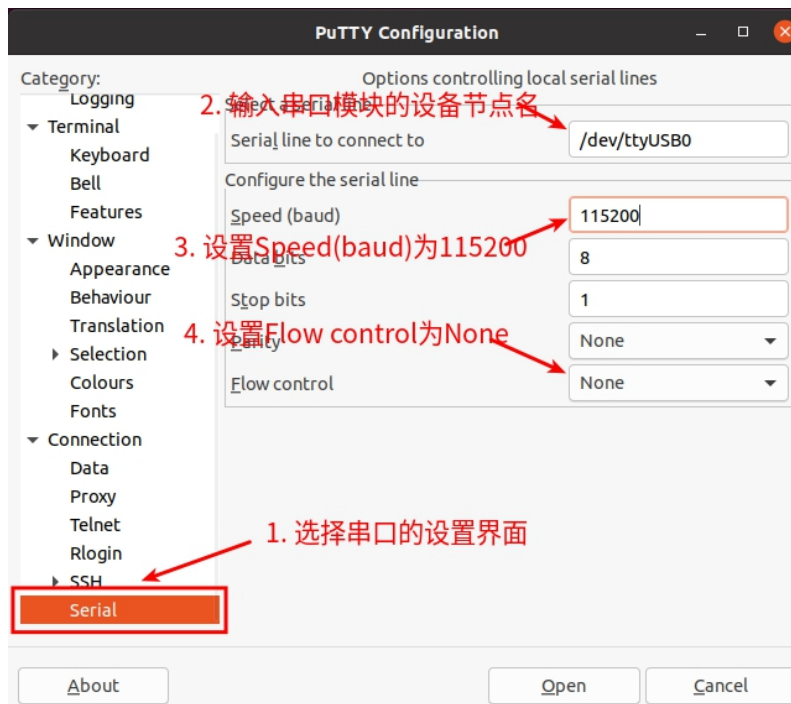


6) 首先选择串口的设置界面



7) 然后设置串口的参数

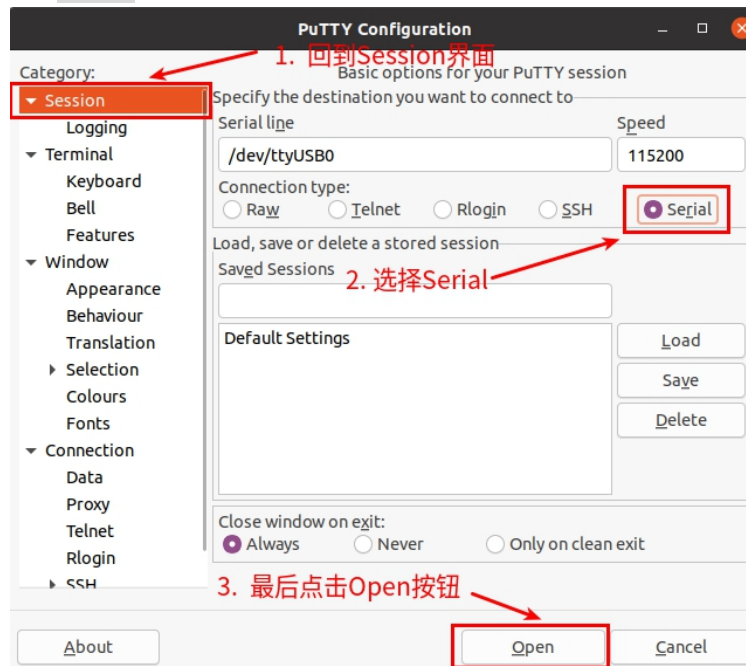
- 设置 **Serial line to connect to** 为 `/dev/ttyUSB0` (修改为对应的节点名, 一般为 `/dev/ttyUSB0`)
- 设置 **Speed(baud)** 为 115200
- 设置 **Flow control** 为 None



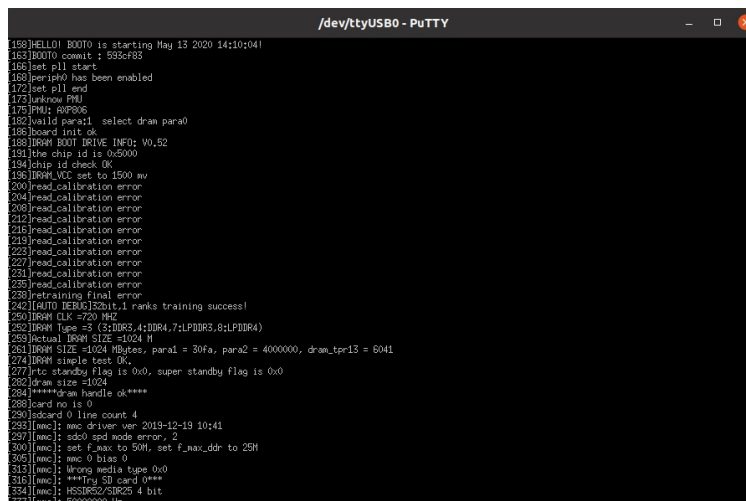
8) 在串口的设置界面设置完后, 在回到 Session 界面



- a. 首先选择 **Connection type** 为 Serial
- b. 然后点击 **Open** 按钮连接串口



9) 启动开发板后，就能从打开的串口终端中看到系统输出的 Log 信息了



2.7.3. Windows 平台调试串口的使用方法

1) Windows 下可以使用的串口调试工具有很多，如 SecureCRT、MobaXterm 等，下面演示 MobaXterm 的使用方法，这款软件是免费的，无需购买序列号即可使用

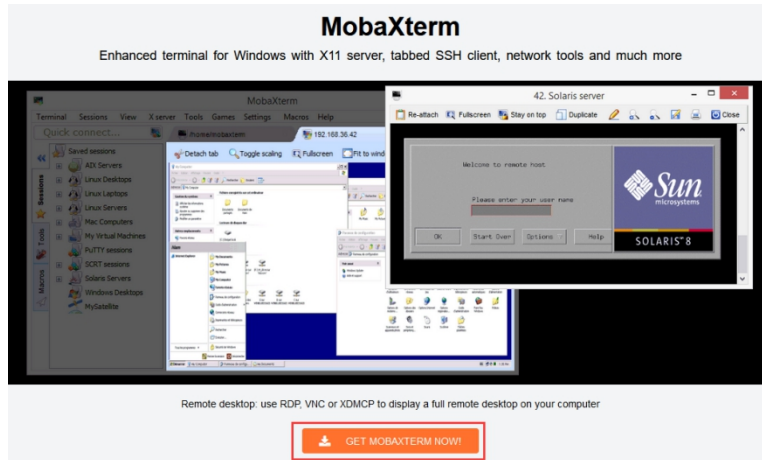
2) 下载 MobaXterm

- a. 下载 MobaXterm 网址如下

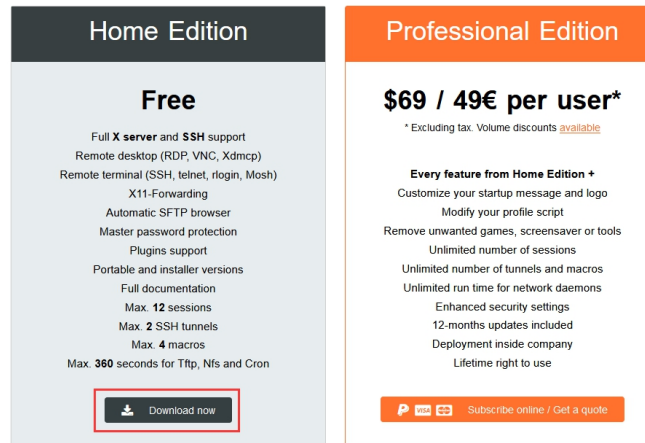


<https://mobaxterm.mobatek.net/>

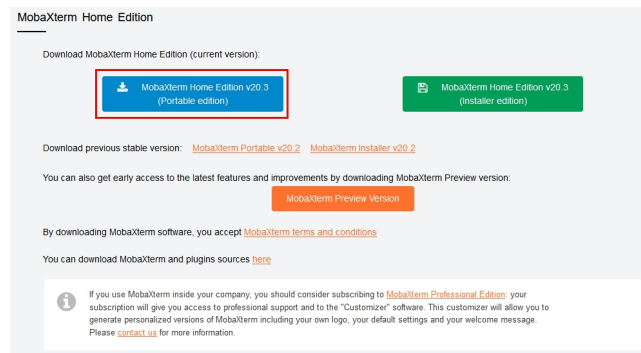
b. 进入 MobaXterm 下载网页后点击 **GET XOBATERM NOW!**



c. 然后选择下载 Home 版本



d. 然后选择 Portable 便携式版本，下载完后无需安装，直接打开就可以使用



3) 下载完后使用解压缩软件解压下载的压缩包，即可得到 MobaXterm 的可执软件，然后双击打开



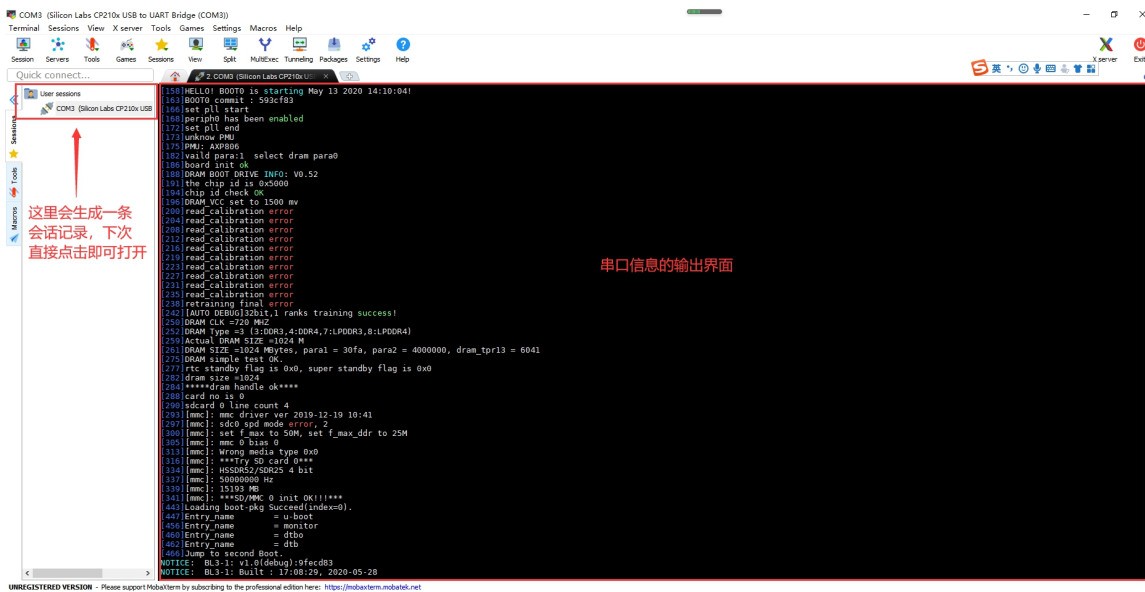
名称	修改日期	类型	大小
CygUtils.plugin	2020/5/21 4:06	PLUGIN 文件	15,570 KB
MobaXterm_Personal_20.3	2020/6/5 4:30	应用程序	14,104 KB

4) 打开软件后，设置串口连接的步骤如下

- 打开会话的设置界面
- 选择串口类型
- 选择串口的端口号（根据具体的情况选择对应的端口号），如果看不到端口号，请使用 [360 驱动大师](#) 扫描安装 USB 转 TTL 串口芯片的驱动
- 选择串口的波特率为 115200
- 最后点击 **OK** 按钮完成设置



5) 点击 OK 后会进入下面的界面，此时启动开发板就能看到串口的输出信息了





3. Linux 系统使用说明

3.1. 已支持的 linux 发行版类型和内核版本

发行版类型	内核版本	服务器版	桌面版
Ubuntu 20.04	linux5.4	支持	支持
Ubuntu 18.04	linux5.4	支持	支持
Debian 10	linux5.4	支持	支持
Ubuntu 16.04	linux3.10	支持	支持

3.2. Linux3.10 内核镜像驱动适配情况

功能	状态
HDMI 视频	OK
HDMI 音频	OK
USB2.0 x 3	OK
TF 卡启动	OK
红外	OK
WIFI	OK
蓝牙	OK
网口	OK
耳机音频接口	OK
USB 摄像头	OK
LED 灯	OK
40pin GPIO	OK
I2C	OK
SPI	OK
UART	OK
温度传感器	OK
硬件看门狗	OK
OV5640 摄像头	OK
GC2035 摄像头	OK
TV-OUT	NO



3.3. Linux5.4 内核镜像驱动适配情况

功能	状态
HDMI 视频	OK
HDMI 音频	OK
USB2.0 x 3	OK
TF 卡启动	OK
红外	OK
WIFI	OK
蓝牙	OK
网口	OK
耳机音频接口	OK
MIC 录音	OK
USB 摄像头	OK
LED 灯	OK
40pin GPIO	OK
I2C	OK
SPI	OK
UART	OK
温度传感器	OK
硬件看门狗	OK
OV5640 摄像头	OK
GC2035 摄像头	NO
TV-OUT	NO

3.4. 登录账号和密码

账号	密码
root	orangePi
orangePi	orangePi



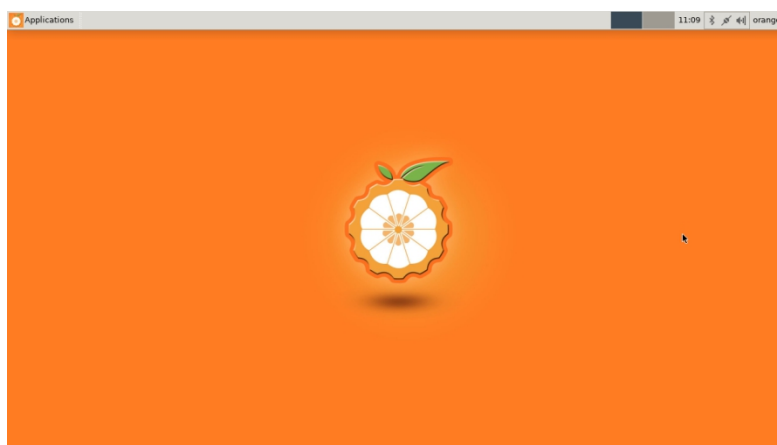
3.5. 板载 LED 灯测试说明

开发板上有两个 LED 灯，一个绿灯，一个红灯，系统启动时 LED 灯默认显示情况如下所示

	绿灯	红灯
u-boot 启动阶段	灭	亮
内核启动到进入系统	亮	灭
GPIO 口	PL10	PA20

3.6. Linux 桌面版系统自动登录说明

1) linux 桌面版系统默认启动后会自动登录进入桌面，无需输入密码



2) 修改`/etc/lightdm/lightdm.conf.d/22-orangepi-autologin.conf`中的配置可以禁止桌面版系统自动登录桌面，修改命令如下，也可以打开配置文件直接修改

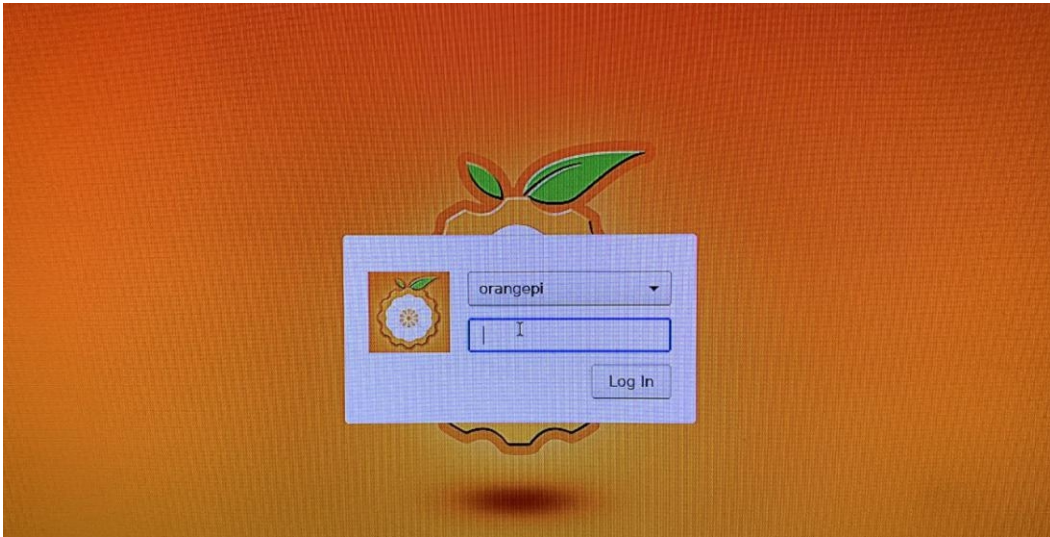
```
root@orangepi:~# sed -i "s/autologin-user=orangepi/#autologin-user=orangepi/"  
/etc/lightdm/lightdm.conf.d/22-orangepi-autologin.conf
```

3) 修改完后`/etc/lightdm/lightdm.conf.d/22-orangepi-autologin.conf`的配置如下所示

```
root@orangepi:~# cat /etc/lightdm/lightdm.conf.d/22-orangepi-autologin.conf  
[Seat:]*]  
#autologin-user=orangepi  
autologin-user-timeout=0  
user-session=xfce
```



- 4) 然后重启系统就会出现登录对话框，此时需要输入密码才能进入系统



3.7. Linux 系统第一次启动自动扩容 rootfs

- 1) 通过 TF 卡第一次启动 linux 系统时会通过 `orange pi-resize-filesystem.service` 这个 systemd 服务来调用 `orange pi-resize-filesystem` 脚本自动进行 rootfs 的扩容
- 2) 登录系统后可以通过 `df -h` 命令来查看 rootfs 的大小，如果和 TF 卡的实际容量一致，说明自动扩容运行正确

```
root@orange pi:~# df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
udev	430M	0	430M	0%	/dev
tmpfs	100M	5.6M	95M	6%	/run
/dev/mmcblk0p1	15G	915M	14G	7%	/
tmpfs	500M	0	500M	0%	/dev/shm

- 3) 需要注意的是，linux 系统只有一个 ext4 格式的分区，没有使用单独的 BOOT 分区来存放内核镜像等文件，也就不存在 BOOT 分区扩容的问题
- 4) 另外如果不需要自动扩容 rootfs，可以使用下面的方法来禁止
- 首先将 linux 镜像烧录到 TF 卡中
 - 然后将 TF 卡插入 Ubuntu PC 中（Windows 不行），Ubuntu PC 一般会自



挂载 TF 卡的分区，如果自动挂载正常，使用 ls 命令可以看到下面的输出，TF 卡的分区名和下面命令所示名字不一定相同，请根据实际情况进行修改

```
test@test:~$ ls /media/test/27e62f92-8250-4ef1-83db-3d8f0c2e23db/
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root  run
sbin  selinux  srv  sys  tmp  usr  var
```

c. 然后在 Ubuntu PC 中将当前用户切换成 root 用户

```
test@test:~$ sudo -i
[sudo] test 的密码:
root@test:~#
```

d. 然后进入 TF 卡中的 linux 系统的 root 目录下新建一个名为 **.no_rootfs_resize** 的文件

```
root@test:~# cd /media/test/27e62f92-8250-4ef1-83db-3d8f0c2e23db
root@test:/media/test/27e62f92-8250-4ef1-83db-3d8f0c2e23db# cd root
root@test:/media/test/27e62f92-8250-4ef1-83db-3d8f0c2e23db/root# touch .no_rootfs_resize
root@test:/media/test/27e62f92-8250-4ef1-83db-3d8f0c2e23db/root# ls .no_rootfs*
.no_rootfs_resize
```

e. 然后就可以卸载 TF 卡，再拔出 TF 插到开发板启动，linux 系统启动时，当检测到 **/root** 目录下有 **.no_rootfs_resize** 这个文件就不会再自动扩容 rootfs 了

f. 禁止 rootfs 自动扩容后可以看到 TF 卡可用容量只有 200M 左右

```
root@orangepi:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            927M    0  927M   0% /dev
tmpfs           200M   5.6M  194M   3% /run
/dev/mmcblk0p1  1.5G  1.3G  196M  87% /
tmpfs           997M    0  997M   0% /dev/shm
tmpfs           5.0M   4.0K   5.0M   1% /run/lock
tmpfs           997M    0  997M   0% /sys/fs/cgroup
tmpfs           997M   4.0K   997M   1% /tmp
/dev/zram0       49M   1.5M   44M   4% /var/log
tmpfs           200M    0  200M   0% /run/user/0
```

3.8. Linux 系统修改日志级别（loglevel）的方法

1) linux 系统的 loglevel 默认设置为 1，当使用串口查看启动信息时，内核输出 log 如下所示，基本全部屏蔽了



```
Starting kernel ...

Uncompressing Linux... done, booting the kernel.

Orange Pi 2.1.2 Bionic ttyS0

orangePi login:
```

2) 当系统启动出现问题时，可以使用下面的方法来修改 loglevel 的值，从而打印更多的 log 信息到串口显示，方便调试

```
root@orangePi:~# sed -i "s/verbosity=1/verbosity=7/" /boot/orangepiEnv.txt
root@orangePi:~# sed -i "s/console=both/console=serial/" /boot/orangepiEnv.txt
```

3) 上面的命令其实都是设置 `/boot/orangepiEnv.txt` 中的变量，设置完后可以打开 `/boot/orangepiEnv.txt` 检查下

```
root@orangePi:~# cat /boot/orangepiEnv.txt
verbosity=7
bootlogo=false
console=serial
```

4) 然后重启开发板，内核的输出信息就都会打印到串口输出了

```
[ OK ] Started Dispatcher daemon for systemd-networkd.
[ OK ] Reached target Multi-User System.
[ OK ] Reached target Graphical Interface.
        Starting Update UTMP about System Runlevel Changes...
[ OK ] Started Update UTMP about System Runlevel Changes.
[ OK ] Started Authorization Manager.

Orange Pi 2.1.2 Bionic ttyS0

orangePi login:
```

3.9. HDMI 显示测试

1) 使用 HDMI 转 HDMI 线连接 Orange Pi 开发板和 HDMI 显示器



- 2) 启动 linux 系统后如果 HDMI 显示器有图像输出说明 HDMI 接口使用正常
- 3) 在没有网络和串口的情况下, 可以使用 HDMI 显示器, 然后连接鼠标和键盘来控制开发板

3. 10. 以太网口测试

- 1) 首先将网线插入开发板的以太网接口, 并确保网络是畅通的
- 2) 系统启动后会通过 DHCP 自动给以太网卡分配 IP 地址
- 3) 查看 IP 地址的命令如下

```
root@orangepi:~# ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.1.47  netmask 255.255.255.0  broadcast 192.168.1.255
    inet6 fe80::e56:c34d:62f0:8d6e  prefixlen 64  scopeid 0x20<link>
    ether 02:81:3e:a8:58:d8  txqueuelen 1000  (Ethernet)
    RX packets 2165  bytes 177198 (177.1 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 312  bytes 40435 (40.4 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
    device interrupt 39
```

- 4) 测试网络连通性的命令如下

```
root@orangepi:~# ping www.orangepi.org -I eth0
PING www.orangepi.org (182.92.236.130) from 192.168.1.47 eth0: 56(84) bytes of data.
```



```
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=39.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=39.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=39.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=39.7 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=39.7 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 39.323/39.617/39.922/0.274 ms
```

3. 11. WIFI 连接测试

请不要通过修改 `/etc/network/interfaces` 配置文件的方式来连接 WIFI，通过这种方式连接 WIFI 网络使用会有问题

3. 11. 1. 服务器版镜像通过命令连接 WIFI

当开发板没有连接以太网，没有连接 HDMI 显示屏，只连接了串口时，推荐使用此小节演示的命令来连接 WIFI 网络。因为 nmtui 在某些串口软件（如 minicom）中只能显示字符，无法正常显示图形界面。当然，如果开发板连接了以太网或者 HDMI 显示屏，也可以使用此小节演示的命令来连接 WIFI 网络的

1) 先登录 linux 系统，有下面三种方式

- a. 如果开发板连接了网线，可以通过 [ssh 远程登录 linux 系统](#)
- a. 如果开发板连接好了调试串口，可以使用串口终端登录 linux 系统
- b. 如果连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 linux 系统

2) 首先使用 `nmcli dev wifi` 命令扫描周围的 WIFI 热点

```
root@orangepi:~# nmcli dev wifi
```




```

root@orangePi:~# nmcli dev wifi
IN-USE BSSID SSID MODE CHAN RATE SIGNAL BARS SECURITY
28:6C:07:6E:87:2E orangePi Infra 9 260 Mbit/s 97 WPA1 WPA2
D8:D8:66:A5:BD:D1 orangePi Infra 10 270 Mbit/s 90 WPA1 WPA2
A0:40:A0:A1:72:20 orangePi Infra 4 405 Mbit/s 82 WPA2
28:6C:07:6E:87:2F orangePi Infra 149 540 Mbit/s 80 WPA1 WPA2
CA:50:E9:89:E2:44 orangePi Infra 1 130 Mbit/s 79 WPA1 WPA2
A0:40:A0:A1:72:31 orangePi Infra 100 405 Mbit/s 67 WPA2
D4:EE:07:08:A9:E0 orangePi Infra 4 130 Mbit/s 55 WPA1 WPA2
88:C3:97:49:25:13 orangePi Infra 6 130 Mbit/s 52 WPA1 WPA2
00:BD:82:51:53:C2 orangePi Infra 12 130 Mbit/s 49 WPA1 WPA2
C0:61:18:FA:49:37 orangePi Infra 149 270 Mbit/s 47 WPA1 WPA2
04:79:70:8D:0C:B8 orangePi Infra 153 270 Mbit/s 47 WPA2
04:79:70:FD:0C:B8 orangePi Infra 153 270 Mbit/s 47 WPA2
9C:A6:15:DD:E6:0C orangePi Infra 10 270 Mbit/s 45 WPA1 WPA2
B4:0F:3B:45:D1:F5 orangePi Infra 48 270 Mbit/s 45 WPA1 WPA2
E8:CC:18:4F:7B:44 orangePi Infra 157 135 Mbit/s 45 WPA1 WPA2
B0:95:8E:D8:2F:ED orangePi Infra 11 405 Mbit/s 39 WPA1 WPA2
C0:61:18:FA:49:36 orangePi Infra 11 270 Mbit/s 24 WPA1 WPA2
root@orangePi:~#

```

3) 然后使用 **nmcli** 命令连接扫描到的 WIFI 热点，其中：

- wifi_name** 需要换成想连接的 WIFI 热点的名字
- wifi_passwd** 需要换成想连接的 WIFI 热点的密码

```

root@orangePi:~# nmcli dev wifi connect wifi_name password wifi_passwd
Device 'wlan0' successfully activated with 'cf937f88-ca1e-4411-bb50-61f402eef293'.

```

4) 通过 **ifconfig** 命令可以查看 wifi 的 IP 地址

```

root@orangePi:~# ifconfig wlan0
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.49 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::76bb:f67d:ef98:2f9a prefixlen 64 scopeid 0x20<link>
    ether 12:81:3e:a8:58:d8 txqueuelen 1000 (Ethernet)
    RX packets 185 bytes 109447 (109.4 KB)
    RX errors 0 dropped 61 overruns 0 frame 0
    TX packets 27 bytes 14783 (14.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

5) 使用 **ping** 命令可以测试 wifi 网络的连通性

```

root@orangePi:~# ping www.orangePi.org -I wlan0
PING www.orangePi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of
data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms

```




```
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangepi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

3. 11. 2. 服务器版镜像通过图形化方式连接 WIFI

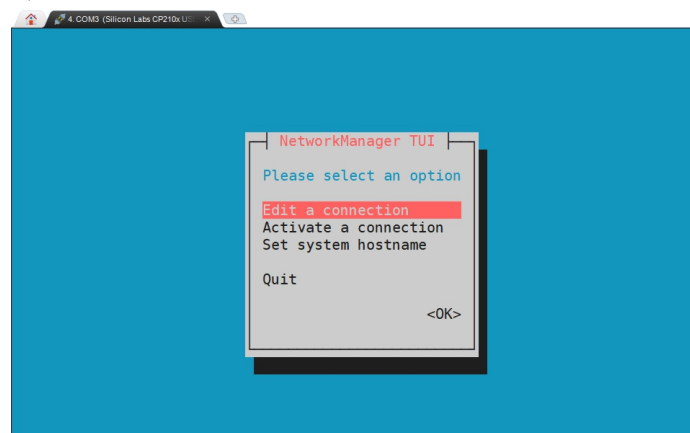
1) 先登录 linux 系统，有下面三种方式

- 如果开发板连接了网线，可以通过 [ssh 远程登录 linux 系统](#)
- 如果开发板连接好了调试串口，可以使用串口终端登录 linux 系统（串口软件请使用 MobaXterm，使用 minicom 无法显示图形界面）
- 如果开发板连接了开发板到 HDMI 显示器，可以通过 HDMI 显示的终端登录到 linux 系统

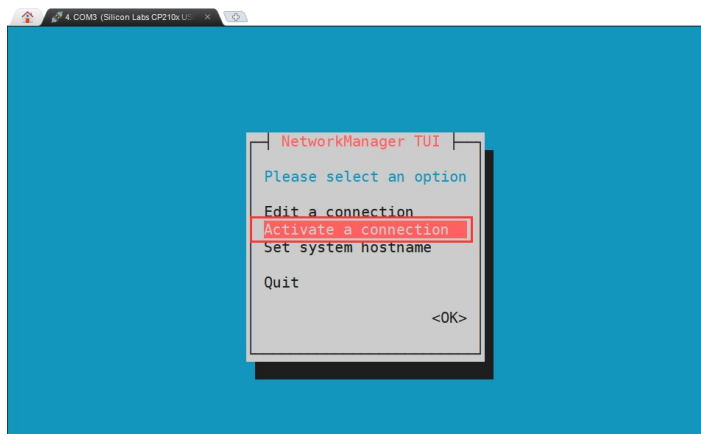
2) 然后在命令行中输入 nmtui 命令打开 wifi 连接的界面

```
root@orangepi:~# nmtui
```

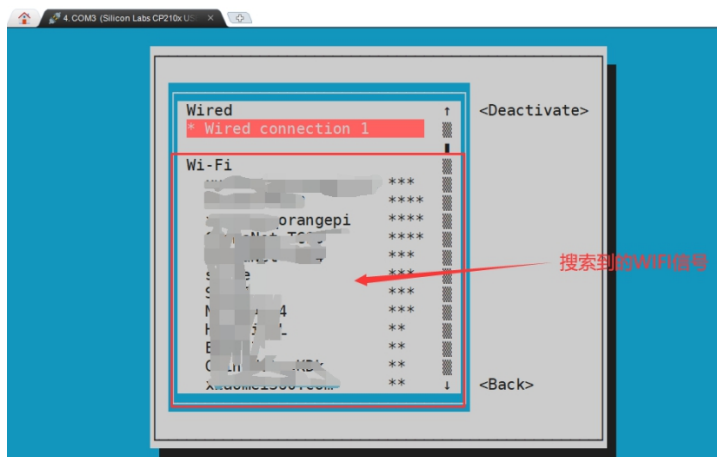
3) 输入 nmtui 命令打开的界面如下所示



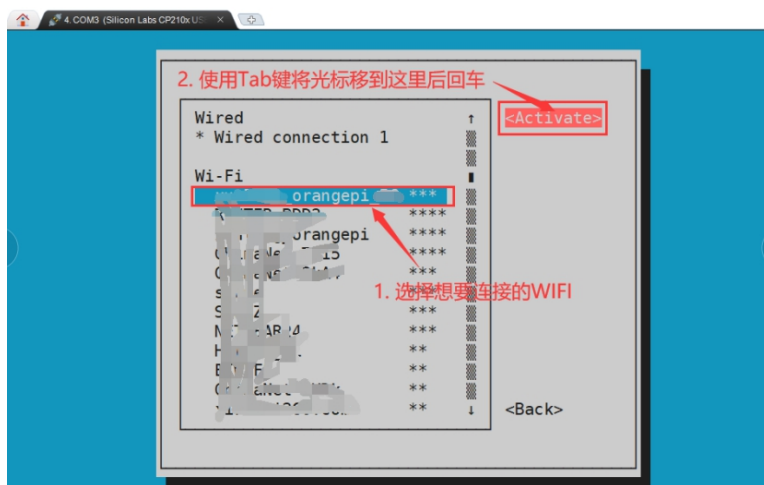
4) 选择 **Activate a connect** 后回车



5) 然后就能看到所有搜索到的 WIFI 热点



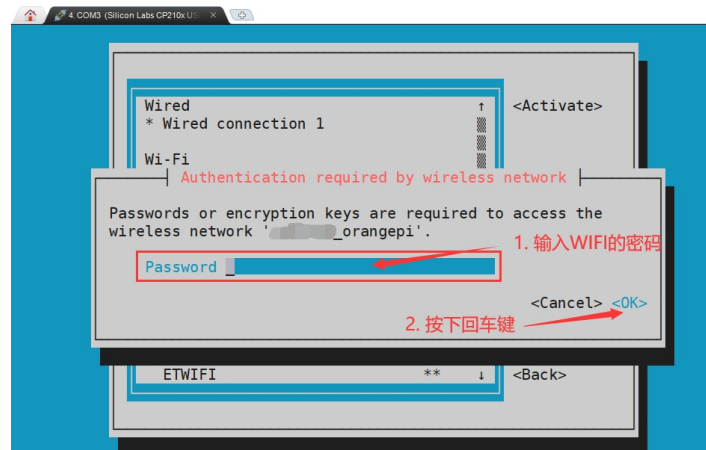
6) 选择想要连接的 WIFI 热点后再使用 Tab 键将光标定位到 **Activate** 后回车



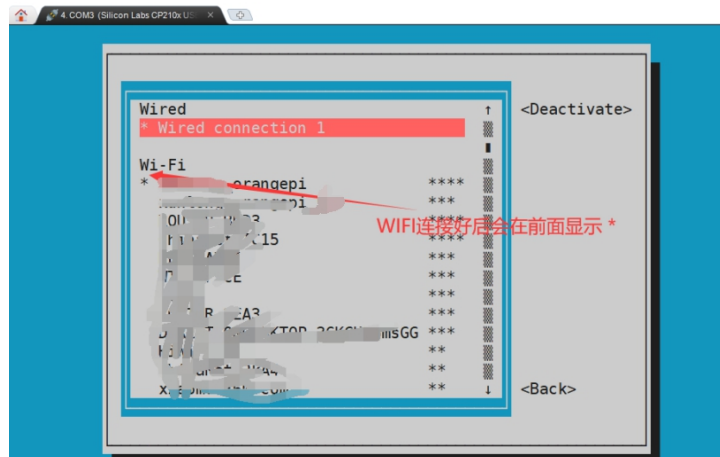
7) 然后会弹出输入密码的对话框, 在 **Pssword** 内输入对应的密码然后回车就会开始



连接 WIFI



8) WIFI 连接成功后会在已连接的 WIFI 名称前显示一个 “*”



9) 通过 ifconfig 命令可以查看 wifi 的 IP 地址

```
root@orangepi:~# ifconfig wlan0
```

```
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.1.49  netmask 255.255.255.0  broadcast 192.168.1.255
    inet6 fe80::76bb:f67d:ef98:2f9a  prefixlen 64  scopeid 0x20<link>
    ether 12:81:3e:a8:58:d8  txqueuelen 1000  (Ethernet)
    RX packets 185   bytes 109447 (109.4 KB)
    RX errors 0    dropped 61  overruns 0  frame 0
    TX packets 27   bytes 14783 (14.7 KB)
    TX errors 0    dropped 0 overruns 0  carrier 0  collisions 0
```

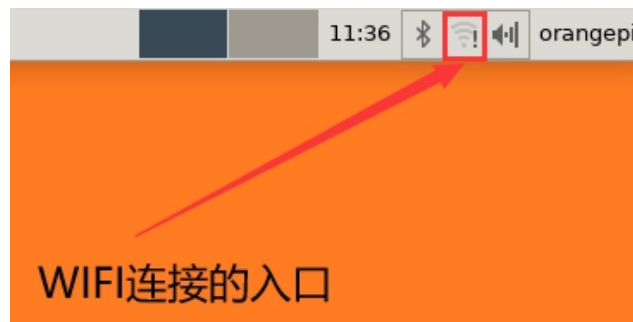
10) 使用 ping 命令可以测试 wifi 网络的连通性



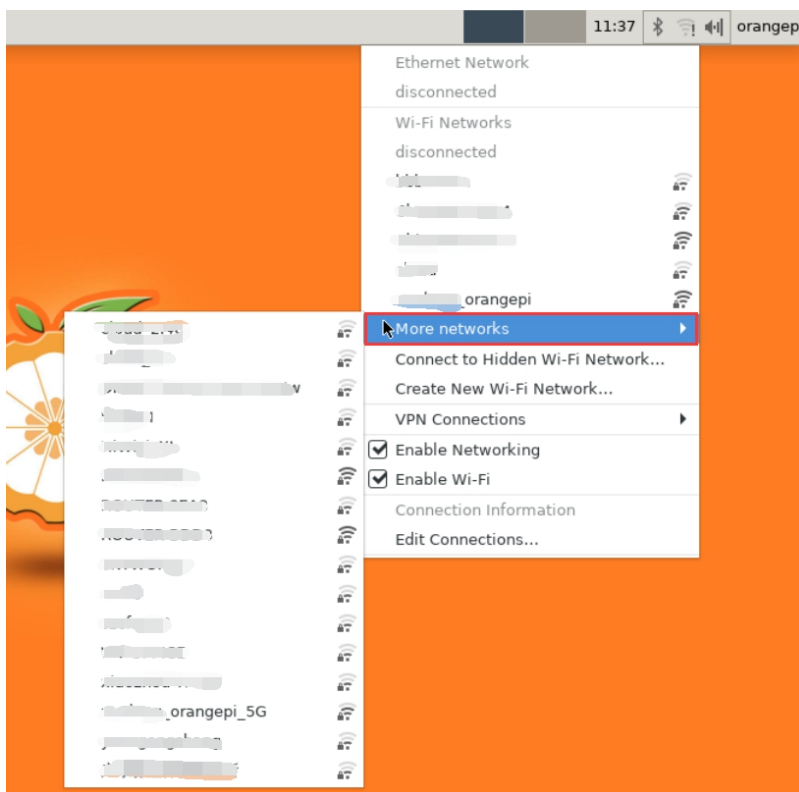
```
root@orangePi:~# ping www.orangePi.org -I wlan0
PING www.orangePi.org (182.92.236.130) from 192.168.1.49 wlan0: 56(84) bytes of
data.
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=1 ttl=52 time=43.5 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=2 ttl=52 time=41.3 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=3 ttl=52 time=44.9 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=4 ttl=52 time=45.6 ms
64 bytes from 182.92.236.130 (182.92.236.130): icmp_seq=5 ttl=52 time=48.8 ms
^C
--- www.orangePi.org ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4006ms
rtt min/avg/max/mdev = 41.321/44.864/48.834/2.484 ms
```

3. 11. 3. 桌面版镜像的测试方法

- 1) 点击桌面右上角的网络配置图标（测试 WIFI 时请不要连接网线）



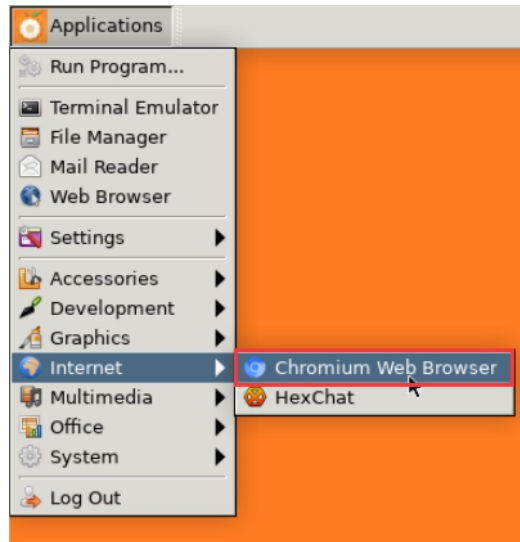
- 2) 在弹出的下拉框中点击 **More networks** 可以看到所有扫描到的 WIFI 热点，然后选择想要连接的 WIFI 热点



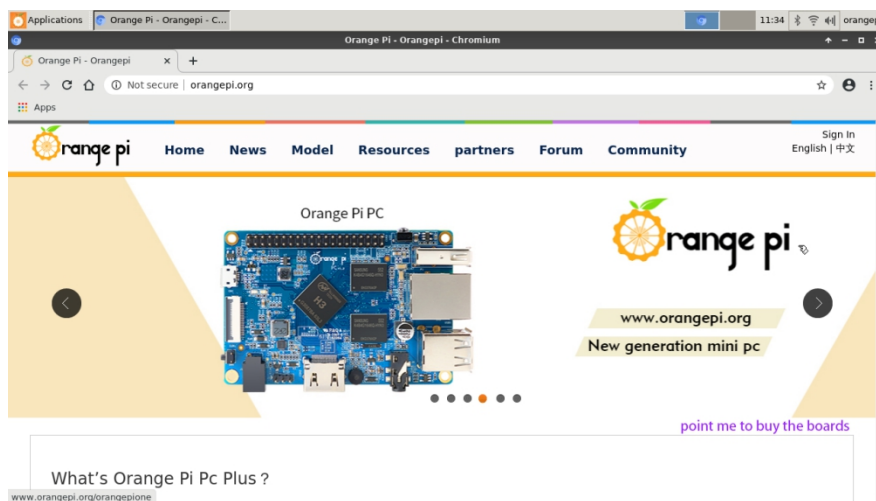
3) 然后输入 WIFI 热点的密码，再点击 **Connect** 就会开始连接 WIFI



4) 连接好 WIFI 后，可以打开浏览器查看是否能上网，浏览器的入口如下图所示



5) 打开浏览器后如果能看到 Orange Pi 网站的页面，或者能打开其他网页说明 WIFI 连接正常



3. 12. SSH 远程登录开发板

Linux 系统默认都开启了 ssh 远程登录，并且允许 root 用户登录系统。ssh 登录前首先需要确保以 wifi 或者网口已连接，然后使用 ifconfig 命令或者通过查看路由器的方式获取开发板的 IP 地址

3. 12. 1. Ubuntu 下 SSH 远程登录开发板

1) 首先获取开发板的 IP 地址



2) 然后就可以通过 ssh 命令远程登录 linux 系统

```
test@test:~$ ssh root@192.168.1.36 //需要替换为开发板的 IP 地址
root@192.168.1.36's password: //在这里输入密码，默认密码为 orangepi
```

3) 成功登录系统后的显示如下图所示

```
test@test:~$ ssh root@192.168.1.36
root@192.168.1.36's password:

Welcome to Orange Pi Bionic with Linux 5.4.65-sunxi

System load:  0.05 0.04 0.02  Up time:    9 min
Memory usage: 8 % of ████████ IP:        192.168.1.36
CPU temp:     44°C
Usage of /:   7% of 15G

Last login: Tue Oct 13 08:21:45 2020 from 192.168.1.48
root@orangepi:~#
```

4) 如果 ssh 登录时提示下面的错误

```
test@test:~$ ssh root@192.168.1.36
Connection reset by 192.168.1.149 port 22
lost connection
```

可以在开发板上输入下面的命令再尝试是否能连接

```
root@orangepi:~# rm /etc/ssh/ssh_host_*
root@orangepi:~# dpkg-reconfigure openssh-server
```

3.12.2. Windows 下 SSH 远程登录开发板

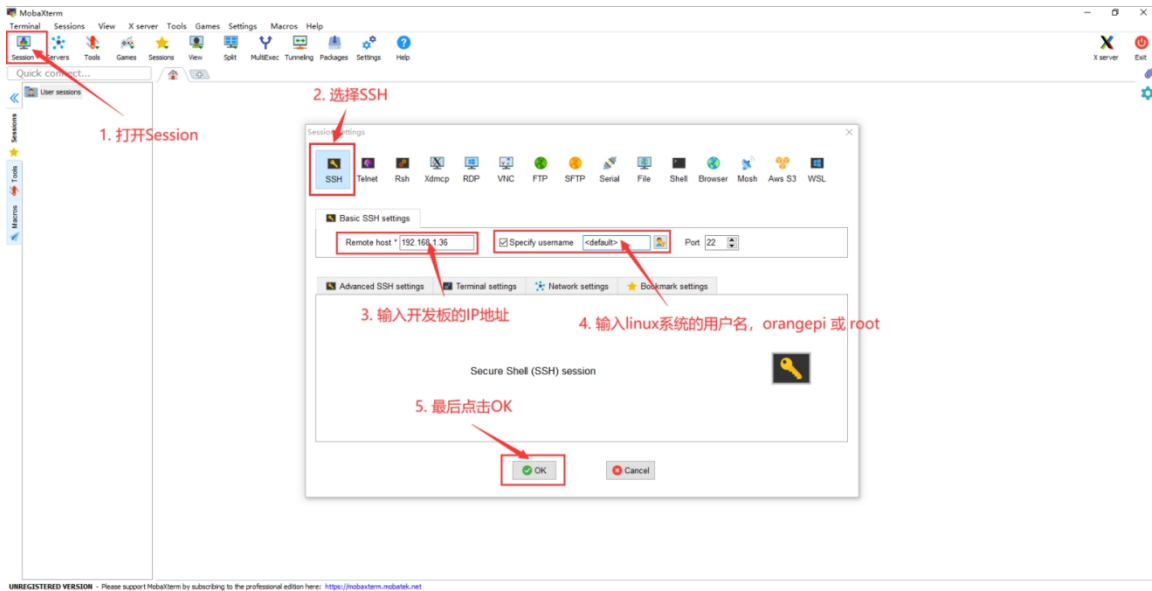
1) 首先获取开发板的 IP 地址

2) 在 windows 下可以使用 MobaXterm 远程登录开发板，首先新建一个 ssh 会话

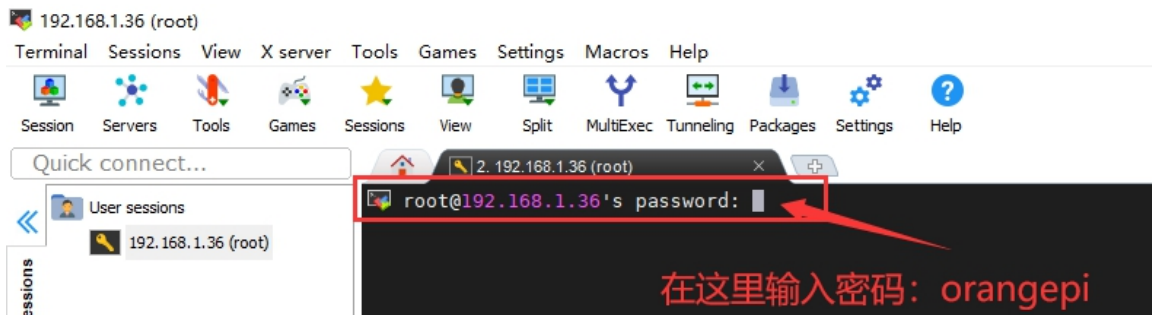
- a. 打开 **Session**
- b. 然后在 **Session Setting** 中选择 **SSH**
- c. 然后在 **Remote host** 中输入开发板的 IP 地址



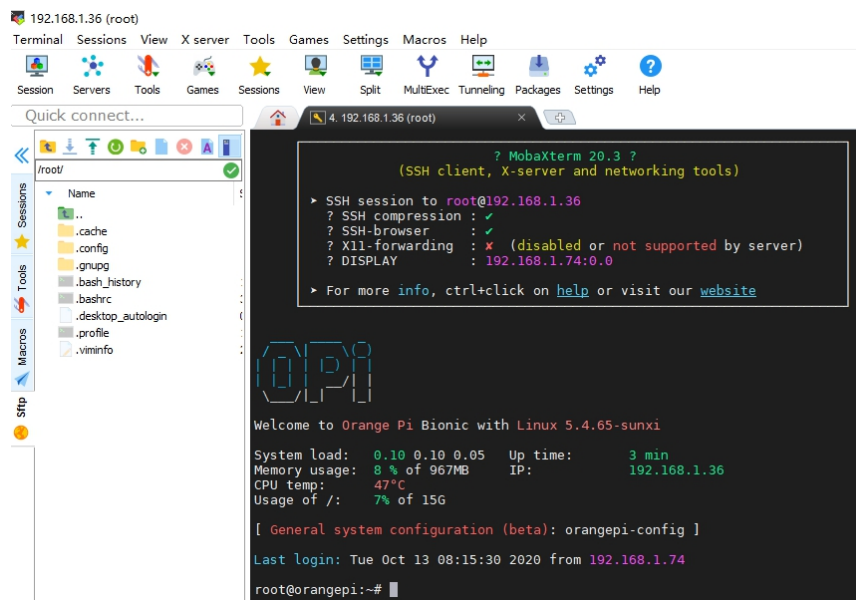
- d. 然后 **Specify username** 中输入 linux 系统的用户名 **root** 或 **orangepi**
- e. 最后点击 OK 即可



- 3) 然后会提示输入密码，默认 root 和 orangepi 用户的密码都为 orangepi



- 4) 成功登录系统后的显示如下图所示



3. 13. Linux3.10 系统 HDMI 分辨率设置

注意：此方法只适用于 linux3.10 内核的系统，linux5.4 内核的系统无法使用

1) 在 linux3.10 系统的 `/boot/orangepiEnv.txt` 中有个 `disp_mode` 变量, 可以通过它来设置 HDMI 输出的分辨率, linux3.10 系统默认设置的分辨率为 1080p60

```
root@orangepi:/boot# cat orangepiEnv.txt
```

```
verbosity=1
```

```
disp_mode=1080p60
```

2) `disp_mode` 变量支持设置的值如下表所示

disp_mode 支持的值	HDMI 分辨率	HDMI 刷新率
480p	720x480	60
576p	720x576	50
720p50	1280x720	50
720p60	1280x720	60
1080i50	1920x1080	50
1080i60	1920x1080	60
1080p50	1920x1080	50
1080p60	1920x1080	60



3) 将 `disp_mode` 变量的值修改为想要输出的分辨率，然后重启系统，HDMI 就会输出所设置的分辨率了

3.14. Linux3.10 系统 Framebuffer 宽度和高度的修改方法

注意：此方法只适用于 linux3.10 内核的系统，linux5.4 内核的系统无法使用

1) 在 linux3.10 系统的 `/boot/orangepiEnv.txt` 中有 `fb0_width` 和 `fb0_height` 两个变量，可以通过它们来设置 Framebuffer 的宽度和高度，linux 系统默认设置 `fb0_width=1280`、`fb0_height=720`

```
root@orangepi:~# cat /boot/orangepiEnv.txt
verbosity=1
console=both
disp_mode=1080p60
fb0_width=1280
fb0_height=720
```

2) `fb0_width` 和 `fb0_height` 不同分辨率对应的参考值如下所示

HDMI 分辨率	fb0_width	fb0_height
480p	720	480
576p	720	576
720p	1280	720
1080p	1920	1080
2160p	1920	1080

3) 在相同的 HDMI 分辨下，当 `fb0_width` 和 `fb0_height` 设置的值越大时，屏幕显示的文字就越小，当 `fb0_width` 和 `fb0_height` 设置的值越小时，屏幕显示的文字就越大

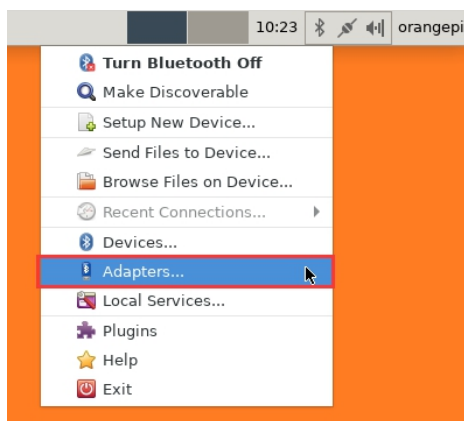
3.15. 蓝牙测试

3.15.1. 桌面版镜像的测试方法

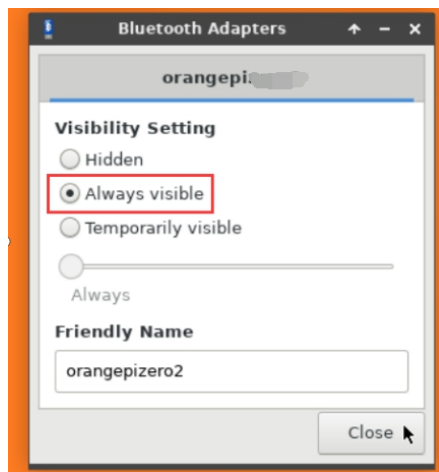
1) 点击桌面右上角的蓝牙图标



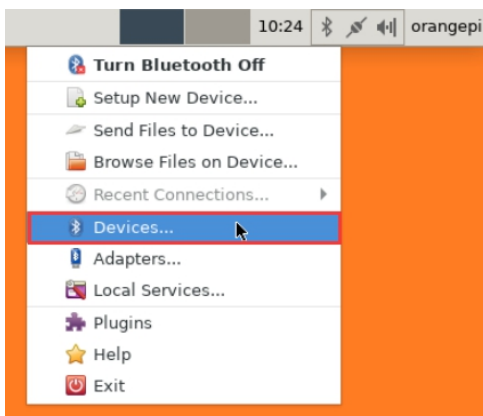
2) 然后选择适配器



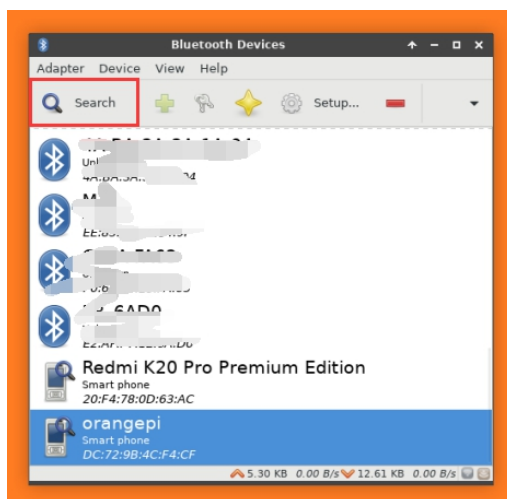
3) 在蓝牙的适配器设置界面中设置 **Visibility Setting** 为 **Always visible**，然后点击 **close** 关闭



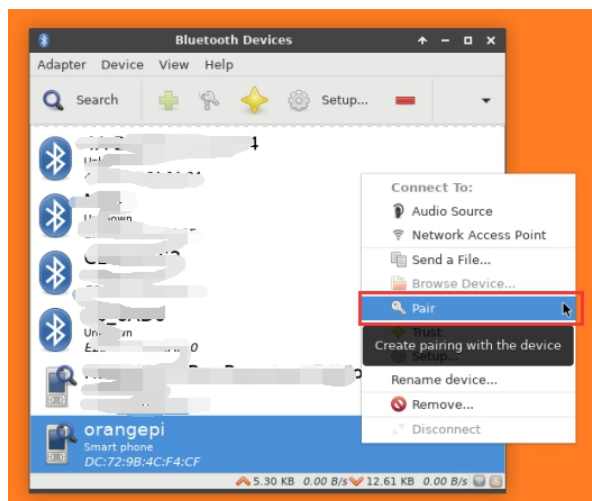
6) 然后打开蓝牙设备的配置界面



7) 点击 Search 即可开始扫描周围的蓝牙设备

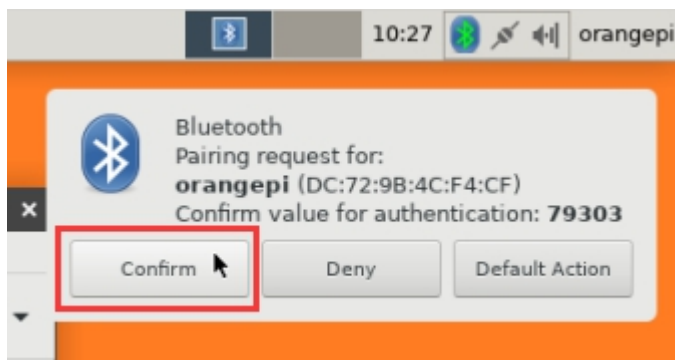


8) 然后选择想要连接的蓝牙设备，再点击鼠标右键就会弹出对此蓝牙设备的操作界面，选择 **Pair** 即可开始配对，这里演示的是和 Android 手机配对

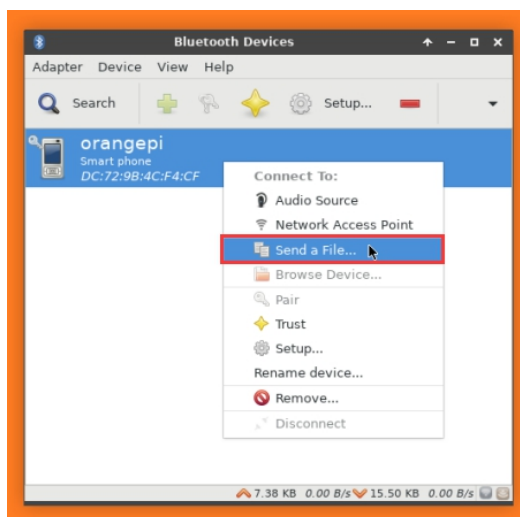




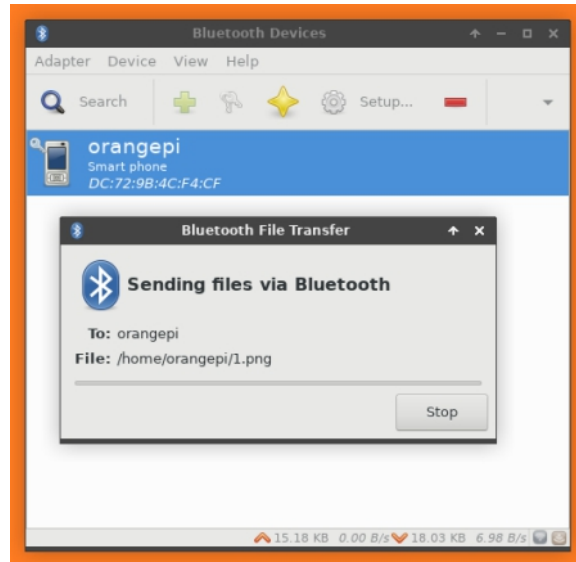
9) 和手机配对时，桌面的右上角会弹出配对确认框，选择 **Confirm** 确认即可，此时手机上也同样需要进行确认



10) 和手机配对完后，可以选择已配对的蓝牙设备，然后右键选择 **Send a File** 即可开始给手机发送一张图片



11) 发送图片的界面如下所示



3. 15. 2. 服务器版镜像的测试方法

1) 首先通过 `hciconfig -a` 来查看蓝牙设备信息，出现下面的信息说明蓝牙初始化正常

```
root@orangepi:~# hciconfig -a
hci0:  Type: Primary  Bus: UART
        BD Address: 28:ED:E0:D6:7D:CA  ACL MTU: 1021:8  SCO MTU: 64:1
        UP RUNNING
        RX bytes:1543 acl:0 sco:0 events:52 errors:0
        TX bytes:2743 acl:0 sco:0 commands:52 errors:0
        Features: 0xbf 0xfe 0xcf 0xfe 0xdb 0xff 0x7b 0x87
        Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV2 HV3
        Link policy: RSWITCH SNIFF
        Link mode: SLAVE ACCEPT
        Name: 'orangepi'
        Class: 0x000000
        Service Classes: Unspecified
        Device Class: Miscellaneous,
        HCI Version: 4.0 (0x6)  Revision: 0x6a
        LMP Version: 4.0 (0x6)  Subversion: 0x2209
        Manufacturer: Broadcom Corporation (15)
```

2) 使用 `bluetoothctl` 可以打开蓝牙扫描周围的设备



```
root@orangepiprime:~# bluetoothctl
[NEW] Controller 28:ED:E0:D6:7D:CA orangepiprime [default]
Agent registered
[bluetooth]# power on
Changing power on succeeded
[bluetooth]# discoverable on
Changing discoverable on succeeded
[CHG] Controller 28:ED:E0:D6:7D:CA Discoverable: yes
[bluetooth]# pairable on
Changing pairable on succeeded
[bluetooth]# scan on
[NEW] Device 6A:31:DF:62:08:78 6A-31-DF-62-08-78
[NEW] Device 56:73:C1:98:C6:63 56-73-C1-98-C6-63
[NEW] Device 9C:2E:A1:42:71:11 小米手机
[NEW] Device DC:72:9B:4C:F4:CF orangepi
```

3) 扫描到想配对的设备后就可以进行配对了，配对需要使用设备的 MAC 地址

```
[bluetooth]# pair DC:72:9B:4C:F4:CF
Attempting to pair with DC:72:9B:4C:F4:CF
[CHG] Device DC:72:9B:4C:F4:CF Connected: yes
Request confirmation
[leeb1m[agent] Confirm passkey 396142 (yes/no): yes
[CHG] Device DC:72:9B:4C:F4:CF Paired: yes
Pairing successful
[bluetooth]# paired-devices
Device DC:72:9B:4C:F4:CF orangepi
```

3. 16. USB 接口测试

3. 16. 1. 连接鼠标或键盘测试

- 1) 将 USB 接口的键盘插入 Orange Pi 开发板的 USB 接口中
- 2) 连接 Orange Pi 开发板到 HDMI 显示器



3) 如果鼠标或键盘能正常操作系统说明 USB 接口使用正常（鼠标只有在桌面版的镜像中才能使用）

3.16.2. 连接 USB 存储设备测试

1) 先格式化 U 盘，然后在 U 盘中放入一些文件

2) 然后将 U 盘插入开发板的 USB 接口中

3) 执行下面的命令如果能看到 sdX 的输出说明 U 盘识别成功

```
root@orangepi:~# cat /proc/partitions | grep "sd*"
major minor #blocks name
 8          0  30044160 sda
 8          1  30043119 sda1
```

4) 使用 mount 命令可以将 U 盘挂载到/mnt 中，然后就能查看 U 盘中的文件了

```
root@orangepi:~# mount /dev/sda1 /mnt/
root@orangepi:~# ls /mnt/
test.txt
```

5) 挂载完后通过 df 命令能查看 U 盘的容量使用情况和挂载点

```
root@orangepi:~# df -h | grep "sd"
/dev/sda1      29G  208K  29G   1% /mnt
```

3.17. USB 以太网卡测试

1) 目前测试过能用的 USB 以太网卡如下所示，其中 RTL8153 USB 千兆网卡插入开发板的 USB 2.0 Host 接口中测试可以正常使用，但是速率是达不到千兆的，这点请注意

序号	型号
1	RTL8152B USB 百兆网卡
2	RTL8153 USB 千兆网卡

2) 首先将 USB 网卡插入开发板的 USB 接口中，然后在 USB 网卡中插入网线，确保网线能正常上网，如果通过 dmesg 命令可以看到下面的 log 信息，说明 USB 网卡



识别正常

```
root@orangepi:~# dmesg | tail
[ 121.985016] usb 3-1: USB disconnect, device number 2
[ 126.873772] sunxi-ehci 5311000.ehci3-controller: ehci_irq: highspeed device connect
[ 127.094054] usb 3-1: new high-speed USB device number 3 using sunxi-ehci
[ 127.357472] usb 3-1: reset high-speed USB device number 3 using sunxi-ehci
[ 127.557960] r8152 3-1:1.0 eth1: v1.08.9
[ 127.602642] r8152 3-1:1.0 enx00e04c362017: renamed from eth1
[ 127.731874] IPv6: ADDRCONF(NETDEV_UP): enx00e04c362017: link is not ready
[ 127.763031] IPv6: ADDRCONF(NETDEV_UP): enx00e04c362017: link is not ready
[ 129.892465] r8152 3-1:1.0 enx00e04c362017: carrier on
[ 129.892583] IPv6: ADDRCONF(NETDEV_CHANGE): enx00e04c362017: link becomes ready
```

3) 然后通过 ifconfig 命令可以看到 USB 网卡的设备节点，以及自动分配的 IP 地址

```
root@orangepi:~# ifconfig
enx00e04c362017: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>      mtu
1500
    inet 192.168.1.177  netmask 255.255.255.0  broadcast 192.168.1.255
    inet6 fe80::681f:d293:4bc5:e9fd  prefixlen 64  scopeid 0x20<link>
    ether 00:e0:4c:36:20:17  txqueuelen 1000  (Ethernet)
    RX packets 1849  bytes 134590 (134.5 KB)
    RX errors 0  dropped 125  overruns 0  frame 0
    TX packets 33  bytes 2834 (2.8 KB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

4) 测试网络连通性的命令如下

```
root@orangepi:~# ping www.baidu.com -I enx00e04c362017
PING www.a.shifen.com (14.215.177.38) from 192.168.1.12 eth0: 56(84) bytes of data.
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=1 ttl=56 time=6.74 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=2 ttl=56 time=6.80 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=3 ttl=56 time=6.26 ms
64 bytes from 14.215.177.38 (14.215.177.38): icmp_seq=4 ttl=56 time=7.27 ms
^C
--- www.a.shifen.com ping statistics ---
```



```
4 packets transmitted, 4 received, 0% packet loss, time 3002ms
rtt min/avg/max/mdev = 6.260/6.770/7.275/0.373 ms
```

3. 18. USB 摄像头测试

1) 首先将 USB 摄像头插入到 Orange Pi 开发板的 USB 接口中

2) 然后使用 lsmod 命令查看系统是否自动加载了 uvcvideo 内核模块

```
root@orangeypi:~# lsmod
Module                               Size  Used by
uvcvideo                             106496  0
```

3) 然后通过 v4l2-ctl (注意 v4l2 中的 l 是小写字母 l, 不是数字 1) 命令查看下 USB 摄像头的设备节点, 从下面的输出可知 USB 摄像头对应的设备节点为 `/dev/video1`, 如果看不到 usb 相关的 video 节点, 说明 USB 摄像头无法识别

```
root@orangeypi:~# apt update
root@orangeypi:~# apt -y install v4l-utils
root@orangeypi:~# v4l2-ctl --list-devices
sun6i-csi (platform:camera):
    /dev/video0

USB 2.0 Camera: HD USB Camera (usb-1c1c000.usb-1):
    /dev/video1
    /dev/video2
```

4) 安装 fswebcam

```
root@orangeypi:~# apt update
root@orangeypi:~# apt-get -y install fswebcam
```

5) 安装完 fswebcam 后可以使用下面的命令来拍照

- d 选项用于指定 USB 摄像头的设备节点
- no-banner 用于去除照片的水印
- r 选项用于指定照片的分辨率
- S 选项用设置于跳过前面的帧数

```
root@orangeypi:~# fswebcam -d /dev/video1 --no-banner -r 1280x720 -S 5 ./image.jpg
```



- 6) 在服务器版的 Linux 系统中，拍完照后可以使用 `scp` 命令将拍好的图片传到 Ubuntu PC 上镜像观看

```
root@orangePi:~# scp image.jpg test@192.168.1.55:/home/test //需要修改为对应的路径
```

- 7) 在桌面版的 linux 系统中，可以通过 HDMI 显示器直接查看拍摄的图片

3. 19. 音频测试

3. 19. 1. 耳机接口播放音频测试

- 1) 将耳机插入开发板的耳机接孔中
- 2) 通过 `aplay -l` 命令可以查看系统支持的声卡设备，其中 **card 0** 就是耳机的声卡设备

```
root@orangePi:~# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: Codec [H3 Audio Codec], device 0: CDC PCM Codec-0 [CDC PCM Codec-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: allwinnerhdm [allwinner-hdmi], device 0: 1c22800.i2s-i2s-hifi i2s-hifi-0
[1c22800.i2s-i2s-hifi i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

- 3) 上传需要播放的音频文件到 linux 系统的 /root 文件夹中，在 Ubuntu PC 中可以使用 `scp` 命令上传，命令中的 IP 地址为 Orange Pi 开发板的 IP 地址，需要进行替换

```
test@test:~/AudioTest$ scp audio.wav root@192.168.1.47:/root
```

- 4) 然后使用 `aplay` 命令播放音频耳机就能听到声音了

```
root@orangePi:~# aplay -D hw:0,0 audio.wav
```



3. 19. 2. HDMI 音频播放测试

1) 首先使用 `aplay -l` 命令确保能看到 HDMI 的声卡设备，其中 **card 1** 就是 HDMI 的声卡设备

```
root@orangepi:~# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: Codec [H3 Audio Codec], device 0: CDC PCM Codec-0 [CDC PCM Codec-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 1: allwinnerhdm1 [allwinner-hdmi], device 0: 1c22800.i2s-i2s-hifi i2s-hifi-0
[1c22800.i2s-i2s-hifi i2s-hifi-0]
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

2) HDMI 音频播放无需其他设置，直接使用 `aplay` 命令播放即可

```
root@orangepi:~# aplay -D hw:1,0 audio.wav
```

3. 19. 3. Mic 录音测试

1) 录音命令如下

```
root@orangepi:~# arecord -d 5 -f cd -t wav test.wav
Recording WAVE test.wav' : Signed 16 bit Little Endian, Rate 44100 Hz, Stereo
root@orangepi:~#
```

2) 录音完毕后会当前路径下生成一个名为 `test.wav` 的录音文件，使用 `aplay` 命令可以播放 `test.wav` 来查看是否有声音输出

3. 20. linux5.4 红外接收测试

1) 红外功能在 `dtb` 中默认是关闭的，需要在 `/boot/orangepiEnv.txt` 中设置 `overlays=cir` 才会打开

```
root@orangepi:~# cat /boot/orangepiEnv.txt
overlays=cir
```

2) 然后重启系统，启动时，在 `u-boot` 的启动 `log` 中可以看到红外 DT overlays 的配置输出



```
Applying kernel provided DT overlay sun50i-h5-cir.dtbo
4185 bytes read in 8 ms (510.7 KiB/s)
Applying kernel provided DT fixup script (sun50i-h5-fixup.scr)
```

3) 安装 ir-keytable 红外测试软件

```
root@orangepi:~# apt update
root@orangepi:~# apt-get install ir-keytable
```

4) 然后执行 ir-keytable 可以查看红外设备的信息

```
root@orangepi:~# ir-keytable
Found /sys/class/rc/rc0/ (/dev/input/event5) with:
  Name: sunxi-ir
  Driver: sunxi-ir, table: rc-empty
  lirc device: /dev/lirc0
  Supported protocols: other lirc rc-5 rc-5-sz jvc sony nec sanyo mce_kbd rc-6 sharp
xmp
  Enabled protocols: lirc
  bus: 25, vendor/product: 0001:0001, version: 0x0100
  Repeat delay = 500 ms, repeat period = 125 ms
```

5) 测试红外接收功能前需要准备一个红外遥控器



6) 在终端中输入 `ir-keytable -c -p NEC -t` 命令，然后使用红外遥控器对着 Orange Pi 开发板的红外接收头按下按键就能在终端中看到接收到的按键编码了

```
root@orangepi:/# ir-keytable -c -p NEC -t
Old keytable cleared
Protocols changed to nec
```



```
Testing events. Please, press CTRL-C to abort.  
5514.329534: lirc protocol(nec): scancode = 0x45c  
5514.329619: event type EV_MSC(0x04): scancode = 0x45c  
5514.329619: event type EV_SYN(0x00).  
5514.832257: lirc protocol(nec): scancode = 0x45c  
5514.832323: event type EV_MSC(0x04): scancode = 0x45c
```

3. 21. linux3.10 红外接收测试

1) 首先安装 ir-keytable 红外测试软件

```
root@orangePi:~# apt update  
root@orangePi:~# apt-get install ir-keytable
```

2) 测试红外接收功能前需要准备一个红外遥控器



3) 在终端中输入 `ir-keytable -t` 命令，然后使用红外遥控器对着 Orange Pi 开发板的红外接收头按下按键就能在终端中看到接收到的按键编码了

```
root@orangePi:~# ir-keytable -t  
Testing events. Please, press CTRL-C to abort.  
1616733050.624909: event type EV_MSC(0x04): scancode = 0x1fb045c  
1616733050.624909: event type EV_SYN(0x00).  
1616733050.812158: event type EV_MSC(0x04): scancode = 0xfb045c
```

3. 22. 硬件看门狗测试

1) 下载 wiringOP 的代码

```
root@orangePi:~# apt update
```




```
root@orangepi:~# apt -y install git
root@orangepi:~# git clone https://github.com/orangepi-xunlong/wiringOP
```

2) 编译 wiringOP

```
root@orangepi:~# cd wiringOP
root@orangepi:~/wiringOP# ./build clean
root@orangepi:~/wiringOP# ./build
```

3) 编译 watchdog 测试程序

```
root@orangepi:~/wiringOP# cd examples/
root@orangepi:~/wiringOP/examples# make watchdog
[CC] watchdog.c
[link]
```

4) 运行看门狗测试程序

- 第二个参数 10 表示看门狗的计数时间，如果这个时间内没有喂狗，系统会重启
- 我们可以通过按下键盘上的任意键（ESC 除外）来喂狗，喂狗后，程序会打印一行 keep alive 表示喂狗成功

```
root@orangepi:~/wiringOP/examples# ./watchdog 10
open success
options is 33152,identity is sunxi-wdt
put_usr return,if 0,success:0
The old reset time is: 16
return ENOTTY,if -1,success:0
return ENOTTY,if -1,success:0
put_user return,if 0,success:0
put_usr return,if 0,success:0
keep alive
keep alive
```

3.23. CSI 摄像头测试

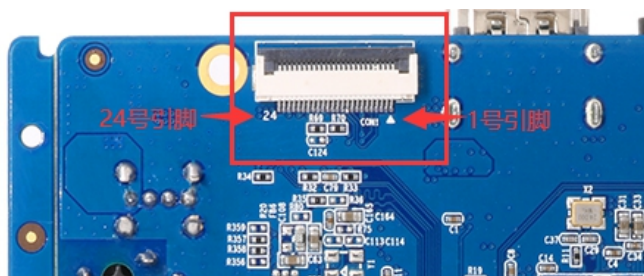
3.23.1. CSI 摄像头接口规格

1) Orange Pi Prime 的 CSI 接口支持 **gc2035** 和 **ov5640** 两款摄像头，不同的系统对摄像头的支持说明如下

	GC2035	OV5640
Linux3.10	支持	支持
Linux5.4	不支持	支持

2) CSI 接口引脚的序号如下图所示

- CSI 接口的 1 号引脚接到摄像头转接板的 24 号引脚
- CSI 接口的 24 号引脚接到摄像头转接板的 1 号引脚



3) CSI接口引脚定义如下

引脚	功能	GPIO 口
CON1-P01	DCIN-5V	
CON1-P02	GND	
CON1-P03	CAM-SDA	PE13
CON1-P04	CSI-PWR-EN	PA17
CON1-P05	CAM-SCK	PE12
CON1-P06	CAM0-RESET	PE14
CON1-P07	CSI-VSYNC	PE3
CON1-P08	CAM0-STBY-EN	PE15
CON1-P09	CSI-HSYNC	PE2
CON1-P10	VDD1V8-CSI	PA17
CON1-P11	AFCC_EN	PC13
CON1-P12	CSI-D7	PE11



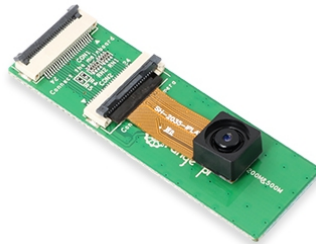
CON1-P13	CSI-MCLK	PE1
CON1-P14	CSI-D6	PE10
CON1-P15	GND	
CON1-P16	CSI-D5	PE9
CON1-P17	CSI-PCLK	PE0
CON1-P18	CSI-D4	PE8
CON1-P19	CSI-D0	PE4
CON1-P20	CSI-D3	PE7
CON1-P21	CSI-D1	PE5
CON1-P22	CSI-D2	PE6
CON1-P23	GND	
CON1-P24	DCIN-5V	

3. 23. 2. Linux3.10 系统 gc2035 摄像头测试

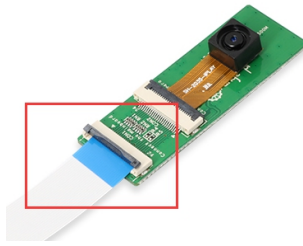
1) Gc2035 摄像头套件包含包含一个 gc2035 摄像头、一个转接板和一根排线



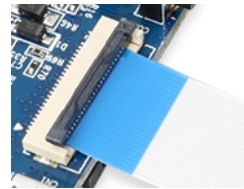
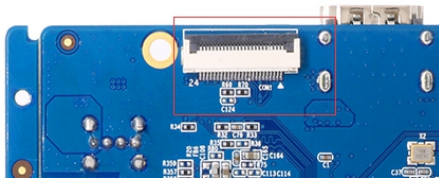
2) 首先将 gc2035 摄像头插到转接板中



3) 然后将排线插入转接板的另一卡槽内



4) 然后把排线的另一头插入开发板的 CIS 摄像头接口中。接好摄像头后再启动 linux 系统（**不要在上电后再插入摄像头**）



5) 然后设置 `/boot/orangepiEnv.txt` 中的 `camera_type` 变量的值为 `gc2035`

```
root@orangepi:~# cat /boot/orangepiEnv.txt
verbosity=1
console=both
disp_mode=1080p60
fb0_width=1280
fb0_height=720
extraargs=cma=25M
camera_type=gc2035
```

6) 然后重启 linux 系统

7) 重启进入系统后首加载 gc2035 摄像头相关的内核模块，命令如下所示

```
root@orangepi:~# rmmod vfe_v4l2 vfe_io
root@orangepi:~# modprobe gc2035
root@orangepi:~# modprobe vfe_v4l2
```

8) 然后通过 `v4l2-ctl`（**注意 v4l2 中的 l 是小写字母 l，不是数字 1**）命令查看下 CSI 摄像头的设备节点，从下面的输出可知摄像头对应的设备节点为 `/dev/video0`

```
root@orangepi:~# apt update
root@orangepi:~# apt -y install v4l-utils
```



```
root@orangePi:~# v4l2-ctl --list-devices
sunxi-vfe (sunxi_vfe vfe.14):
    /dev/video0
```

9) 然后开始安装摄像头测试软件 motion

```
root@orangePi:~# apt update
root@orangePi:~# apt -y install motion
```

10) 修改 `/etc/default/motion` 的配置，将 `start_motion_daemon=no` 修改为 `start_motion_daemon=yes`

```
root@orangePi:~# sed -i "s/start_motion_daemon=no/start_motion_daemon=yes/" \
/etc/default/motion
```

11) 修改 `/etc/motion/motion.conf` 的配置

```
root@orangePi:~# sed -i "s/stream_localhost on/stream_localhost off/" \
/etc/motion/motion.conf
```

12) 然后重新启动 motion 服务

```
root@orangePi:~# /etc/init.d/motion restart
[ ok ] Restarting motion (via systemctl): motion.service.
root@orangePi:~#
```

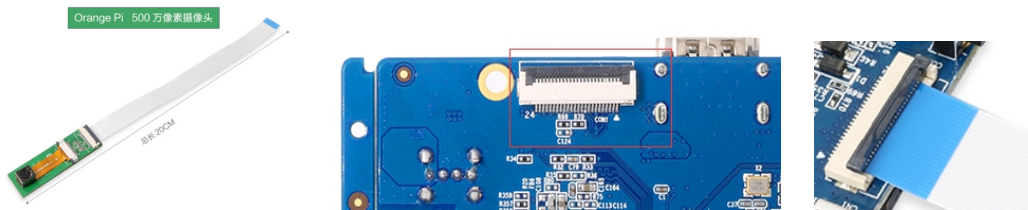
13) 使用 motion 前请先确保 Orange Pi 开发板能正常连接网络, 然后通过 `ifconfig` 命令获取开发板的 IP 地址

14) 然后在火狐浏览器中输入【开发板的 IP 地址:8081】就能看到 gc2035 输出的图像



3. 23. 3. Linux3.10 系统 ov5640 摄像头测试

1) 首先将 ov5640 摄像头转接板通过排线接到开发板的 CIS 摄像头接口上，接好摄像头后再启动 linux 系统（**不要在上电后再插入摄像头**）



2) 然后设置 `/boot/orangepiEnv.txt` 中的 `camera_type` 变量的值为 `ov5640`

```
root@orangepi:~# cat /boot/orangepiEnv.txt
verbosity=1
console=both
disp_mode=1080p60
fb0_width=1280
fb0_height=720
extraargs=cma=25M
camera_type=ov5640
```

3) 然后重启 linux 系统

4) 重启进入系统后首先加载 ov5640 摄像头相关的内核模块，命令如下所示



```
root@orangepi:~# rmmod vfe_v4l2 vfe_io
root@orangepi:~# modprobe ov5640
root@orangepi:~# modprobe vfe_v4l2
```

5) 然后通过 v4l2-ctl (注意 v4l2 中的 l 是小写字母 l, 不是数字 1) 命令查看下 CSI 摄像头的设备节点, 从下面的输出可知摄像头对应的设备节点为 `/dev/video0`

```
root@orangepi:~# apt update
root@orangepi:~# apt install -y v4l-utils
root@orangepi:~# v4l2-ctl --list-devices
sunxi-vfe (sunxi_vfe vfe.14):
    /dev/video0
```

6) 然后开始安装摄像头测试软件 motion

```
root@orangepi:~# apt update
root@orangepi:~# apt install -y motion
```

7) 修改 `/etc/default/motion` 的配置, 将 `start_motion_daemon=no` 修改为 `start_motion_daemon=yes`

```
root@orangepi:~# sed -i "s/start_motion_daemon=no/start_motion_daemon=yes/" \
/etc/default/motion
```

8) 修改 `/etc/motion/motion.conf` 的配置

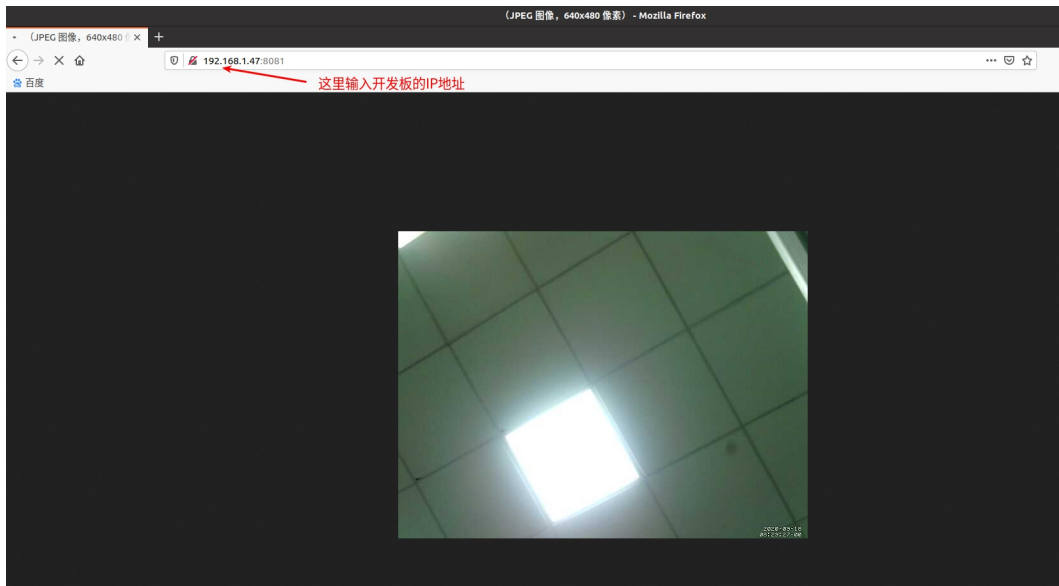
```
root@orangepi:~# sed -i "s/stream_localhost on/stream_localhost off/" \
/etc/motion/motion.conf
```

9) 然后重新启动 motion 服务

```
root@orangepi:~# /etc/init.d/motion restart
[ ok ] Restarting motion (via systemctl): motion.service.
root@orangepi:~#
```

10) 使用 motion 前请先确保 Orange Pi 开发板能正常连接网络, 然后通过 `ifconfig` 命令获取开发板的 IP 地址

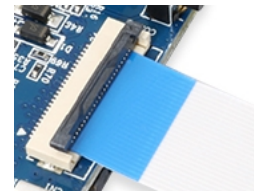
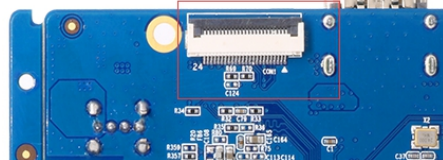
11) 然后在火狐浏览器中输入【开发板的 IP 地址:8081】就能看到 ov5640 输出的图像



3. 23. 4. Linux5.4 系统 ov5640 摄像头测试

1) linux5.4 系统目前只支持 ov5640 摄像头，**不支持 gc2035**

2) 首先将 ov5640 摄像头转接板通过排线接到开发板的 CIS 摄像头接口上，接好摄像头后再启动 linux 系统（**不要在上电后再插入摄像头**）



3) 进入系统后查看下 ov5640 内核模块的加载状态

```
root@orangePi:~# lsmod | grep "ov5640"
ov5640                28672  1
v4l2_fwnode           24576  2 ov5640,sun6i_csi
videodev              151552  7
ov5640,v4l2_fwnode,sunxi_cedrus,videobuf2_common,sun6i_csi,v4l2_mem2mem,videobuf2_v4l2
mc                    36864  7
ov5640,sunxi_cedrus,videobuf2_common,videodev,sun6i_csi,v4l2_mem2mem,videobuf2_v4l2
```

4) 然后通过 v4l2-ctl（**注意 v4l2 中的 l 是小写字母 l，不是数字 1**）命令查看下 CSI



摄像头的设备节点，从下面的输出可知 USB 摄像头对应的设备节点为 **/dev/video0**

```
root@orangePi:~# apt update
root@orangePi:~# apt -y install v4l-utils
root@orangePi:~# v4l2-ctl --list-devices
sun6i-csi (platform:camera):
/dev/video0
```

5) 然后开始安装摄像头测试软件 motion

```
root@orangePi:~# apt update
root@orangePi:~# apt -y install motion
```

6) 修改 **/etc/default/motion** 的配置，将 **start_motion_daemon=no** 修改为 **start_motion_daemon=yes**

```
root@orangePi:~# sed -i "s/start_motion_daemon=no/start_motion_daemon=yes/" \
/etc/default/motion
```

7) 修改 **/etc/motion/motion.conf** 的配置，设置分辨率为 640x480（其他分辨率目前不支持）

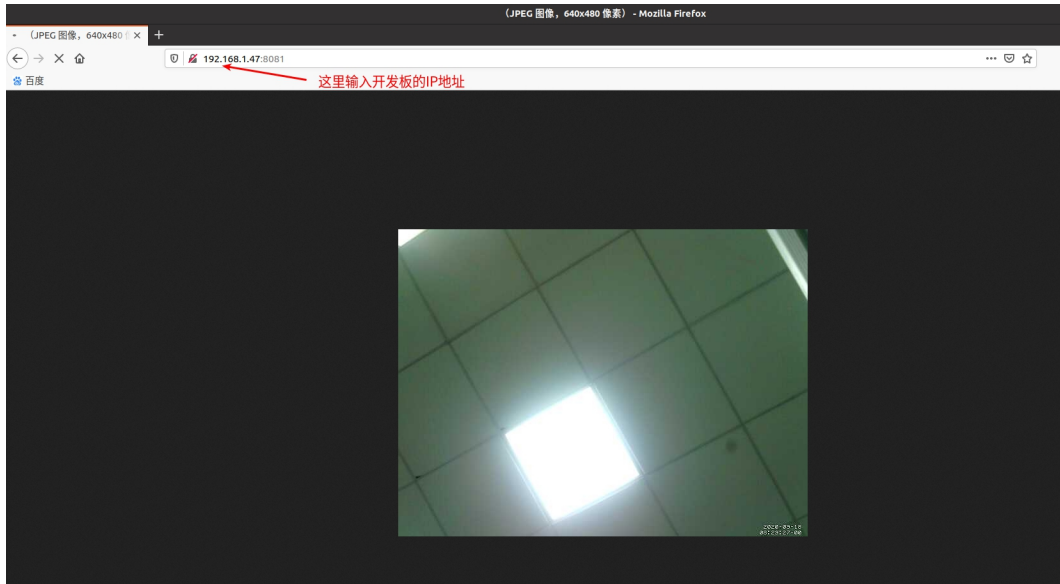
```
root@orangePi:~# sed -i "s/width 320/width 640/" /etc/motion/motion.conf
root@orangePi:~# sed -i "s/height 240/height 480/" /etc/motion/motion.conf
root@orangePi:~# sed -i "s/stream_localhost on/stream_localhost off/" \
/etc/motion/motion.conf
```

8) 然后重新启动 motion 服务

```
root@orangePi:~# /etc/init.d/motion restart
[ ok ] Restarting motion (via systemctl): motion.service.
root@orangePi:~#
```

9) 使用 motion 前请先确保 Orange Pi 开发板能正常连接网络，然后通过 ifconfig 命令获取开发板的 IP 地址

10) 然后在火狐浏览器中输入【开发板的 IP 地址:8081】就能看到 ov5640 输出的图像



3. 24. 40pin 引脚说明

1) Orange Pi Prime开发板 40 pin引脚的顺序请参考下图



2) Orange Pi Prime开发板 40 pin引脚的功能如下表所示

GPIO序号	GPIO	功能	引脚	引脚	功能	GPIO	GPIO序号
		3.3v	1	2	5v		
12	PA12	SDA. 0	3	4	5v		
11	PA11	SCL. 0	5	6	GND		
6	PA6	PWM. 1	7	8	PC5	PC5	69
		GND	9	10	PC6	PC6	70
1	PA1	RXD. 2	11	12	PD14	PD14	110
0	PA0	TXD. 2	13	14	GND		
3	PA3	CTS. 2	15	16	PC4	PC4	68



		3. 3v	17	18	PC7	PC7	71
15	PA15	MOSI. 1	19	20	GND		
16	PA16	MISO. 1	21	22	RTS. 2	PA2	2
14	PA14	SCLK. 1	23	24	CE. 1	PA13	13
		GND	25	26	PC8	PC8	72
19	PA19	SDA. 1	27	28	SCL. 1	PA18	18
7	PA7	PA7	29	30	GND		
8	PA8	PA8	31	32	PC9	PC9	73
9	PA9	PA9	33	34	GND		
10	PA10	PA10	35	36	PC10	PC10	74
107	PA20	PD11	37	38	PC11	PC11	75
		GND	39	40	PC12	PC12	76

3. 25. 安装 wiringOP

1) 下载 wiringOP 的代码

```
root@orangepi:~# apt update
root@orangepi:~# apt -y install git
root@orangepi:~# git clone https://github.com/orangepi-xunlong/wiringOP
```

2) 编译 wiringOP

```
root@orangepi:~# cd wiringOP
root@orangepi:~/wiringOP# ./build clean
root@orangepi:~/wiringOP# ./build
```

3) 测试 gpio readall 命令的输出如下



```
root@orangeprime:~/wiringOP# gpio readall
```

GPIO	wPi	Name	Mode	V	Physical	PRIME	V	Mode	Name	wPi	GPIO
		3.3V			1	2			5V		
12	0	SDA.0	OFF	0	3	4			5V		
11	1	SCL.0	OFF	0	5	6			GND		
6	2	PWM.1	OFF	0	7	8	0	OFF	PC05	3	69
		GND			9	10	0	OFF	PC06	4	70
1	5	RXD.2	OFF	0	11	12	0	OFF	PD14	6	110
0	7	TXD.2	OFF	0	13	14			GND		
3	8	CTS.2	OFF	0	15	16	0	OFF	PC04	9	68
		3.3V			17	18	0	OFF	PC07	10	71
15	11	MOSI.1	OUT	1	19	20			GND		
16	12	MISO.1	OFF	0	21	22	0	OFF	RTS.2	13	2
14	14	SCLK.1	OFF	0	23	24	0	OFF	CE.1	15	13
		GND			25	26	0	OFF	PC08	16	72
19	17	SDA.1	OFF	0	27	28	0	OFF	SCL.1	18	18
7	19	PA07	OFF	0	29	30			GND		
8	20	PA08	OFF	0	31	32	0	OFF	PC09	21	73
9	22	PA09	OFF	0	33	34			GND		
10	23	PA10	OFF	0	35	36	0	OFF	PC10	24	74
107	25	PD11	OFF	0	37	38	0	OFF	PC11	26	75
		GND			39	40	0	OFF	PC12	27	76

3.26. 40pin 的 GPIO、I2C、UART、SPI 测试

wiringOP 已适配 Orange Pi 开发板，使用 wiringOP 可以测试 GPIO、I2C、UART 和 SPI 的功能

开始测试前，请确保已经参照安装 wiringOP 一节编译安装好了 wiringOP

3.26.1. 普通 GPIO 口测试

1) 下面以 7 号引脚——对应 GPIO 为 PA6 ——对应 wPi 序号为 2——为例演示如何设置 GPIO 的高低电平

```
root@orangeprime:~/wiringOP# gpio readall
```

GPIO	wPi	Name	Mode	V	Physical	PRIME	V	Mode	Name	wPi	GPIO
		3.3V			1	2			5V		
12	0	SDA.0	OFF	0	3	4			5V		
11	1	SCL.0	OFF	0	5	6			GND		
6	2	PWM.1	OFF	0	7	8	0	OFF	PC05	3	69
		GND			9	10	0	OFF	PC06	4	70
1	5	RXD.2	OFF	0	11	12	0	OFF	PD14	6	110
0	7	TXD.2	OFF	0	13	14			GND		
3	8	CTS.2	OFF	0	15	16	0	OFF	PC04	9	68
		3.3V			17	18	0	OFF	PC07	10	71



2) 首先设置 GPIO 口为输出模式, 其中第三个参数需要输入引脚对应的 wPi 的序号

```
root@orangepi:~# gpio mode 2 out
```

使用 gpio readall 可以看到 7 号引脚的 Mode 显示为 OUT

```
root@orangepi:~/wiringOP# gpio mode 2 out
root@orangepi:~/wiringOP# gpio readall
```

GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO
		3.3V			1	2		5V		
12	0	SDA.0	OFF	0	3	4		5V		
11	1	SCL.0	OFF	0	5	6		GND		
6	2	PWM.1	OUT	0	7	8	0	PC05	3	69
		GND			9	10	0	PC06	4	70

3) 然后设置 GPIO 口输出低电平, 设置完后可以使用万用表测量引脚的电压的数值, 如果为 0v, 说明设置低电平成功

```
root@orangepi:~# gpio write 2 0
```

使用 gpio readall 可以看到 7 号引脚的值(V)变为了 0

```
root@orangepi:~/wiringOP# gpio write 2 0
root@orangepi:~/wiringOP# gpio readall
```

GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO
		3.3V			1	2		5V		
12	0	SDA.0	OFF	0	3	4		5V		
11	1	SCL.0	OFF	0	5	6		GND		
6	2	PWM.1	OUT	0	7	8	0	PC05	3	69
		GND			9	10	0	PC06	4	70

4) 然后设置 GPIO 口输出高电平, 设置完后可以使用万用表测量引脚的电压的数值, 如果为 3.3v, 说明设置高电平成功

```
root@orangepi:~# gpio write 2 1
```

使用 gpio readall 可以看到 7 号引脚的值(V)变为了 1

```
root@orangepi:~/wiringOP# gpio write 2 1
root@orangepi:~/wiringOP# gpio readall
```

GPIO	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	GPIO
		3.3V			1	2		5V		
12	0	SDA.0	OFF	0	3	4		5V		
11	1	SCL.0	OFF	0	5	6		GND		
6	2	PWM.1	OUT	1	7	8	0	PC05	3	69
		GND			9	10	0	PC06	4	70

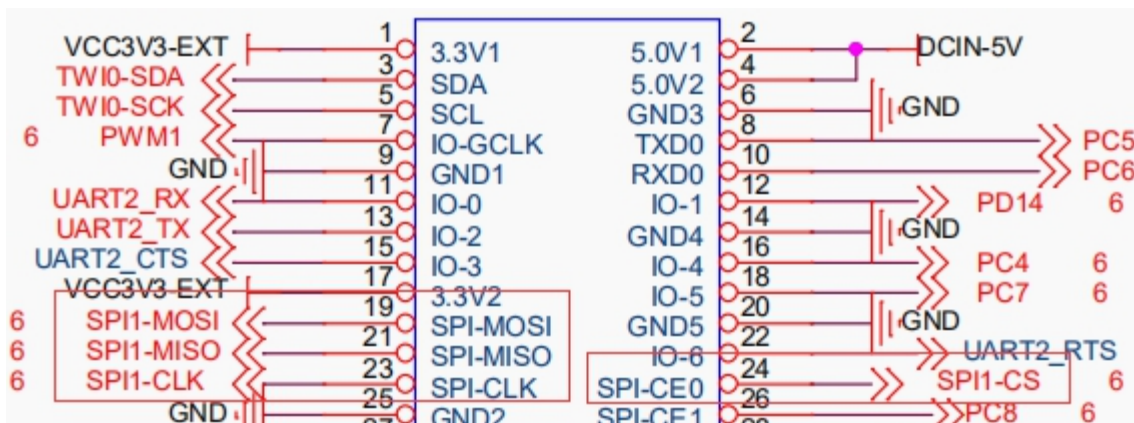


5) 其他引脚的设置方法类似，只需修改 wPi 的序号为引脚对应的序号即可

3.26.2. SPI 接口测试

1) linux5.4 系统在 dts 中默认关闭了 40pin 中的 spi 控制器，如果需要使用 spi，首先需要先打开 spi 的配置，linux3.10 系统默认都是打开的，不需要额外的配置，linux5.4 系统 spi 的打开方法如下所示：

a. 由 40pin 的原理图可知，Orange Pi Prime 可用的 spi 为 spi1



b. 然后在 /boot/orangepiEnv.txt 中设置 overlays=spi-spidev，设置 param_spidev_spi_bus=1，其中 1 代表的就是 spi1

overlays=spi-spidev

param_spidev_spi_bus=1 #修改为开发板所支持的对应的 spi 总线号

c. 然后重启系统，启动时，在 u-boot 的启动 log 中可以看到 SPI DT overlays 的配置输出

```
788 bytes read in 8 ms (95.7 KiB/s)
```

```
Applying kernel provided DT overlay sun50i-h5-spi-spidev.dtb
```

```
4185 bytes read in 8 ms (510.7 KiB/s)
```

```
Applying kernel provided DT fixup script (sun50i-h5-fixup.scr)
```

d. 系统启动后，在 /dev 下如果可以看到 SPI 的设备节点就说明配置正确

```
root@orangepi:~# ls /dev/spi*
```

```
/dev/spidev1.0
```

2) 然后编译 spidev_test 测试程序

a. linux5.4 系统的编译命令为

```
root@orangepi:~/wiringOP/examples# make spidev_test
```

```
[CC] spidev_test.c
```

[\[link\]](#)

b. linux3.10 系统的编译命令为

```
root@orangepi:~/wiringOP/examples# make spidev_test_linux3_4
```

```
[CC] spidev_test.c
```

[\[link\]](#)

3) 先不短接 spi 的 mosi 和 miso 两个引脚，运行 spidev_test 的输出结果如下所示，可以看到发送和接收的数据不一致

a. linux5.4 系统的测试命令和结果为

```
root@orangepi:~/wiringOP/examples# ./spidev_test -v -D /dev/spidev1.0
```

```
spi mode: 0x0
```

```
bits per word: 8
```

```
max speed: 500000 Hz (500 KHz)
```

```
TX | FF FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF
```

```
FF FF FF FF FF FF F0 0D | .....@.....
```

```
RX | FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
```

```
FF FF FF FF FF FF FF FF | .....
```

b. linux3.10 系统的测试命令和结果为

```
root@orangepi:~/wiringOP/examples# ./spidev_test_linux3_4 -D /dev/spidev1.0
```

```
spi mode: 0
```

```
bits per word: 8
```

```
max speed: 500000 Hz (500 KHz)
```

```
00 00 00 00 00 00 00
```

```
00 00 00 00 00 00 00
```

```
00 00 00 00 00 00 00
```

```
00 00 00 00 00 00 00
```

```
00 00 00 00 00 00 00
```

```
00 00 00 00 00 00 00
```

```
00 00
```

4) 然后使用杜邦线短接 spi 的 mosi（对应 19 号引脚）和 miso（对应 21 号引脚）两个引脚，重新测试的输出如下，可以看到发送和接收的数据一样，说明 spi 能正常使用

a. linux5.4 系统的测试命令和结果为



```

root@orangepi:~/wiringOP/examples# ./spidev_test -v -D /dev/spidev1.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D | .....@.....
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF F0 0D | .....@.....

```

b. linux3.10 系统的测试命令和结果为

```

root@orangepi:~/wiringOP/examples# ./spidev_test_linux3_4 -D /dev/spidev1.0
spi mode: 0
bits per word: 8
max speed: 500000 Hz (500 KHz)

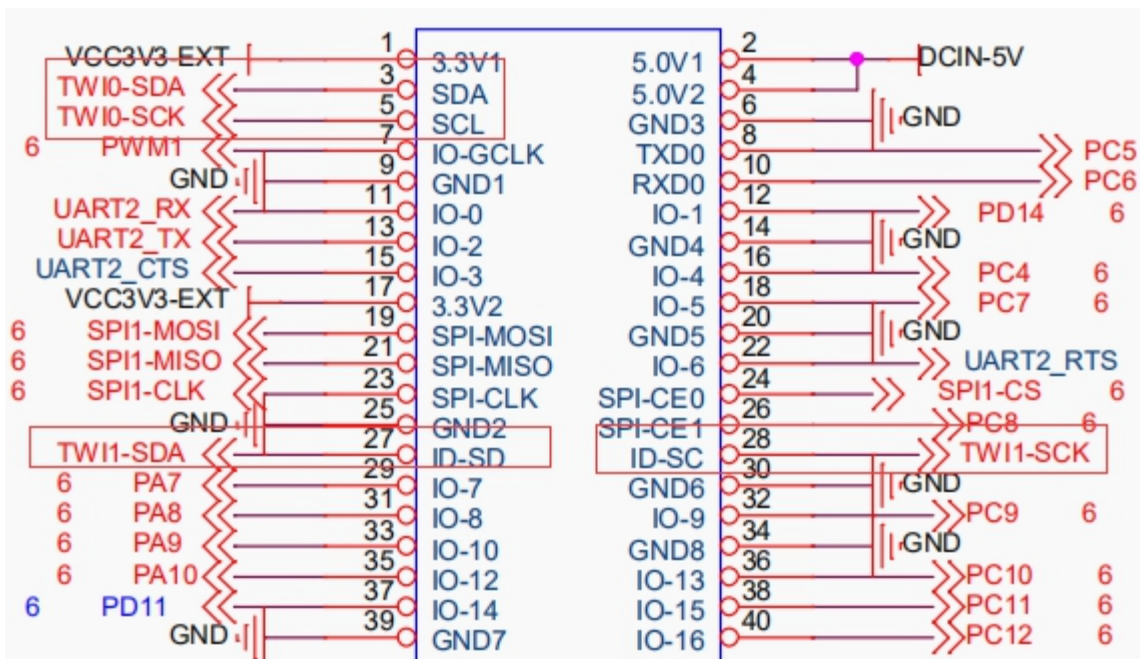
FF FF FF FF FF FF
40 00 00 00 00 95
FF FF FF FF FF FF
FF FF FF FF FF FF
FF FF FF FF FF FF
DE AD BE EF BA AD
F0 0D

```

3. 26. 3. I2C 测试

1) linux5.4 系统在 dts 中默认关闭了 40pin 中的 i2c 控制器, 如果需要使用 i2c, 首先需要先打开 i2c 的配置, linux3.10 系统默认都是打开的, 不需要额外的配置, linux5.4 系统 i2c 的打开方法如下所示:

c. 由 40pin 的原理图可知, Orange Pi Prime 可用的 i2c 为 i2c0 和 i2c1



- d. 然后在 `/boot/orangepiEnv.txt` 中设置 `overlays=i2c0 i2c1` 就可以同时打开 `i2c0` 和 `i2c1` 的配置

```
overlays=i2c0 i2c1
```

- e. 然后重启系统，启动时，在 `u-boot` 的启动 `log` 中可以看到 `I2C DT overlays` 的配置输出

```
Found mainline kernel configuration
29940 bytes read in 6 ms (4.8 MiB/s)
374 bytes read in 8 ms (44.9 KiB/s)
Applying kernel provided DT overlay sun50i-h5-i2c0.dtbo
374 bytes read in 8 ms (44.9 KiB/s)
Applying kernel provided DT overlay sun50i-h50-i2c1.dtbo
```

- f. 系统启动后，在 `/dev` 下如果多了两个 `i2c` 的设备节点就说明配置正确

```
root@orangepi:~# ls /dev/i2c*
/dev/i2c-0 /dev/i2c-1 /dev/i2c-2 /dev/i2c-3 /dev/i2c-4
```

- g. 不同的 `i2c` 设备节点的对应关系如下所示，其中

- 40pin 中的 `i2c0` 对应 `/dev/i2c-0`
- 40pin 中的 `i2c1` 对应 `/dev/i2c-1`

```
root@orangepi:~# ls /sys/class/i2c-adapter/ -lh
total 0
lrwxrwxrwx 1 root root 0 Nov 16 12:12 i2c-0 -> ../../devices/platform/soc/1c2ac00.i2c/i2c-0
lrwxrwxrwx 1 root root 0 Nov 16 12:12 i2c-1 -> ../../devices/platform/soc/1c2b000.i2c/i2c-1
lrwxrwxrwx 1 root root 0 Nov 16 12:12 i2c-2 -> ../../devices/platform/soc/1c2b400.i2c/i2c-2
lrwxrwxrwx 1 root root 0 Nov 16 12:12 i2c-3 -> ../../devices/platform/soc/1f02400.i2c/i2c-3
lrwxrwxrwx 1 root root 0 Nov 16 12:13 i2c-4 -> ../../devices/platform/soc/1ee0000.hdma/i2c-4
```



2) 然后开始测试 i2c，首先安装 i2c-tools

```
root@orangepi:~# apt update
root@orangepi:~# apt -y install i2c-tools
```

3) 然后在 40pin 的 i2c0 或者 i2c1 上接上一个 i2c 设备

	i2c0	i2c1
sda 引脚	对应 3 号引脚	对应 27 号引脚
sck 引脚	对应 5 号引脚	对应 28 号引脚
vcc 引脚	对应 1 号引脚	对应 17 号引脚
gnd 引脚	对应 6 号引脚	对应 25 号引脚

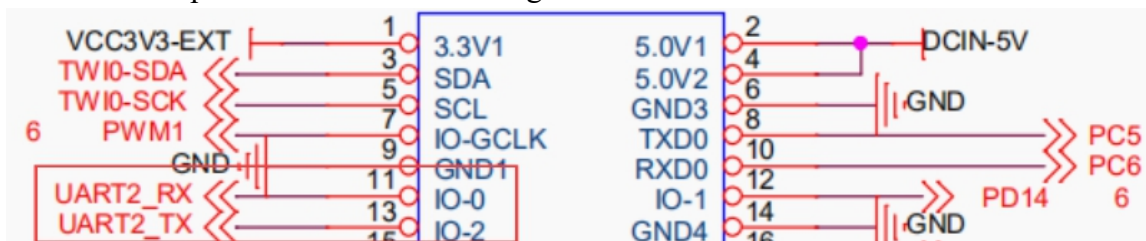
4) 然后使用 `i2cdetect -y 0`（其中 0 表示 i2c0，i2c1 需要修改为 `i2cdetect -y 1`）命令如果能检测到连接的 i2c 设备的地址，就说明 i2c 能正常使用

```
root@orangepiprime:~# i2cdetect -y 0
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
30:  -- -- -- -- -- -- 38 -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

3.26.4. UART 测试

1) linux5.4 系统在 dts 中默认关闭了 40pin 中的 uart 控制器，如果需要使用 uart，首先需要先打开 uart 的配置，linux3.10 系统默认都是打开的，不需要额外的配置，linux5.4 系统 uart 的打开方法如下所示：

a. 由 40pin 的原理图可知，Orange Pi Prime 可用的 uart2





- b. 然后在 `/boot/orangepiEnv.txt` 中设置 `overlays=uart2` 就可以打开 `uart2` 的配置

`overlays=uart2`

- c. 然后重启系统, 启动时, 在 u-boot 的启动 log 中可以看到 UART DT overlays 的配置输出

Applying kernel provided DT overlay **sun50i-h5-uart2.dtbo**

502 bytes read in 5 ms (97.7 KiB/s)

Applying kernel provided DT fixup script (sun50i-h5-fixup.scr)

- d. 系统启动后, 在 `/sys/class/tty` 下可以看到 `ttyS0`、`ttyS1` 和 `ttyS2` 的信息, 其中

- a) 40pin 中的 `uart2` 对应 `/dev/ttyS2`

```
root@orangepiprime:~# ls /sys/class/tty/ttyS* -l
lrwxrwxrwx 1 root root 0 Nov 17 06:57 /sys/class/tty/ttyS0 -> ../../devices/platform/soc/1c28000.serial/tty/ttyS0
lrwxrwxrwx 1 root root 0 Nov 17 06:57 /sys/class/tty/ttyS1 -> ../../devices/platform/soc/1c28400.serial/tty/ttyS1
lrwxrwxrwx 1 root root 0 Nov 17 06:57 /sys/class/tty/ttyS2 -> ../../devices/platform/soc/1c28800.serial/tty/ttyS2
```

- 2) 然后开始测试 `uart` 接口, 先使用杜邦线短接要测试的 `uart` 接口的 `rx` 和 `tx`

	uart2
tx 引脚	对应 13 号引脚
rx 引脚	对应 11 号引脚

- 3) 然后修改 `wiringOP` 中的串口测试程序 `serialTest`

```
int main ()
{
    int fd ;
    int count ;
    unsigned int nextTime ;

    if ((fd = serialOpen ("/dev/ttyS2", 115200)) < 0)
    {
        fprintf (stderr, "Unable to open serial device: %s\n", strerror (errno)) ;
        return 1 ;
    }
}
```

修改为要测试的串口对应的设备节点名

- 4) 再编译 `wiringOP` 中的串口测试程序 `serialTest`

```
root@orangepi:~/wiringOP/examples# make serialTest
```

```
[CC] serialTest.c
```

```
[link]
```

```
root@orangepi:~/wiringOP/examples#
```

- 5) 最后运行 `serialTest`, 如果能看到下面的打印, 说明串口通信正常

```
root@orangepi:~/wiringOP/examples# ./serialTest
```



```

Out:  0:  ->  0
Out:  1:  ->  1
Out:  2:  ->  2
Out:  3:  ->  3
Out:  4:  ->  4
Out:  5:  ->  5
Out:  6:  ->  6
Out:  7:  ->  7
Out:  8:  ->  8^C

```

3. 27. SPI LCD 显示屏使用方法

注意：此方法只适用于 **linux3.10** 内核的系统，**linux5.4** 内核的系统无法使用

3. 27. 1. 2.4 寸 SPI LCD 显示屏

1) 测试的 LCD 显示屏详情页链接如下

http://www.lcdwiki.com/2.4inch_SPI_Module_ILI9341_SKU:MSP2402

2) LCD 显示屏和开发板的接线方式如下所示

TFT SPI 模块引脚	开发板 26pin 对应的引脚	GPIO -- GPIO num
VCC	2 号引脚	
GND	6 号引脚	
CS	24 号引脚	
RESET	8 号引脚	PC5 -- 69
D/C	10 号引脚	PC6 -- 70
SDI(MOSI)	19 号引脚	
SCK	23 号引脚	
LED	1 号引脚	
SDO(MISO)	21 引脚	

3) 将显示屏接到开发板后，再使用下面的命令加载 **fbtft_device** 内核模块

```

root@orangePi:~# modprobe fbtft_device custom name=fb_ili9341 busnum=1 cs=0
gpios=reset:69,dc:70 rotate=90 speed=16000000 bgr=1

```




4) **fbtft_device** 内核模块加载时 **dmesg** 命令正确的输出 log 如下所示, 而且由 log 可以知道 LCD 显示屏使用的 framebuffer 为 **fb8**

```
root@orangepi:~# dmesg | tail
[ 55.702608] fbtft_device: SPI devices registered:
[ 55.702625] fbtft_device:      dh2228fv spi1.0 8000kHz 8 bits mode=0x00
[ 55.702631] fbtft_device: 'fb' Platform devices registered:
[ 55.702806] fbtft_device: Deleting spi1.0
[ 55.703339] fbtft_device: GPIOs used by 'fb_ili9341':
[ 55.703347] fbtft_device:   'reset' = GPIO69
[ 55.703352] fbtft_device:   'dc' = GPIO70
[ 55.703357] fbtft_device: SPI devices registered:
[ 55.703364] fbtft_device:      fb_ili9341 spi1.0 16000kHz 8 bits mode=0x00
[ 56.312655] graphics fb8: fb_ili9341 frame buffer, 320x240, 150 KiB video memory,
16 KiB DMA buffer memory, fps=20, spi1.0 at 16 MHz
```

5) 然后使用下面的命令就可以在 LCD 显示屏上显示 Orange Pi 的 logo 图片

```
root@orangepi:~# apt update
root@orangepi:~# apt -y install fbi
root@orangepi:~# fbi -vt 1 -noverbose -d /dev/fb8 /boot/boot.bmp
```





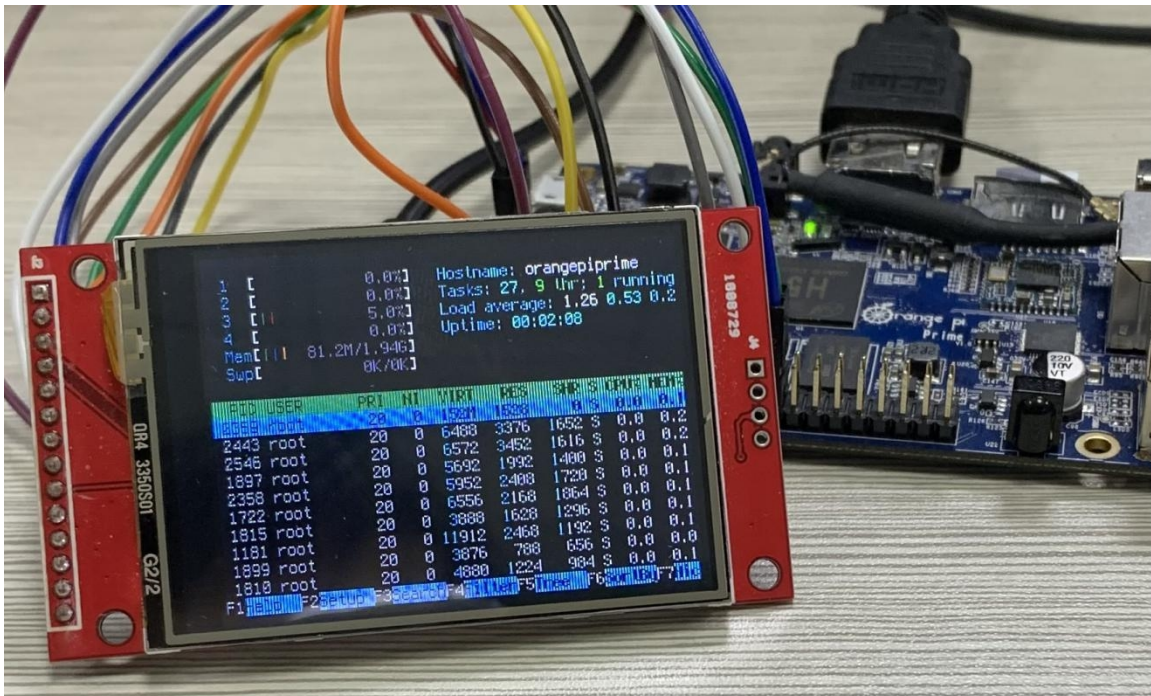
6) 还可以将 tty1 的输出映射到 LCD 显示屏的 fb 设备——**fb8**，映射完后，HDMI 就不会再有图像输出了

```
root@orangepi:~# con2fbmap 1 8
```

如果要切换回 HDMI 显示，请使用下面的命令

```
root@orangepi:~# con2fbmap 1 0
```

下面是运行 htop 命令的输出



7) 由于默认的终端字体太大，导致显示屏无法显示太多的内容，可以通过下面的方法来缩小终端的字体

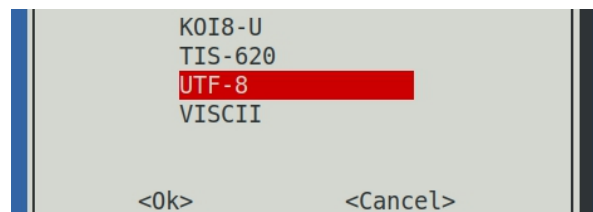
a. 首先运行 **dpkg-reconfigure console-setup**

```
root@orangepi:~# apt-get update
```

```
root@orangepi:~# apt-get install kbd
```

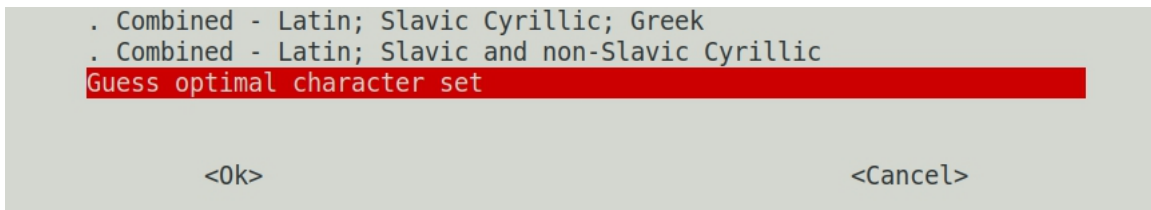
```
root@orangepi:~# dpkg-reconfigure console-setup
```

b. 终端编码选择 **UTF-8**

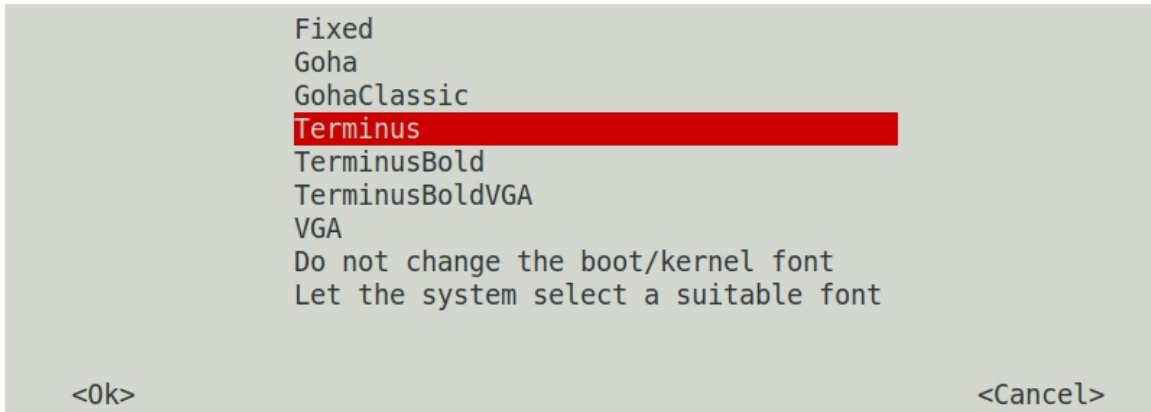




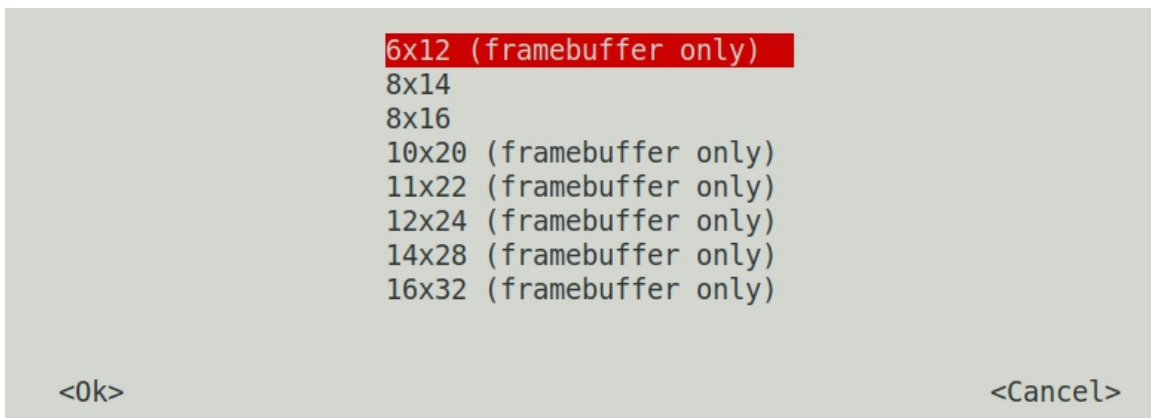
- c. 然后选择 **Guess optimal character set**



- d. 然后选择 **Terminus**



- e. 最后选择字体大小为 6x12



- f. 设置完后就能看到 LCD 显示屏上的字体变小了

8) 设置系统启动自动加载 fbft_device 模块的方法

- a. 新建 **/etc/modules-load.d/fbft.conf** 配置文件，文件内容如下所示

```
root@orangepi:~# cat /etc/modules-load.d/fbft.conf
fbft_device
```

- b. 新建 **/etc/modprobe.d/fbft.conf** 配置文件，文件内容如下所示

```
root@orangepi:~# cat /etc/modprobe.d/fbft.conf
options fbft_device custom name=fb_ili9341 busnum=1 cs=0 gpios=reset:69,dc:70
```




```
rotate=90 speed=16000000 bgr=1
```

c. 然后重启 linux 系统就能看到 fbftf_device 相关的内核模块都已自动加载

9) 如果希望 linux 系统启动后自动将 console 映射到 LCD 显示屏，请在 **/boot/orangepiEnv.txt** 中加入下面的配置，然后重启系统就能看到 LCD 显示屏有输出了

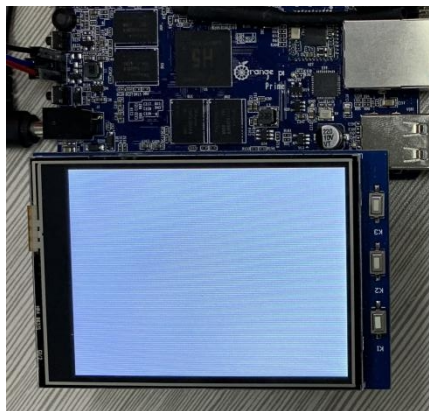
```
root@orangepi:~# cat /boot/orangepiEnv.txt | grep "fbcon"
extraargs=cma=25M fbcon=map:8
```

3.27.2. 3.2 寸 RPi SPI LCD 显示屏

1) 测试的 LCD 显示屏详情页链接如下

http://www.lcdwiki.com/3.2inch_RPi_Display

2) LCD 显示屏和开发板接线方式如下所示



3) 将 LCD 显示屏接到开发板后，再使用下面的命令加载 fbftf_device 内核模块

```
root@orangepi:~# modprobe fbftf_device custom name=fb_ili9341 busnum=1 cs=0
gpios=reset:0,dc:3 rotate=90 speed=16000000 bgr=1
```

4) fbftf_device 内核模块加载时 dmesg 命令正确的输出 log 如下所示，而且由 log 可以知道 LCD 屏幕使用的 framebuffer 为 fb8

```
root@orangepi:~# dmesg | tail
[ 117.376735] fbftf_device: SPI devices registered:
[ 117.376752] fbftf_device: 'fb' Platform devices registered:
[ 117.377098] fbftf_device: GPIOs used by 'fb_ili9341':
```



```
[ 117.377105] fbtft_device: 'reset' = GPIO0
[ 117.377110] fbtft_device: 'dc' = GPIO3
[ 117.377115] fbtft_device: SPI devices registered:
[ 117.377122] fbtft_device: fb_ili9341 spi1.0 16000kHz 8 bits mode=0x00
[ 117.713159] graphics fb8: fb_ili9341 frame buffer, 320x240, 150 KiB video memory,
16 KiB buffer memory, fps=20, spi1.0 at 16 MHz
```

5) 然后使用下面的命令就可以在 LCD 屏幕上显示 Orange Pi 的 logo 图片

```
root@orangepi:~# apt update
root@orangepi:~# apt -y install fbi
root@orangepi:~# fbi -vt 1 -noverbose -d /dev/fb8 /boot/boot.bmp
```



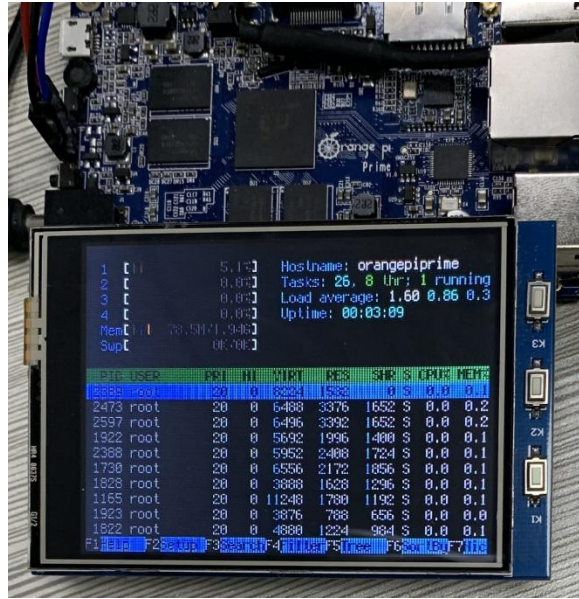
6) 还可以将 tty1 的输出映射到 LCD 屏幕的 fb 设备——fb8, 映射完后, HDMI 就不会再有图像输出了

```
root@orangepi:~# con2fbmap 1 8
```

如果要切换回 HDMI 显示, 请使用下面的命令

```
root@orangepi:~# con2fbmap 1 0
```

下面是运行 htop 命令的输出



7) 由于默认的终端字体太大，导致屏幕无法显示太多的内容，可以通过下面的方法来缩小终端的字体

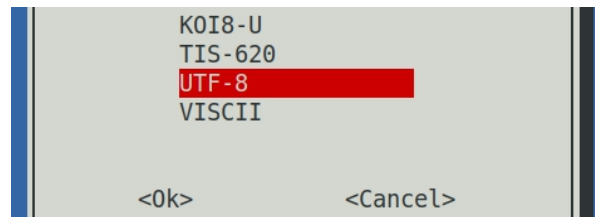
a. 首先运行 **dpkg-reconfigure console-setup**

```
root@orangepi:~# apt-get update
```

```
root@orangepi:~# apt-get install kbd
```

```
root@orangepi:~# dpkg-reconfigure console-setup
```

b. 终端编码选择 **UTF-8**



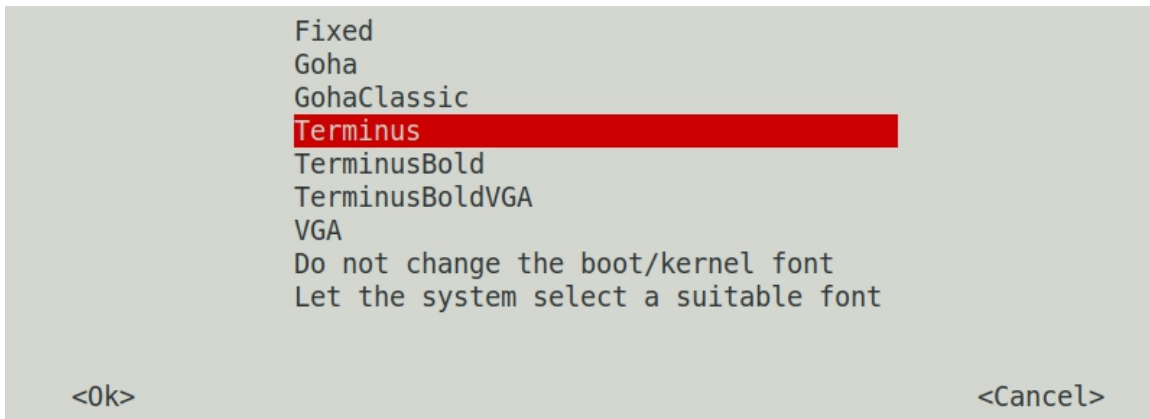
c. 然后选择 **Guess optimal character set**

```
. Combined - Latin; Slavic Cyrillic; Greek
. Combined - Latin; Slavic and non-Slavic Cyrillic
Guess optimal character set
```

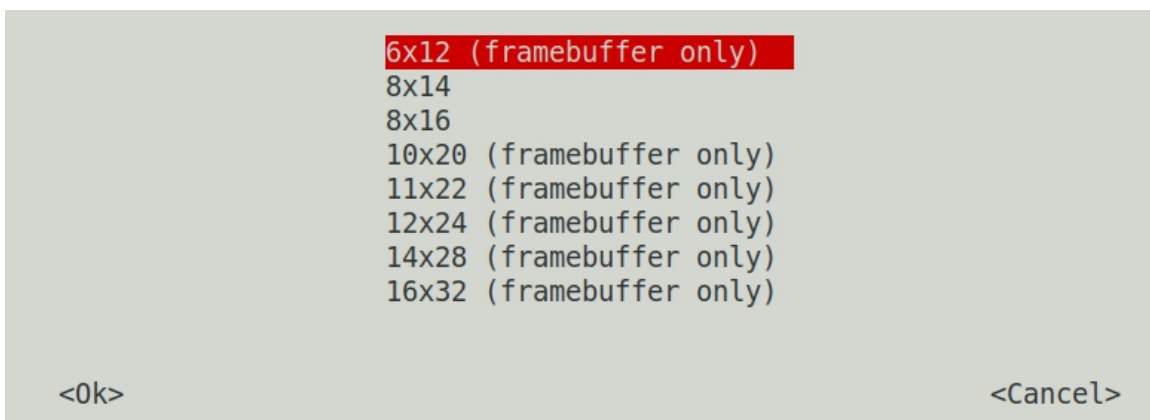
<0k>

<Cancel>

d. 然后选择 **Terminus**



e. 最后选择字体大小为 6x12



f. 设置完后就能看到 LCD 屏幕上的字体变小了

8) 设置系统启动自动加载 fbtf_device 模块的方法

a. 新建/etc/modules-load.d/fbtf.conf 配置文件，文件内容如下所示

```
root@orangePi:~# cat /etc/modules-load.d/fbtf.conf
fbtf_device
```

b. 新建/etc/modprobe.d/fbtf.conf 配置文件，文件内容如下所示

```
root@orangePi:~# cat /etc/modprobe.d/fbtf.conf
options fbtf_device custom name=fb_ili9341 busnum=1 cs=0 gprios=reset:0,dc:3
rotate=90 speed=16000000 bgr=1
```

c. 然后重启 linux 系统就能看到 fbtf_device 相关的内核模块都已自动加载

9) 如果希望 linux 系统启动后自动将 console 映射到 LCD 显示屏，请在 /boot/orangepiEnv.txt 中加入下面的配置，然后重启系统就能看到 LCD 显示屏有输出了

```
root@orangePi:~# cat /boot/orangepiEnv.txt | grep "fbcon"
```



```
extraargs=cma=25M fbcon=map:8
```

3. 27. 3. 3.5 寸 SPI LCD 显示屏

1) 测试的 LCD 显示屏详情页链接如下

http://www.lcdwiki.com/3.5inch_SPI_Module_ILI9488_SKU:MSP3520

2) LCD 显示屏和开发板的接线方式如下所示

TFT SPI 模块引脚	开发板 26pin 对应的引脚	GPIO -- GPIO num
VCC	2 号引脚	
GND	6 号引脚	
CS	24 号引脚	
RESET	8 号引脚	PC5 -- 69
DC/RS	10 号引脚	PC6 -- 70
SDI(MOSI)	19 号引脚	
SCK	23 号引脚	
LED	12 号引脚	PD14 -- 110
SDO(MISO)	21 引脚	

3) 将显示屏接到开发板后，再使用下面的命令加载 **fbtft_device** 内核模块

```
root@orangepi:~# modprobe fbtft_device custom name=fb_ili9488 busnum=1 cs=0
gpios=reset:69,dc:70,led:110 rotate=270 speed=16000000 bgr=1
```

4) **fbtft_device** 内核模块加载时 **dmesg** 命令正确的输出 log 如下所示，而且由 log 可以知道 LCD 显示屏使用的 framebuffer 为 **fb8**

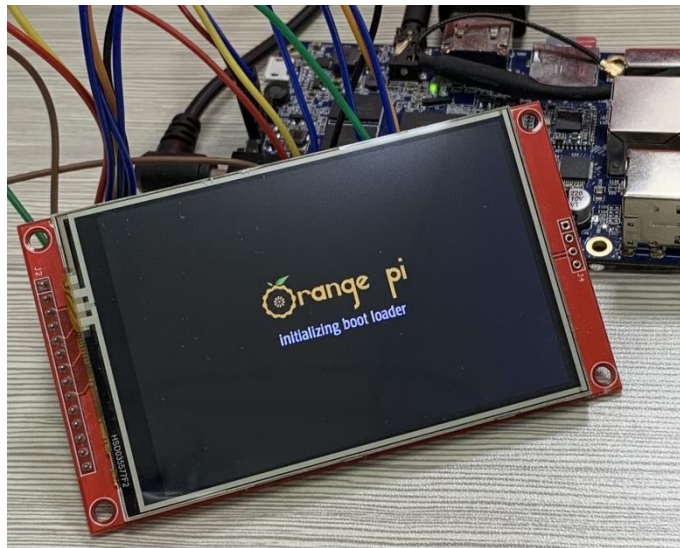
```
root@orangepi:~# dmesg | tail
[ 170.676628] fbtft_device: SPI devices registered:
[ 170.676657] fbtft_device: 'fb' Platform devices registered:
[ 170.679720] fbtft_device: GPIOs used by 'fb_ili9488':
[ 170.679746] fbtft_device: 'reset' = GPIO69
[ 170.679758] fbtft_device: 'dc' = GPIO70
[ 170.679769] fbtft_device: 'led' = GPIO110
[ 170.679779] fbtft_device: SPI devices registered:
[ 170.679796] fbtft_device: fb_ili9488 spi1.0 25000kHz 8 bits mode=0x00
[ 171.117937] graphics fb8: fb_ili9488 frame buffer, 480x320, 300 KiB video memory,
```



```
16 KiB buffer memory, fps=100, spi1.0 at 16MHz
```

5) 然后使用下面的命令就可以在 LCD 显示屏上显示 Orange Pi 的 logo 图片

```
root@orangepi:~# apt update
root@orangepi:~# apt -y install fbi
root@orangepi:~# fbi -vt 1 -noverbose -d /dev/fb8 /boot/boot.bmp
```

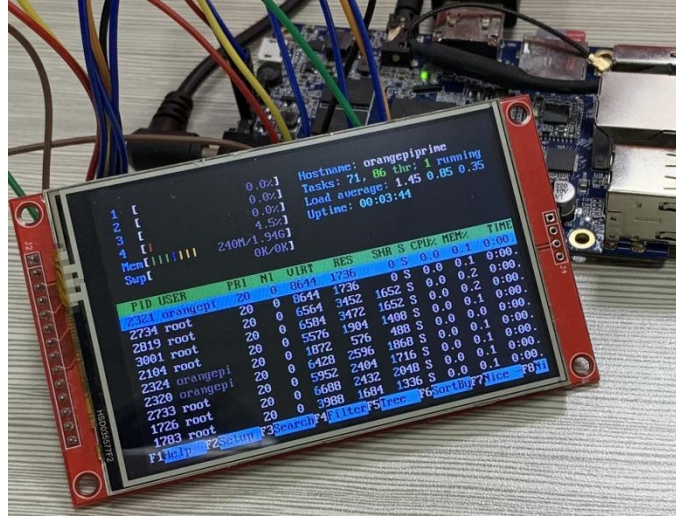


6) 还可以将 tty1 的输出映射到 LCD 显示屏的 fb 设备——**fb8**，映射完后，LCD 屏幕将会显示终端的输出，HDMI 就不会再有图像输出了

```
root@orangepi:~# con2fbmap 1 8
```

如果要切换回 HDMI 显示，请使用下面的命令

```
root@orangepi:~# con2fbmap 1 0
```

7) 设置系统启动自动加载 fbtf_device 模块的方法

- a. 新建/etc/modules-load.d/fbtf.conf 配置文件，文件内容如下所示

```
root@orange:~# cat /etc/modules-load.d/fbtf.conf
fbtf_device
```

- b. 新建/etc/modprobe.d/fbtf.conf 配置文件，文件内容如下所示

```
root@orange:~# cat /etc/modprobe.d/fbtf.conf
options fbtf_device custom name=fb_ili9488 busnum=1 cs=0
gpios=reset:69,dc:70,led:110 rotate=270 speed=16000000 bgr=1
```

- c. 然后重启 linux 系统就能看到 fbtf_device 相关的内核模块都已自动加载

8) 如果希望 linux 系统启动后自动将 console 映射到 LCD 显示屏，请在 /boot/orangepiEnv.txt 中加入下面的配置，然后重启系统就能看到 LCD 显示屏有输出了

```
root@orange:~# cat /boot/orangepiEnv.txt | grep "fbcon"
extraargs=cma=25M fbcon=map:8
```




4. Linux SDK 使用说明

Linux SDK 的编译都是在安装有 **Ubuntu 18.04 的 PC 或者虚拟机 (VirtualBox 或 VMware)** 上进行的, 请不要使用其他版本的 Ubuntu 系统或者在 WSL 上编译 Linux SDK

4.1. 获取 linux sdk 的源码

4.1.1. 从 github 下载 orangepi-build

1) 首先下载 orangepi-build 的代码, orangepi-build 的代码是基于 armbian build 编译系统修改而来的, 目前 H5 系列开发板已经支持 legacy 分支和 current 分支

```
test@test:~$ sudo apt update
test@test:~$ sudo apt install git
test@test:~$ git clone https://github.com/orangepi-xunlong/orangepi-build.git
```

2) legacy 分支一般使用 BSP 版本的 u-boot 和内核, current 分支一般使用接近主线版本的 u-boot 和内核, H5 系列开发板当前使用的 u-boot 和 linux 内核如下所示

分支	u-boot 版本	linux 内核版本
legacy	u-boot 2014.07	linux3.10.65
current	u-boot 2020.04	linux5.4.65

3) orangepi-build 下载完后会包含下面的文件和文件夹

- a. **build.sh**: 编译启动脚本
- b. **external**: 包含编译镜像需要用的配置文件、特定的脚本以及部分程序的源码等
- c. **LICENSE**: GPL 2 许可证文件
- d. **README.md**: orangepi-build 说明文件
- e. **scripts**: 编译 linux 镜像的通用脚本

```
test@test:~/orangepi-build$ ls
build.sh  external  LICENSE  README.md  scripts
```



4.1.2. 下载交叉编译工具链

1) orangepi-build 第一次运行的时候会自动下载交叉编译工具链放在 **toolchains** 文件夹中，每次运行 orangepi-build 的 build.sh 脚本后，都会检查 **toolchains** 中的交叉编译工具链是否都存在，如果不存在则会重新开始下载，如果存在则直接使用，不会重复下载

```
[ o.k. ] Checking for external GCC compilers
[ ... ] downloading using http(s) network [ gcc-linaro-aarch64-none-elf-4.8-2013.11_linux.tar.xz ]
#8d7029 16MiB/24MiB(65%) CN:1 DL:7.9MiB ETA:1s]
[ o.k. ] Verified [ PGP ]
[ ... ] decompressing
[ ... ] gcc-linaro-aarch64-none-elf-4.8-2013.11_linux.tar.xz: 24.9MiB [14.4MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-linaro-arm-none-eabi-4.8-2014.04_linux.tar.xz ]
#e38eec 17MiB/33MiB(50%) CN:1 DL:10MiB ETA:1s]
[ o.k. ] Verified [ PGP ]
[ ... ] decompressing
[ ... ] gcc-linaro-arm-none-eabi-4.8-2014.04_linux.tar.xz: 33.9MiB [9.6MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-linaro-arm-linux-gnueabi-4.8-2014.04_linux.tar.xz ]
#041c24 48MiB/48MiB(99%) CN:1 DL:2.7MiB]
[ o.k. ] Verified [ PGP ]
[ ... ] decompressing
[ ... ] gcc-linaro-arm-linux-gnueabi-4.8-2014.04_linux.tar.xz: 48.8MiB [13.6MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi.tar.xz ]
#3dee3e 72MiB/76MiB(93%) CN:1 DL:3.7MiB ETA:1s]
[ o.k. ] Verified [ MD5 ]
[ ... ] decompressing
[ ... ] gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi.tar.xz: 77.0MiB [14.2MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi.tar.xz ]
#42e728 104MiB/104MiB(99%) CN:1 DL:2.0MiB]
[ o.k. ] Verified [ MD5 ]
[ ... ] decompressing
[ ... ] gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi.tar.xz: 104MiB [13.9MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu.tar.xz ]
#2c865e 108MiB/111MiB(97%) CN:1 DL:3.9MiB]
[ o.k. ] Verified [ MD5 ]
[ ... ] decompressing
[ ... ] gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu.tar.xz: 111MiB [13.4MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi.tar.xz ]
#d232ee 250MiB/251MiB(99%) CN:1 DL:2.0MiB]
[ o.k. ] Verified [ MD5 ]
[ ... ] decompressing
[ ... ] gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi.tar.xz: 251MiB [13.7MiB/s] [=====] 100%
[ ... ] downloading using http(s) network [ gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz ]
#88b441 268MiB/269MiB(99%) CN:1 DL:0.9MiB]
[ o.k. ] Verified [ MD5 ]
[ ... ] decompressing
```

2) 交叉编译工具链在中国境内的镜像网址为清华大学的开源软件镜像站

https://mirrors.tuna.tsinghua.edu.cn/armbian-releases/_toolchain/

3) **toolchains** 下载完后会包含多个版本的交叉编译工具链

```
test@test:~/orangepi-build$ ls toolchains/
gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu/
gcc-arm-9.2-2019.12-x86_64-arm-none-linux-gnueabi/
gcc-linaro-4.9.4-2017.01-x86_64_aarch64-linux-gnu/
gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi/
gcc-linaro-5.5.0-2017.10-x86_64_arm-linux-gnueabi/
gcc-linaro-7.4.1-2019.02-x86_64_aarch64-linux-gnu/
gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabi/
gcc-linaro-aarch64-none-elf-4.8-2013.11_linux/
gcc-linaro-arm-linux-gnueabi-4.8-2014.04_linux/
gcc-linaro-arm-none-eabi-4.8-2014.04_linux/
```



4) 编译 H5 linux 内核源码使用的交叉编译工具链为

- a. linux3.10

`gcc-linaro-4.9.4-2017.01-x86_64_aarch64-linux-gnu`

- b. linux5.4

`gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu`

5) 编译 H5 u-boot 源码使用的交叉编译工具链为

- a. u-boot 2014.07

`gcc-linaro-4.9.4-2017.01-x86_64_arm-linux-gnueabi`

- b. u-boot 2020.04

`gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu`4.1.3. **orange-pi-build** 完整目录结构说明

1) orange-pi-build 仓库下载完后并不包含 linux 内核、u-boot 的源码以及交叉编译工具链，linux 内核和 u-boot 的源码存放在独立的 git 仓库中（请不要单独下载使用内核和 u-boot 的源码进行编译操作，除非你知道怎么用）

- a. linux 内核源码存放的 git 仓库如下

- a) linux3.10

<https://github.com/orangepi-xunlong/linux-orangepi/tree/orange-pi-3.10-sun50iw2>

- b) linux5.4

<https://github.com/orangepi-xunlong/linux-orangepi/tree/orange-pi-5.4>

- b. u-boot 源码存放的 git 仓库如下

- a) u-boot 2014.07

<https://github.com/orangepi-xunlong/u-boot-orangepi/tree/v2014.07-sun50iw2-linux3.10>

- b) u-boot 2020.04

<https://github.com/orangepi-xunlong/u-boot-orangepi/tree/v2020.04>

2) orange-pi-build 第一次运行的时候会去下载交叉编译工具链、u-boot 和 linux 内核源码，成功编译完一次 linux 镜像后在 orange-pi-build 中可以看到的文件和文件夹有

- a. **build.sh**: 编译启动脚本
- b. **external**: 包含编译镜像需要用的配置文件、特定功能的脚本以及部分程序的源码，编译镜像过程中缓存的 rootfs 压缩包也存放在 external 中
- c. **kernel**: 存放 linux 内核的源码，里面名为 **orange-pi-3.10-sun50iw2** 的文件夹存放的就是 H5 开发板 legacy 分支的内核源码，里面名为 **orange-pi-5.4** 的文件夹存放的就是 H5 开发板 current 分支的内核源码（如果只编译了



legacy 分支的 linux 镜像，那么则只能看到 legacy 分支的内核源码；如果只编译了 current 分支的 linux 镜像，那么则只能看到 current 分支的内核源码），内核源码的文件夹的名字请不要手动修改，如果修改了编译系统运行时会重新下载内核源码

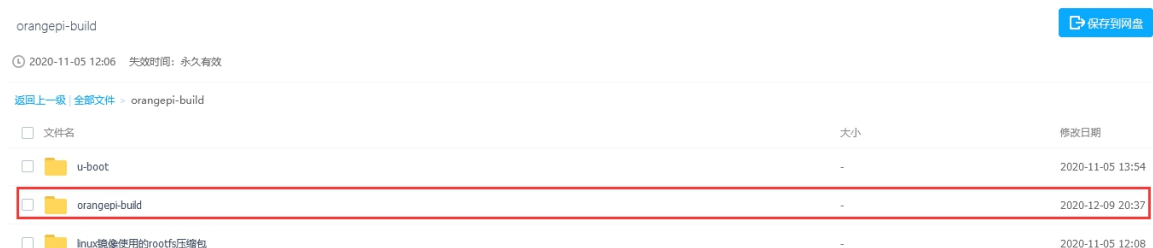
- d. **LICENSE**: GPL 2 许可证文件
- e. **README.md**: orangepi-build 说明文件
- f. **output**: 存放编译生成的 u-boot、linux 等 deb 包、编译日志以及编译生成的镜像等文件
- g. **scripts**: 编译 linux 镜像的通用脚本
- h. **toolchains**: 存放交叉编译工具链
- i. **u-boot**: 存放 u-boot 的源码，里面名为 **v2014.07-sun50iw2-linux3.10** 的文件夹存放的就是 H5 开发板 legacy 分支的 u-boot 源码，里面名为 **v2020.04** 的文件夹存放的就是 H5 开发板 current 分支的 u-boot 源码（如果只编译了 legacy 分支的 linux 镜像，那么则只能看到 legacy 分支的 u-boot 源码；如果只编译了 current 分支的 linux 镜像，那么则只能看到 current 分支的 u-boot 源码），u-boot 源码的文件夹的名字请不要手动修改，如果修改了编译系统运行时会重新下载 u-boot 源码
- j. **userpatches**: 存放编译脚本需要用到的配置文件

```
test@test:~/orangepi-build$ ls
build.sh  external  kernel  LICENSE  output  README.md  scripts  toolchains
u-boot   userpatches
```

4.1.4. 从百度云盘下载 orangepi-build

1) 如果从 github 下载 orangepi-build 的速度很慢，还可以从百度云盘下载 orangepi-build 的压缩包，百度云盘的下载链接为

链接: <https://pan.baidu.com/s/1vWQmCmSYdH7iCDFyKpJtVw>
提取码: zero





2) 百度云盘的 orangepi-build 文件夹下有两个文件

- orangepi-build.tar.gz** 为 orangepi-build 源码的压缩包
- orangepi-build.tar.gz.md5sum** 为 orangepi-build 源码的压缩包的 MD5 校验和文件
- 下载完后, 请首先检查下 orangepi-build.tar.gz 压缩包的 MD5 校验和是否正确, 这样可以防止下载的压缩包有问题, 如果不正确, 请重新下载, 检查校验和是否正确的命令为

```
test@test:~$ md5sum -c orangepi-build.tar.gz.md5sum
orangepi-build.tar.gz: 成功
```

orangepi-build 保存到网盘

🕒 2020-11-05 12:06 失效时间: 永久有效

[返回上一级](#) [全部文件](#) > orangepi-build > orangepi-build

☐ 文件名	大小	修改日期
☐ orangepi-build.tar.gz.md5sum	56B	2020-12-09 20:37
☐ orangepi-build.tar.gz	3.45G	2020-12-09 20:37

3) 然后就可以使用 **tar -zxf** 命令解压 orangepi-build.tar.gz

```
test@test:~$ tar -zxf orangepi-build.tar.gz
test@test:~$ cd orangepi-build/
test@test:~/orangepi-build$ ls
build.sh  external  kernel  LICENSE  README.md  scripts  toolchains
u-boot   userpatches
```

4) 使用 orangepi-build 编译系统前, 请先将 orangepi-build 和 github 服务器进行同步, 确保代码为最新的状态

```
test@test:~/orangepi-build$ git pull
```

如果 orangepi-build 和 github 服务器同步过程有问题, 可以尝试下面的方法

```
test@test:~/orangepi-build$ sudo rm -rf external scripts
test@test:~/orangepi-build$ git checkout .
test@test:~/orangepi-build$ git pull
```

5) 百度云盘上的 **orangepi-build.tar.gz** 压缩包除了包含 orangepi-build 编译系统的代码外, 还缓存了交叉编译工具链, u-boot 和 linux 内核的源码。所以在编译镜像的过程中就不会去 github 服务器上从头开始下载 u-boot 和 linux 内核的源码以及交叉编译工具链, 这样可以节省大量的时间。orangepi-build 编译系统开始运行时, 默认会



自动从 github 同步 u-boot 和 linux 内核的源码，以确保代码为最新状态，所以无需手动同步 u-boot 和 linux 内核的源码

6) 百度云盘上的 **orange-pi-build.tar.gz** 解压后的 kernel 文件夹中会包含多个版本的内核源码，请不要修改这些内核源码文件夹的名字，否则会导致编译系统找不到内核源码进而重新从 github 下载内核源码。kernel 文件夹下的多个内核源码并非都能被 H5 系列开发板使用，H5 系列开发板使用到的内核源码有：

orange-pi-3.10-sun50iw2	H5 系列开发板 legacy 分支使用
orange-pi-5.4	H5 系列开发板 current 分支使用
orange-pi-4.9-sun50iw9	H616 开发板使用，H5 系列开发板可以忽略

7) 百度云盘上的 **orange-pi-build.tar.gz** 解压后的 u-boot 文件夹中会包含多个版本的 u-boot 源码，请不要修改这些 u-boot 源码文件夹的名字，否则会导致编译系统找不到 u-boot 源码进而重新从 github 下载 u-boot 源码。u-boot 文件夹下的多个版本的 u-boot 源码并非都能被 H5 系列开发板使用，H5 系列开发板使用到的 u-boot 源码有：

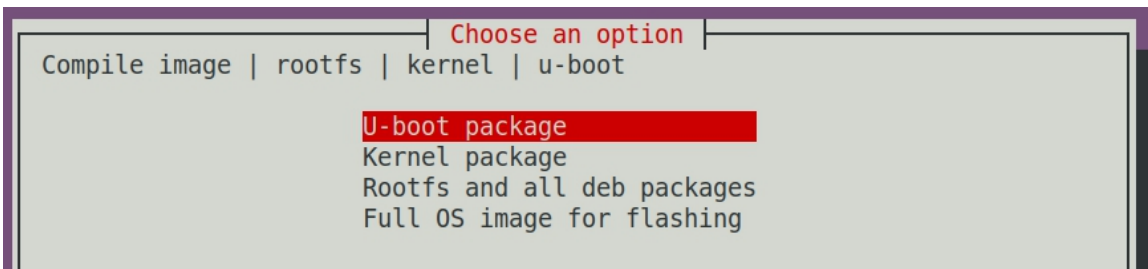
v2014.07-sun50iw2-linux3.10	H5 系列开发板 legacy 分支使用
v2020.04	H5 系列开发板 current 分支使用
v2018.05-sun50iw9	H616 开发板使用，H5 系列开发板可以忽略

4.2. 编译 u-boot

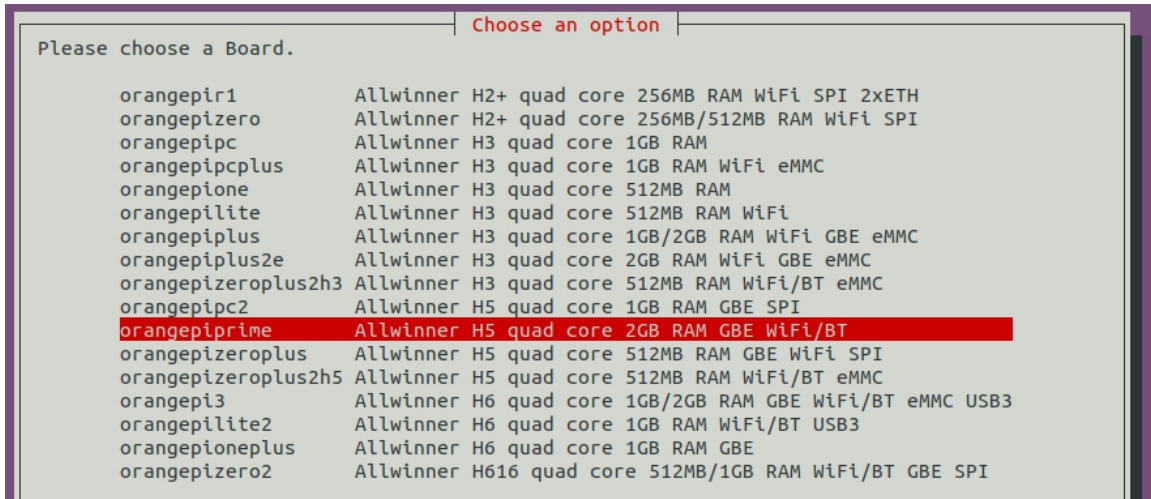
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orange-pi-build$ sudo ./build.sh
```

2) 选择 **U-boot package**，然后回车

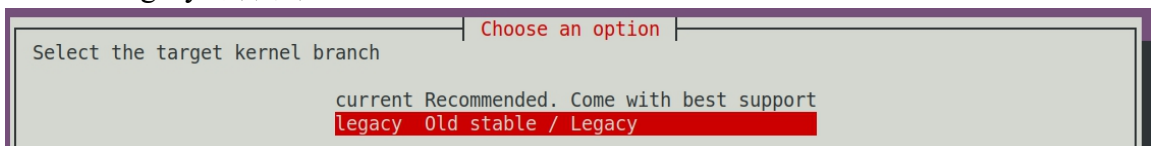


3) 接着选择开发板的型号



4) 然后选择分支

- current 会编译 u-boot v2020.04
- legacy 会编译 u-boot v2014.07



5) 然后就会开始编译 u-boot, 编译时提示的部分信息说明如下(以 legacy 分支为例)

- u-boot 源码的版本

[o.k.] Compiling u-boot [**v2014.07**]

- 交叉编译工具链的版本

[o.k.] Compiler version [**arm-linux-gnueabi-gcc 4.9.4**]

- 编译生成的 u-boot deb 包的路径

[o.k.] Target directory [**output/debs/u-boot**]

- 编译生成的 u-boot deb 包的包名

[o.k.] File name [**linux-u-boot-legacy-orangepiprime_2.1.2_arm64.deb**]

- 编译使用的时间

[o.k.] Runtime [**1 min**]

- 重复编译 u-boot 的命令, 使用下面的命令无需通过图形界面选择, 可以直接开始编译 u-boot

[o.k.] Repeat Build Options [**sudo ./build.sh BOARD=orangepiprime
BRANCH=legacy BUILD_OPT=u-boot KERNEL_CONFIGURE=no**]



6) 查看编译生成的 u-boot deb 包

```
test@test:~/orangepi-build$ ls output/debs/u-boot/
linux-u-boot-legacy-orangepiprime_2.1.2_arm64.deb
```

7) 生成的 u-boot 的 deb 包包含的文件如下所示

a. 使用下面的命令可以解压 deb 包

```
test@test:~/orangepi-build$ cd output/debs/u-boot
test@test:~/orangepi_build/output/debs/u-boot$ $ dpkg -x \
linux-u-boot-legacy-orangepiprime_2.1.2_arm64.deb . (注意命令最后有个 ".")
test@test:~/orangepi_build/output/debs/u-boot$ ls
linux-u-boot-current-orangepiprime_2.1.2_armhf.deb  usr
```

b. 解压后的文件如下所示

```
test@test:~/orangepi-build/output/debs/u-boot$ tree usr
usr
├── lib
│   ├── linux-u-boot-legacy-orangepiprime_2.1.2_arm64
│   │   ├── boot0_sdcard.fex //boot0 的二进制文件
│   │   └── boot_package.fex //u-boot 的二进制文件
│   └── u-boot
│       ├── LICENSE
│       ├── orangepiprime-u-boot.dts
│       └── platform_install.sh
3 directories, 5 files
```

8) orangepi-bulid 编译系统编译 u-boot 源码时首先会将 u-boot 的源码和 github 服务器的 u-boot 源码进行同步，所以如果想修改 u-boot 的源码，首先需要关闭源码的下载更新功能（需要完整编译过一次 u-boot 后才能关闭这个功能，否则会提示找不到 u-boot 的源码），否则所作的修改都会被还原，方法如下：

设置 `userpatches/config-default.conf` 中的 `IGNORE_UPDATES` 变量为 “yes”

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
IGNORE_UPDATES="yes"
```

9) 调试 u-boot 代码时，可以使用下面的方法来更新 linux 镜像中的 u-boot 进行测试

a. 将编译好的 u-boot 的 deb 包上传到开发板的 linux 系统中



```
test@test:~/orange-pi-build$ cd output/debs/u-boot
test@test:~/orange-pi-build/output/debs/u-boot$ scp \
linux-u-boot-legacy-orangepiprime_2.1.2_arm64.deb root@192.168.1.xxx:/root
```

- b. 然后登录到开发板，卸载已安装的 u-boot 的 deb 包

```
root@orange-pi:~# apt purge -y linux-u-boot-orangepiprime-legacy
```

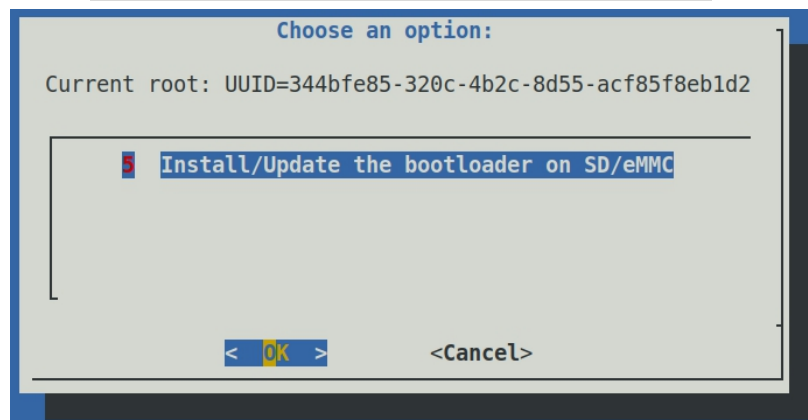
- c. 再安装刚才上传的新的 u-boot 的 deb 包

```
root@orange-pi:~# dpkg -i linux-u-boot-legacy-orangepiprime_2.1.2_arm64.deb
```

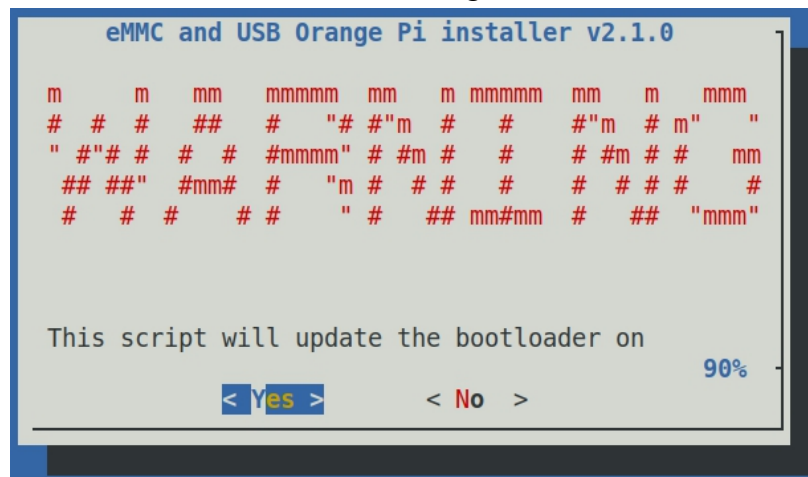
- d. 然后运行 nand-sata-install 脚本

```
root@orange-pi:~# nand-sata-install
```

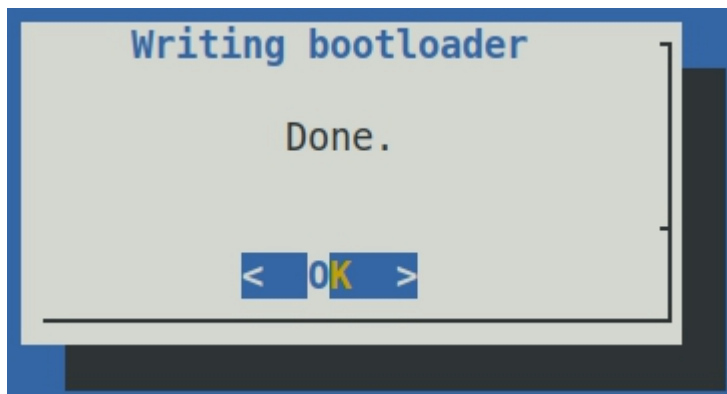
- e. 然后选择 **5 Install/Update the bootloader on SD/eMMC**



- f. 按下回车键后首先会弹出一个 Warning



- g. 再按下回车键就会开始更新 u-boot，更新完后会显示下面的信息



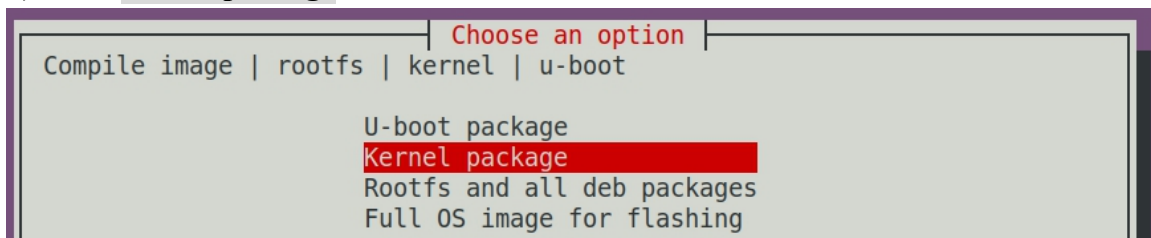
- h. 然后就可以重启开发板来测试 u-boot 的修改是否生效了

4.3. 编译 linux 内核

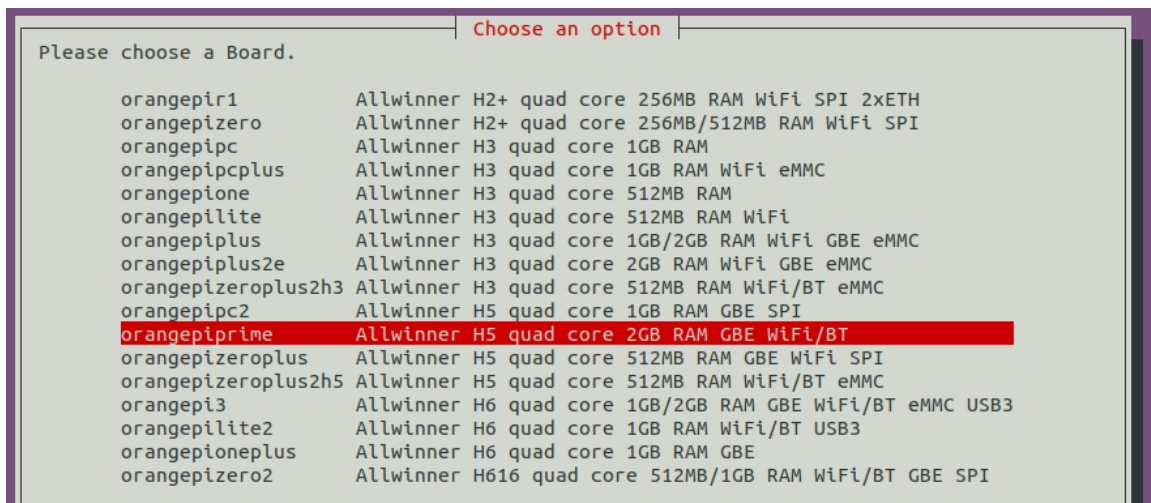
- 1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orange-pi-build$ sudo ./build.sh
```

- 2) 选择 **Kernel package**，然后回车



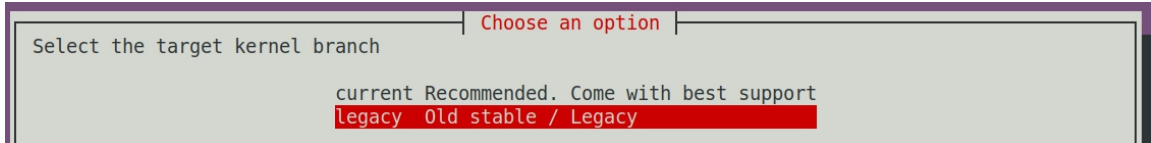
- 3) 接着选择开发板的型号



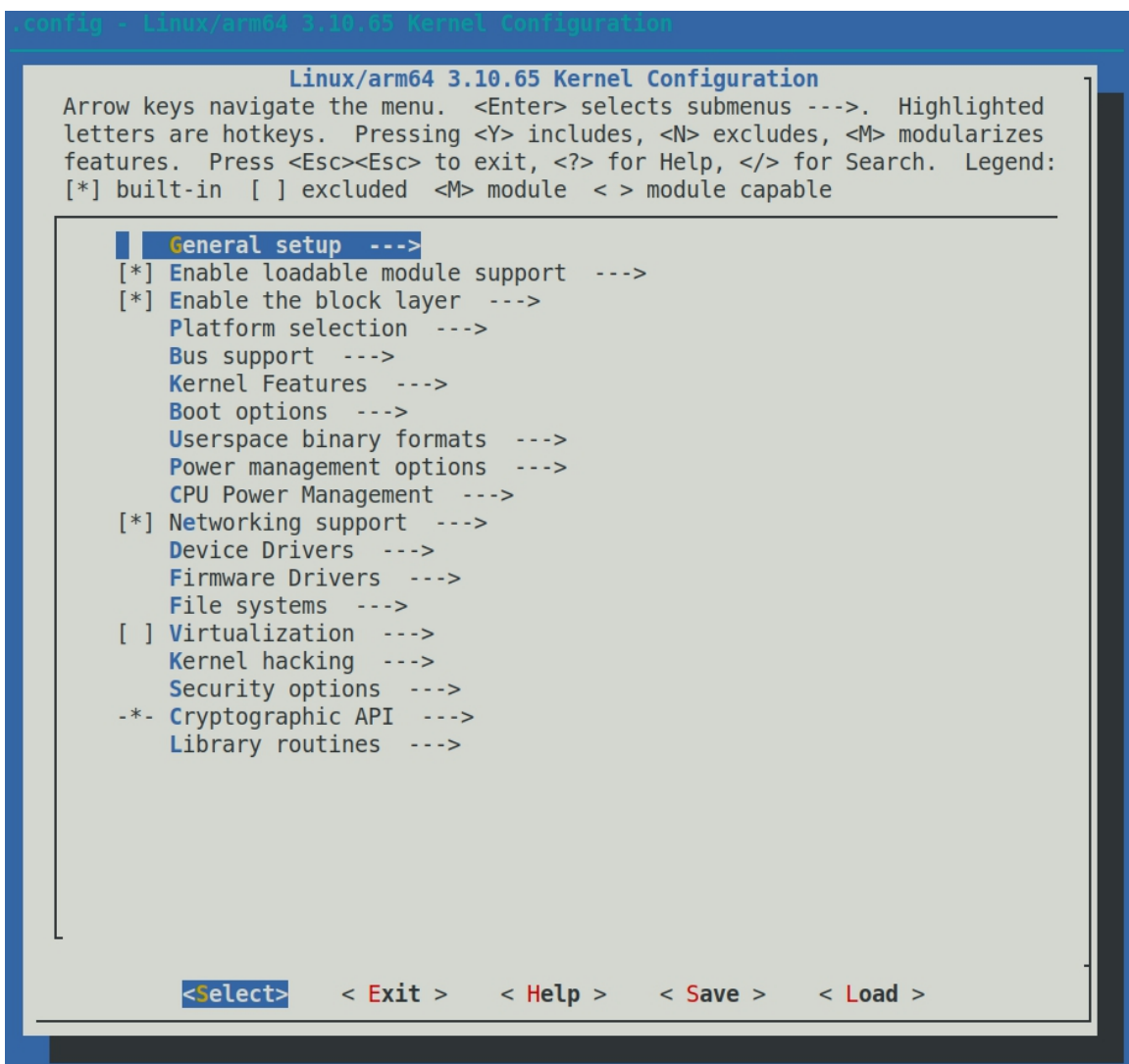


4) 然后选择分支

- a. current 会编译 linux 5.4
- b. legacy 会编译 linux3.10



5) 然后会弹出通过 **make menuconfig** 打开的内核配置的界面, 此时可以直接修改内核的配置, 如果不需要修改内核配置, 直接退出即可, 退出后会开始编译内核源码



- a. 如果不需要修改内核的配置选项, 在运行 **build.sh** 脚本时, 传入 **KERNEL_CONFIGURE=no** 就可临时屏蔽弹出内核的配置界面了



```
test@test:~/orange-pi-build$ sudo ./build.sh KERNEL_CONFIGURE=no
```

b. 也可以设置 `orange-pi-build/userpatches/config-default.conf` 配置文件中的 **KERNEL_CONFIGURE=no**，这样可以永久禁用这个功能

c. 编译内核的时候如果提示下面的错误，这是由于 Ubuntu PC 的终端界面太小，导致 `make menuconfig` 的界面无法显示，请把 Ubuntu PC 的终端调到最大，然后重新运行 `build.sh` 脚本

```
HOSTCC scripts/kconfig/mconf.o
HOSTCC scripts/kconfig/lxdialog/checklist.o
HOSTCC scripts/kconfig/lxdialog/util.o
HOSTCC scripts/kconfig/lxdialog/inputbox.o
HOSTCC scripts/kconfig/lxdialog/textbox.o
HOSTCC scripts/kconfig/lxdialog/yesno.o
HOSTCC scripts/kconfig/lxdialog/menubox.o
HOSTLD scripts/kconfig/mconf
scripts/kconfig/mconf Kconfig
Your display is too small to run Menuconfig!
It must be at least 19 lines by 80 columns.
scripts/kconfig/Makefile:28: recipe for target 'menuconfig' failed
make[1]: *** [menuconfig] Error 1
Makefile:560: recipe for target 'menuconfig' failed
make: *** [menuconfig] Error 2
[ error ] ERROR in function compile kernel [ compilation.sh:376 ]
[ error ] Error kernel menuconfig failed
[ o.k. ] Process terminated
```

6) 编译内核源码时提示的部分信息说明如下（以 legacy 分支为例）

a. linux 内核源码的版本

```
[ o.k. ] Compiling legacy kernel [ 3.10.65 ]
```

b. 使用的交叉编译工具链的版本

```
[ o.k. ] Compiler version [ aarch64-linux-gnu-gcc 4.9.4 ]
```

c. 内核默认使用的配置文件以及它存放的路径

```
[ o.k. ] Using kernel config file [ config/kernel/linux-sun50iw2-legacy.config ]
```

d. 如果设置了 **KERNEL_CONFIGURE=yes**，内核最终使用的配置文件 `.config` 会复制到 `output/config` 中，如果没有对内核配置进行修改，最终的配置文件和默认的配置是一致的

```
[ o.k. ] Exporting new kernel config [ output/config/linux-sun50iw2-legacy.config ]
```

e. 编译生成的内核相关的 deb 包的路径

```
[ o.k. ] Target directory [ output/debs/ ]
```

f. 编译生成的内核镜像 deb 包的包名

```
[ o.k. ] File name [ linux-image-legacy-sun50iw2_2.1.2_arm64.deb ]
```

g. 编译使用的时间

```
[ o.k. ] Runtime [ 5 min ]
```

h. 最后会显示重复编译上一次选择的内核的编译命令，使用下面的命令无需通



过图形界面选择，可以直接开始编译内核源码

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh BOARD=orangepiprime  
BRANCH=legacy BUILD_OPT=kernel KERNEL_CONFIGURE=yes ]
```

7) 查看编译生成的内核相关的 deb 包

- a. **linux-headers-legacy-sun50iw2_2.1.2_arm64.deb** 包含内核头文件
- b. **linux-image-legacy-sun50iw2_2.1.2_arm64.deb** 包含内核镜像和内核模块

```
test@test:~/orangepi-build$ ls output/debs/linux-*  
output/debs/linux-headers-legacy-sun50iw2_2.1.2_arm64.deb  
output/debs/linux-image-legacy-sun50iw2_2.1.2_arm64.deb
```

8) 生成的 linux-image 的 deb 包包含的文件如下所示

- a. 使用下面的命令可以解压 deb 包

```
test@test:~/orangepi-build$ cd output/debs  
test@test:~/orangepi_build/output/debs$ mkdir test  
test@test:~/orangepi_build/output/debs$ cp \  
linux-image-legacy-sun50iw2_2.1.2_arm64.deb test/  
test@test:~/orangepi_build/output/debs$ cd test  
test@test:~/orangepi_build/output/debs/test$ dpkg -x \  
linux-image-legacy-sun50iw2_2.1.2_arm64.deb .  
test@test:~/orangepi_build/output/debs/test$ ls  
boot etc lib linux-image-legacy-sun50iw2_2.1.2_arm64.deb usr
```

- b. 解压后的文件如下所示

```
test@test:~/orangepi_build/output/debs/test$ tree -L 2  
.  
├── boot  
│   ├── config-3.10.65-sun50iw2      //编译内核源码使用的配置文件  
│   ├── dtb-3.10.65-sun50iw2        //dtb 文件，暂未使用  
│   ├── Image-3.10.65-sun50iw2      //编译生成的内核镜像文件  
│   └── System.map-3.10.65-sun50iw2  
├── etc  
│   └── kernel  
├── lib  
│   └── modules                      //编译生成的内核模块  
└── linux-image-legacy-sun50iw2_2.1.2_arm64.deb
```




```
└─ usr
   └─ share
```

8 directories, 4 files

9) orangepi-bulid 编译系统编译 linux 内核源码时首先会将 linux 内核源码和 github 服务器的 linux 内核源码进行同步，所以如果想修改 linux 内核的源码，首先需要关闭源码的更新功能（需要完整编译过一次 linux 内核源码后才能关闭这个功能，否则会提示找不到 linux 内核的源码），否则所作的修改都会被还原，方法如下：

设置 userpatches/config-default.conf 中的 IGNORE_UPDATES 变量为 “yes”

```
test@test:~/orangepi-build$ vim userpatches/config-default.conf
IGNORE_UPDATES="yes"
```

10) 如果对内核做了修改，可以使用下面的方法来更新开发板 linux 系统的内核和内核模块

a. 将编译好的 linux 内核的 deb 包上传到开发板的 linux 系统中

```
test@test:~/orangepi-build$ cd output/debs
test@test:~/orangepi-build/output/debs$ scp \
linux-image-legacy-sun50iw2_2.1.2_arm64.deb root@192.168.1.207:/root
```

b. 然后登录到开发板，卸载已安装的 linux 内核的 deb 包

```
root@orangepi:~# apt purge -y linux-image-legacy-sun50iw2
```

c. 再安装刚才上传的新的 linux 内核的 deb 包

```
root@orangepi:~# dpkg -i linux-image-legacy-sun50iw2_2.1.2_arm64.deb
```

d. 然后重启开发板，再查看内核相关的修改是否已生效

11) 安装内核头文件到 linux 系统中的方法如下所示

a. 将编译好的 linux 头文件的 deb 包上传到开发板的 linux 系统中

```
test@test:~/orangepi-build$ cd output/debs
test@test:~/orangepi-build/output/debs$ scp \
linux-headers-legacy-sun50iw2_2.1.2_arm64.deb root@192.168.1.207:/root
```

b. 然后登录到开发板，安装刚才上传的 linux 头文件的 deb 包

```
root@orangepi:~# dpkg -i linux-headers-legacy-sun50iw2_2.1.2_arm64.deb
```

c. 安装完后在 /usr/src 中就可以看到刚才安装的内核头文件相关的内容了

```
root@orangepi:~# ls /usr/src
linux-headers-3.10.65-sun50iw2
```



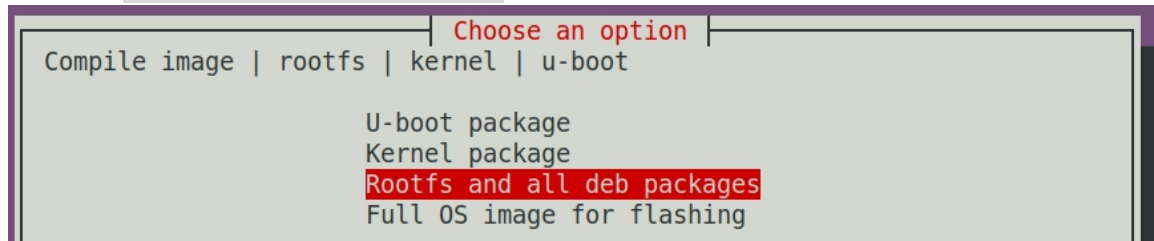

```
root@orangepi:~# ls /usr/src/linux-headers-3.10.65-sun50iw2
Documentation Module.symvers certs firmware init lib net security usr
Kconfig arch crypto fs ipc mm samples sound virt Makefile block
drivers include kernel modules scripts tools
```

4.4. 编译 rootfs

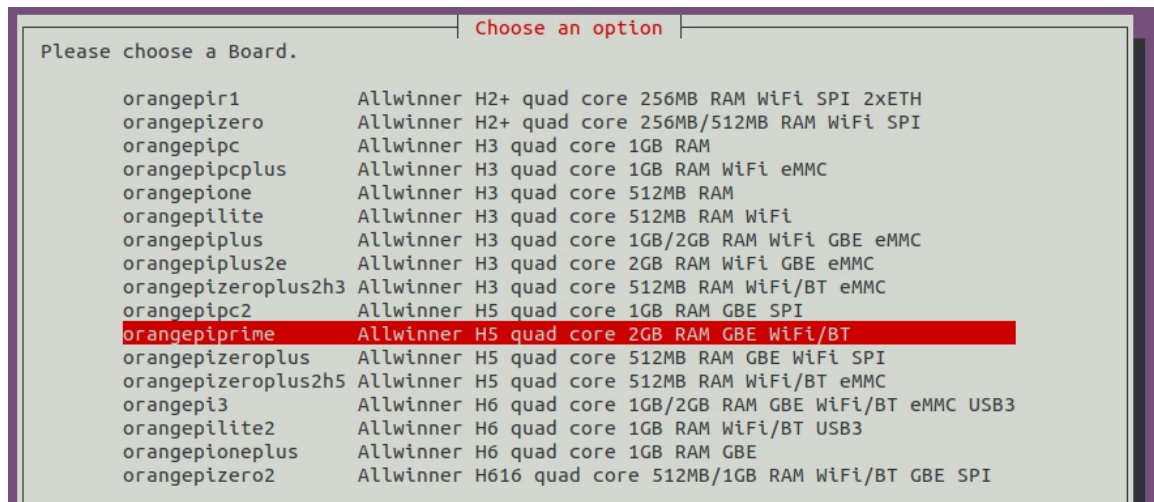
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orangepi-build$ sudo ./build.sh
```

2) 选择 **Rootfs and all deb packages**，然后回车



3) 接着选择开发板的型号

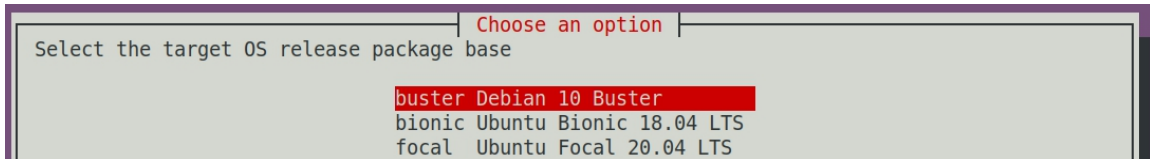


4) 然后选择 rootfs 的类型

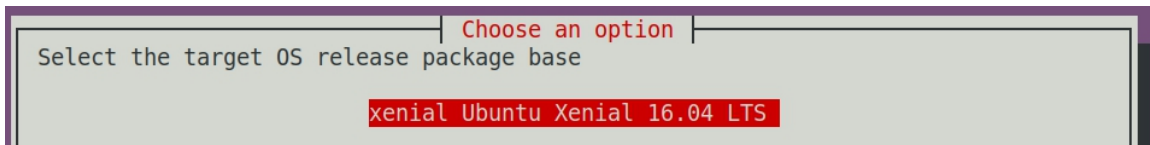
buster	Debian 10
bionic	Ubuntu 18.04
focal	Ubuntu 20.04
xenial	Ubuntu16.04



- a. linux5.4 支持的 linux 发行版如下所示

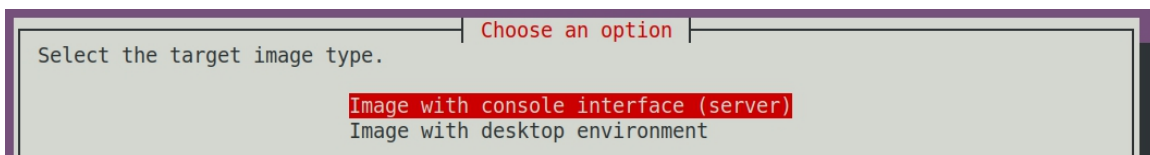


- b. linux3.10 支持的 linux 发行版如下所示

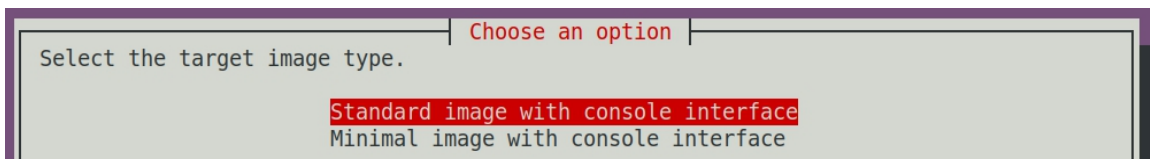


5) 然后选择镜像的类型

- a. **Image with console interface (server)** 表示服务器版的镜像，体积比较小
 b. **Image with desktop environment** 表示带桌面的镜像，体积比较大



- 6) 如果是编译服务器版的镜像，还可以选择编译 Standard 版本或者 Minimal 版本，Minimal 版本预装的软件会比 Standard 版本少很多



- 7) 选择镜像的类型后就会开始编译 rootfs，编译时提示的部分信息说明如下

- a. rootfs 的类型

```
[ o.k. ] local not found [ Creating new rootfs cache for focal ]
```

- b. 编译生成的 rootfs 压缩包的存放路径

```
[ o.k. ] Target directory [ external/cache/rootfs ]
```

- c. 编译生成的 rootfs 压缩包的名字

```
[ o.k. ] File name [ focal-cli-arm64.153618961f14c28107ca023429aa0eb9.tar.lz4 ]
```

- d. 编译使用的时间

```
[ o.k. ] Runtime [ 13 min ]
```



- e. 重复编译 rootfs 的命令，使用下面的命令无需通过图形界面选择，可以直接开始编译 rootfs

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh BOARD=orangepiprime  
BRANCH=legacy BUILD_OPT=rootfs RELEASE=focal BUILD_MINIMAL=no  
BUILD_DESKTOP=no KERNEL_CONFIGURE=yes ]
```

8) 查看编译生成的 rootfs 压缩包

- a. `focal-cli-arm64.153618961f14c28107ca023429aa0eb9.tar.lz4` 是 rootfs 的压缩包，名字各字段的含义为
- a) **focal** 表示 rootfs 的 linux 发行版的类型
 - b) **cli** 表示 rootfs 为服务器版的类型，如果为 **desktop** 则表示桌面版类型
 - c) **arm64** 表示 rootfs 的架构类型
 - d) **153618961f14c28107ca023429aa0eb9** 是由 rootfs 安装的所有软件包的包名生成的 MD5 哈希值，只要没有修改 rootfs 安装的软件包的列表，那么这个值就不会变，编译脚本会通过这个 MD5 哈希值来判断是否需要重新编译 rootfs
- b. `focal-cli-arm64.153618961f14c28107ca023429aa0eb9.tar.lz4.list` 列出了 rootfs 安装的所有软件包的包名

```
test@test:~/orangepi-build$ ls external/cache/rootfs/  
focal-cli-arm64.153618961f14c28107ca023429aa0eb9.tar.lz4  
focal-cli-arm64.153618961f14c28107ca023429aa0eb9.tar.lz4.list
```

- 9) 如果需要的 rootfs 在 `external/cache/rootfs` 下已经存在，那么再次编译 rootfs 就会直接跳过编译过程，不会重新开始编译，编译镜像的时候也会去 `external/cache/rootfs` 下查找是否已经有缓存可用的 rootfs，如果有就直接使用，这样可以节省大量的下载编译时间

- 10) 由于编译 rootfs 的时间较长，如果不想从头开始编译 rootfs，或者编译 rootfs 的过程有问题，可以直接下载 Orange Pi 缓存的 rootfs 压缩包，rootfs 压缩包百度云盘的下载链接如下所示，下载好的 rootfs 压缩包（不要解压哦）需要放在 `orangepi-build` 的 `external/cache/rootfs` 目录下才能被编译脚本正常使用

```
链接: https://pan.baidu.com/s/1vWQmCmSYdH7iCDFyKpJtVw  
提取码: zero
```



orange-pi-build

🕒 2020-11-05 12:06 失效时间: 永久有效

[返回上一级](#) | [全部文件](#) > orange-pi-build

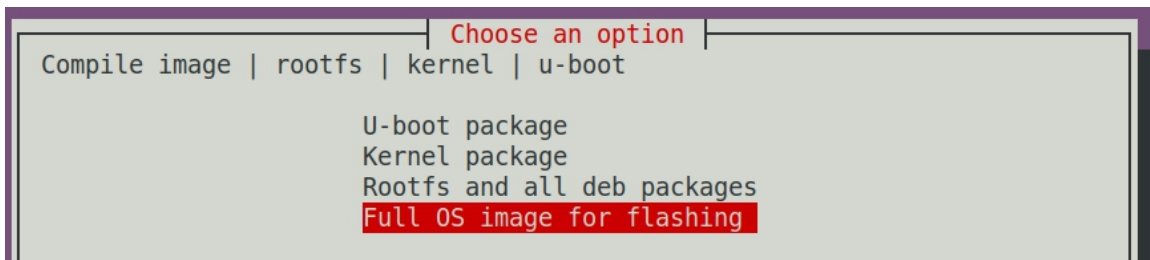
☐ 文件名	大小
☐  linux镜像使用的rootfs压缩包	-
☐  toolchains.tar.gz	1.71G
☐  orange-pi-build.tar.gz	151.7M

4.5. 编译 linux 镜像

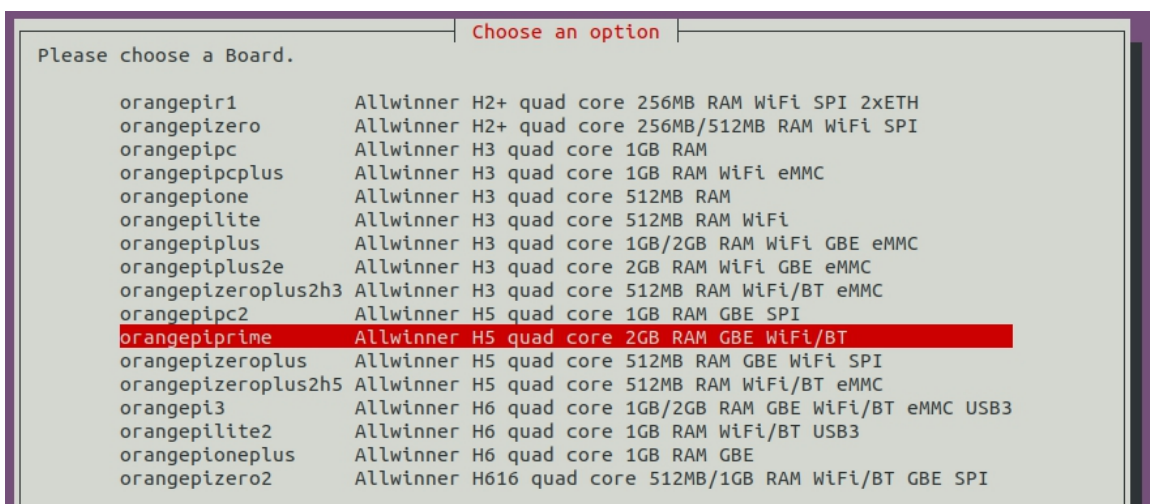
1) 运行 build.sh 脚本，记得加 sudo 权限

```
test@test:~/orange-pi-build$ sudo ./build.sh
```

2) 选择 **Full OS image for flashing**，然后回车



3) 然后选择开发板的型号

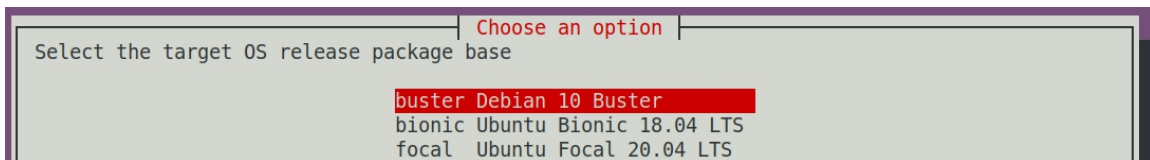


4) 然后选择 rootfs 的类型

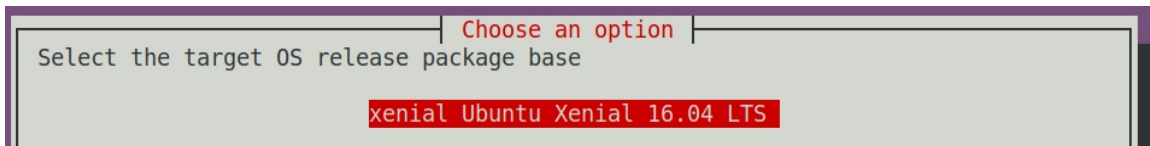


buster	Debian 10
bionic	Ubuntu 18.04
focal	Ubuntu 20.04
xenial	Ubuntu 16.04

- a. linux5.4 支持的 linux 发行版如下所示

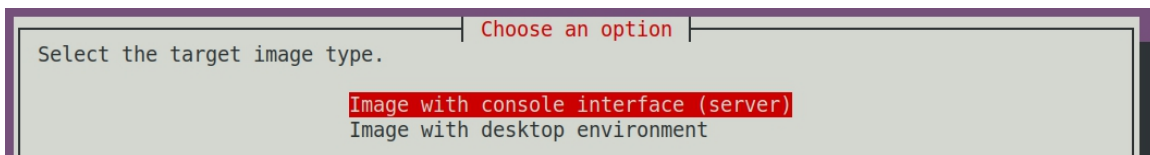


- b. linux3.10 支持的 linux 发行版如下所示

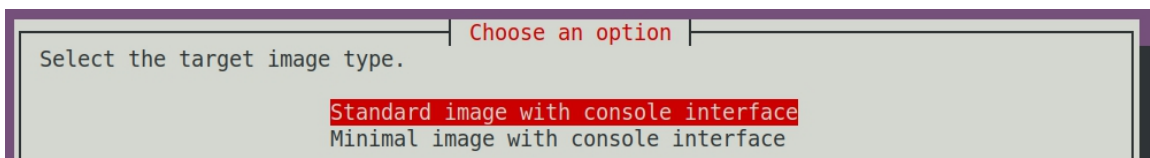


5) 然后选择镜像的类型

- a. **Image with console interface (server)** 表示服务器版的镜像，体积比较小
b. **Image with desktop environment** 表示带桌面的镜像，体积比较大



- 6) 如果是编译服务器版的镜像，还可以选择编译 Standard 版本或者 Minimal 版本，Minimal 版本预装的软件会比 Standard 版本少很多



7) 选择镜像的类型后就会开始编译 linux 镜像，编译的大致流程如下

- 初始化 Ubuntu PC 的编译环境，安装编译过程需要的软件包
- 下载 u-boot 和 linux 内核的源码（如果已经缓存，则只更新代码）
- 编译 u-boot 源码，生成 u-boot 的 deb 包
- 编译 linux 源码，生成 linux 相关的 deb 包



- e. 制作 linux firmware 的 deb 包
- f. 制作 orangepi-config 工具的 deb 包
- g. 制作板级支持的 deb 包
- h. 如果是编译 desktop 版镜像，还会制作 desktop 相关的 deb 包
- i. 检查 rootfs 是否已经缓存，如果没有缓存，则重新制作 rootfs，如果已经缓存，则直接解压使用
- j. 安装前面生成的 deb 包到 rootfs 中
- k. 对不同的开发板和不同类型镜像做一些特定的设置，如预装额外的软件包，修改系统配置等
- l. 然后制作镜像文件，并格式化分区，默认类型为 ext4
- m. 再将配置好的 rootfs 拷贝到镜像的分区中
- n. 然后更新 initramfs
- o. 最后将 u-boot 的 bin 文件通过 dd 命令写入到镜像中

8) 编译完镜像后会提示下面的信息（以 legacy 分支为例）

- a. 编译生成的镜像的存放路径

```
[ o.k. ] Done building  
[ output/images/orangepiprime_2.1.2_ubuntu_focal_server_linux3.10.65/orangepiprime_2.1.2_ubuntu_focal_server_linux3.10.65.img ]
```

- b. 编译使用的时间

```
[ o.k. ] Runtime [ 19 min ]
```

- c. 重复编译镜像的命令，使用下面的命令无需通过图形界面选择，可以直接开始编译镜像

```
[ o.k. ] Repeat Build Options [ sudo ./build.sh BOARD=orangepiprime  
BRANCH=legacy BUILD_OPT=image RELEASE=focal BUILD_MINIMAL=no  
BUILD_DESKTOP=no KERNEL_CONFIGURE=yes ]
```



5. Android 系统使用说明

5.1. 已支持的 Android 版本

Android 版本	内核版本
Android 5.1	linux3.10

5.2. Android 5.1 功能适配情况

功能	状态
HDMI 视频	OK
HDMI 音频	OK
USB2.0 x 3	OK
TF 卡启动	OK
红外	OK
WIFI	OK
蓝牙	OK
以太网卡	OK
耳机音频	OK
TV-OUT	OK
MIC 录音	OK
USB 摄像头	OK
LED 灯	OK
温度传感器	OK
ADB 调试	OK
GC2035 摄像头	OK
OV5640 摄像头	OK
Mali GPU	OK
视频编解码	OK



5.3. 板载 LED 灯显示说明

1) LED 灯显示情况

	绿灯	红灯
u-boot 启动阶段	灭	亮
内核启动到进入系统	亮	灭

2) LED 灯对应的 GPIO 口

	GPIO 口
绿灯	PL10
红灯	PA20

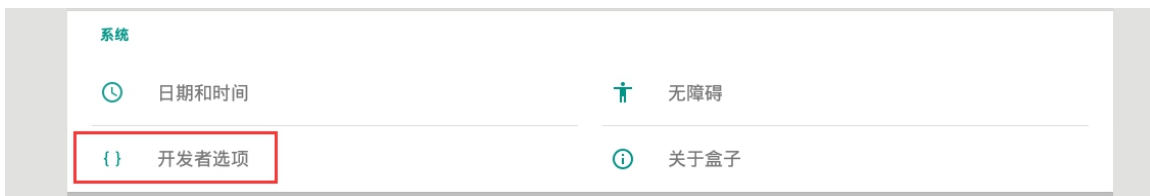
5.4. ADB 使用说明

5.4.1. 打开 USB debugging 选项

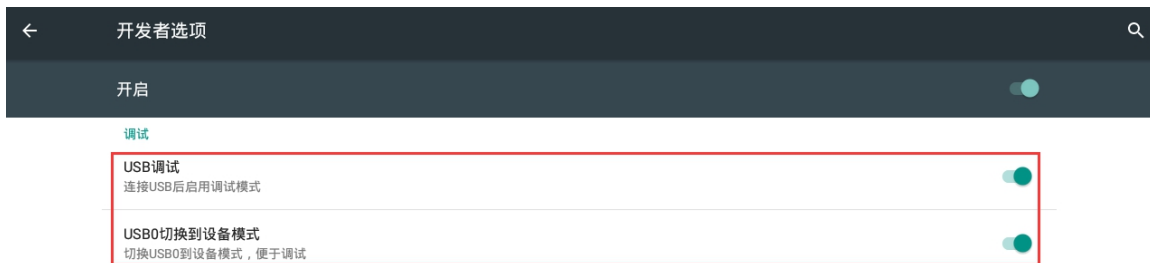
1) USB 调试模式默认是打开的，ADB 调试默认是可以直接使用的，如果 ADB 使用有问题，可以使用下面的方式打开，首先选择设置



2) 然后选择开发者选项



3) 然后打开 **USB 调试**和 **USB0** 切换到设备模式



5. 4. 2. 使用网络连接 adb 调试

1) 使用网络 adb 无需使用 Micro USB 接口的数据线来连接电脑和开发板，而是通过网络来通信，所以首先请确保开发板的有线或者无线网络已经连接好了，然后获取开发板的 IP 地址，后面要用到

2) 确保已经打开 **USB debugging** 选项

3) 确保 Android 系统的 **service.adb.tcp.port** 设置为 5555 端口号

```
root@cheetah-p1:/ # getprop | grep "adb.tcp"
[service.adb.tcp.port]: [5555]
```

4) 如果 **service.adb.tcp.port** 没有设置，可以使用下面的命令设置网络 adb 的端口号

```
root@cheetah-p1:/ # setprop service.adb.tcp.port 5555
root@cheetah-p1:/ # stop adbd
root@cheetah-p1:/ # start adbd
```

5) 在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt update
test@test:~$ sudo apt -y install adb
```

6) 然后在 Ubuntu PC 上连接网络 adb

```
test@test:~$ adb connect 192.168.1.xxx (IP 地址需要修改为开发板的 IP 地址)
```



```
connected to 192.168.1.xxx:5555
```

```
test@test:~$ adb devices
```

```
List of devices attached
```

```
192.168.1.xxx:5555    device
```

7) 然后在 Ubuntu PC 上通过 adb shell 就可以登录 android 系统

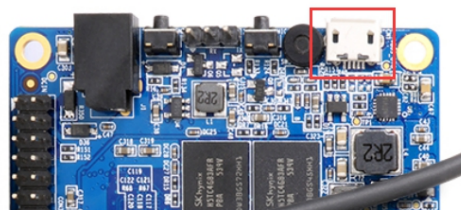
```
test@test:~$ adb shell
```

```
petrel-p1:/ #
```

5.4.3. 使用数据线连接 adb 调试

1) 首先确保打开 **USB debugging** 选项

2) 首先需要使用 Micro USB 接口的数据线将开发板连接到电脑的 USB 接口中



3) 在 Ubuntu PC 上安装 adb 工具

```
test@test:~$ sudo apt update
```

```
test@test:~$ sudo apt -y install adb
```

4) 查看识别到 ADB 设备

```
test@test:~$ adb devices
```

```
List of devices attached
```

```
20080411    device
```

5) 然后在 Ubuntu PC 上通过 adb shell 就可以登录 android 系统

```
test@test:~$ adb shell
```

```
petrel-p1:/ #
```



5.5. USB 摄像头的使用方法

1) 在开发板的 USB 接口中插入 USB 摄像头

2) 在百度云盘中下载 USB 摄像头测试 APP

☐	Win32DiskImager.rar	13.3M	2019-11-11 18:05
☐	usbcamera.apk	20M	2020-05-22 17:17
☐	SDFormatter4.zip	6M	2019-11-11 18:05

3) 然后安装 usbcamera.apk 到安卓系统中, 可以使用 U 盘拷贝安装, 也可以使用 adb 来安装, 使用 adb 安装 usbcamera.apk 的命令为

```
test@test:~$ adb devices
List of devices attached
192.168.1.xxx:5555 device //先确保 adb 已正常连接

test@test:~$ adb install usbcamera.apk
```

4) 安装完后可以在所有应用中看到 USB 摄像头的启动图标



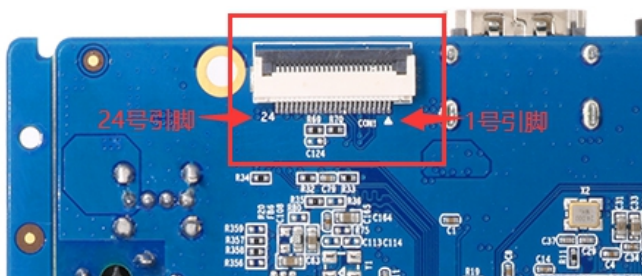
5) 然后打开 USB 摄像头 APP 就能看到 USB 摄像头的视频输出了



5.6. CSI 摄像头接口规格

5.6.1. CSI 摄像头接口规格

- 1) Orange Pi Prime 的 CSI 接口支持 **gc2035** 和 **ov5640** 两款摄像头
- 2) CSI 接口引脚的序号如下图所示
 - a. CSI 接口的 1 号引脚接到摄像头转接板的 24 号引脚
 - b. CSI 接口的 24 号引脚接到摄像头转接板的 1 号引脚



5.6.2. Android 系统 gc2035 摄像头的使用方法

- 1) Android 系统测试 gc2035 摄像头需要使用下面的安卓镜像

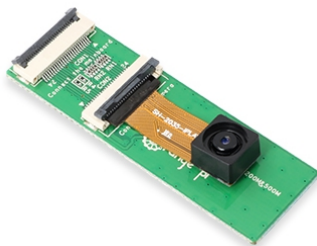
[返回上一级](#) [全部文件](#) > android

☐ 文件名	大小	修改日期
☐ OrangePi_Prime_Android_5.1_zh_ov5640_v2.0.img.tar.gz	241.3M	2019-12-17 10:17
☐ OrangePi_Prime_Android_v1.0.img.tar.gz	241M	2019-12-17 10:17

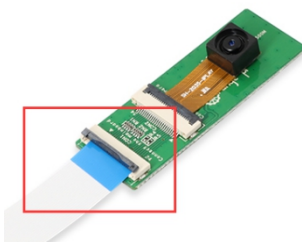
- 2) Gc2035 摄像头套件包含包含一个 gc2035 摄像头、一个转接板和一根排线



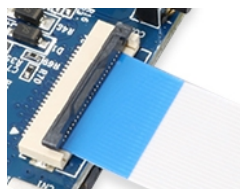
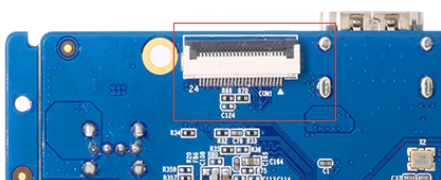
- 3) 首先将 gc2035 摄像头插到转接板中



4) 然后将排线插入转接板的另一卡槽内



5) 然后把排线的另一头插入开发板的 CIS 摄像头接口中。接好摄像头后再启动 Android 系统（**不要在上电后再插入摄像头**）



6) Android 系统启动后打开摄像头的 APP 就可以看到 gc2035 摄像头的输出了，摄像头 APP 所在的位置如下图所示



5.6.3. Android 系统 ov5640 摄像头的使用方法

1) 首先将 Ov5640 摄像头转接板通过排线接到开发板的 CIS 摄像头接口上，接好摄像头后再启动 Android 系统（**不要在上电后再插入摄像头**）



2) Android 系统测试 ov5640 摄像头需要用下面的安卓镜像

☐ 文件名	大小	修改日期
☐ OrangePi_Prime_Android_5.1_zh_ov5640_v2.0.img.tar.gz	241.3M	2019-12-17 10:17
☐ OrangePi_Prime_Android_v1.0.img.tar.gz	241M	2019-12-17 10:17

3) Android 系统启动后打开摄像头的 APP 就可以看到 ov5640 摄像头的输出了，摄像头 APP 所在的位置如下图所示





6. Android SDK 使用说明

1) Allwinner H5 SOC 支持的 Android SDK 如下所示

Android 版本	内核版本
Android 5.1	linux 3.10

2) Android SDK 的编译都是在安装有 **Ubuntu 14.04** 的 PC 上进行的，其它版本的 Ubuntu 系统可能会有一些区别

3) Android 5.1 版本的 SDK 都是芯片厂商释放的原始的 SDK，如果想在 Orange Pi 开发板上使用 SDK 编出来的 Android 镜像，需要针对不同的板子进行适配，才能保证所有功能使用正常

6.1. Android 5.1 SDK 使用说明

6.1.1. 下载 android 5.1 sdk 的源码

1) Android 源码的下载地址为

<http://www.orangepi.cn/downloadresourcescn/>

2) 进入资料页面后，再找到 Orange Pi 开发板对应的资料下载链接，选择 Android 源码选项



3) 然后选择百度云盘，并复制百度云盘的提取码



Android源码

Source Code

版本:

发布日期: 2019-12-17

提取码: 3vyi



Google Drive



百度云盘

4) 然后下载 **H5-Android 5.1** 文件夹的所有内容

H5_Source_Code

🕒 2020-05-14 15:46 过期时间: 永久有效

[返回上一级](#) | [全部文件](#) > H5_Source_Code

☐ 文件名

☐  H5_Android_5.1

5) **H5-Android 5.1** 文件夹包含如下 2 个文件

- a. **h5_v1.1.tar.bz2**: android 源码
- b. **h5_v1.1.tar.bz2.md5sum**: h5_v1.1.tar.bz2 的 MD5 检验和文件

6) 下载完 android 源码后先检查下 MD5 检验和是否正确, 如果不正确, 请重新下载源码

```
test@test:~$ md5sum -c h5_v1.1.tar.bz2.md5sum
```

h5_v1.1.tar.bz2: 确定

7) 然后解压 android sdk 的源码, sdk 解压后会生成 android 和 lichee 两个文件夹

- a. android: 存放 android 相关的代码
- b. linchee: 存放 linux 内核和 u-boot 的代码

```
test@test:~$ tar -jxf h5_v1.1.tar.bz2
```

```
test@test:~$ cd Android
```



```
test@test:~/Android$ ls
android  lichee
```

6.1.2. 搭建 android 编译环境

1) 安装 openjdk-7-jdk

```
test@test:~$ sudo apt install openjdk-7-jdk
```

2) 导出 java 环境变量

```
test@test:~$ export JAVA_HOME=/usr/lib/jvm/java-7-openjdk-amd64
test@test:~$ export JRE_HOME=/usr/lib/jvm/java-7-openjdk-amd64/jre
test@test:~$ export CLASSPATH=.:$JAVA_HOME/lib:$JRE_HOME/lib:$CLASSPATH
test@test:~$ export PATH=$JAVA_HOME/bin:$JRE_HOME/bin:$JAVA_HOME:$PATH
```

3) 然后通过 java -version 命令可以验证 JDK 是否安装成功

```
test@test:~$ java -version
java version "1.7.0_201"
OpenJDK Runtime Environment (IcedTea 2.6.17) (7u211-2.6.17-0ubuntu0.1)
OpenJDK 64-Bit Server VM (build 24.201-b00, mixed mode)
```

4) 安装平台支持软件

```
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get update
test@test:~$ sudo apt-get install git gnupg flex bison gperf build-essential \
zip curl zlib1g-dev gcc-multilib g++-multilib libc6-dev-i386 \
lib32ncurses5-dev x11proto-core-dev libx11-dev lib32z1-dev ccache \
libgl1-mesa-dev libxml2-utils xsltproc unzip
```

6.1.3. 编译 android 镜像

6.1.3.1. 编译 linux 内核源码

1) 第一次编译内核时需要配置编译环境，配置完后会自动开始编译内核代码



```
test@test:~/Android$ cd lichee
test@test:~/Android/lichee$ ./build.sh config
```

Welcome to mkscript setup progress

All available chips:

- 0. sun50iw1p1
- 1. sun50iw2p1
- 2. sun8iw11p1
- 3. sun8iw6p1
- 4. sun8iw7p1
- 5. sun8iw8p1
- 6. sun9iw1p1

Choice: **1**

All available platforms:

- 0. android
- 1. dragonboard
- 2. linux
- 3. eyesee linux

Choice: **0**

2) 编译完后会输出下面的信息

```
sun50iw2p1 compile Kernel successful
```

```
INFO: build kernel OK.
```

```
INFO: build rootfs ...
```

```
INFO: skip make rootfs for android
```

```
INFO: build rootfs OK.
```

```
-----
build sun50iw2p1 android  lichee OK
-----
```

3) 后面如果重新编译内核代码只需要输入 **./build.sh** 命令即可开始编译

```
test@test:~/Android/lichee$ ./build.sh
```



6.1.3.2. 编译 android 源码

1) 编译 android 的命令如下

```
test@test:~/Android$ cd android
test@test:~/Android/android$ source build/envsetup.sh
test@test:~/Android/android$ lunch cheetah_fvd_p1-eng
test@test:~/Android/android$ extract-bsp
test@test:~/Android/android$ make -j8
```

2) 打包生成 android 镜像最后的输出 log 如下所示

```
test@test:~/Android/android$ pack
.....
-----image is at-----

Android/lichee/tools/pack/sun50iw2p1_android_cheetah-p1_uart0.img

pack finish
```

3) 生成的安卓镜像存放的路径为

```
lichee/tools/pack/sun50iw2p1_android_cheetah-p1_uart0.img
```